

Laravel

V8 框架中文文档

编 写 人：	翻译
日 期：	2025-11-25
版 本：	V1.0.1

修改历史

日期	版本	作者	修改内容
2025-11-25	1.0.1.01	翻译	初始版本

目录

第一章 序言	38
1.1. 发行说明	38
1.1.1. 版本化方案	38
1.1.2. 支持策略	38
1.1.3. Laravel 8	39
1.1.3.1. Laravel Jetstream	39
1.1.3.2. 模型类目录	39
1.1.3.3. Blade 组件标签和改进	39
1.1.3.4. 流畅的字符串操作	39
1.1.3.5. 自定义键名	40
1.1.3.6. 自动作用域	40
1.1.3.7. 多邮件驱动程序	40
1.1.3.8. 查询时间类型转换	41
1.1.3.9. MySQL 8+ 数据库队列改进	42
1.2. 升级说明	42
1.2.1. 升级指南	42
1.2.1.1. 从 6.x 升级到 7.0	42
1.2.2. 高影响变化	43
1.2.2.1. 身份认证脚手架	43
1.2.2.2. 日期序列化	43
1.2.2.3. Symfony 5 相关升级	43
1.2.3. 中影响变化	43

1.2.3.1. Blade 组件 & Blade X.....	43
1.2.3.2. CORS 支持	44
1.2.3.3. 工厂类型	44
1.2.3.4. Markdown 邮件模板更新	44
1.3. 贡献导引	44
1.3.1. Bug 报告	44
1.3.2. 支持问题	45
1.3.3. 核心发展讨论	46
1.3.4. 哪个分支	46
1.3.5. 编译资源	46
1.3.6. 安全漏洞	46
1.3.7. 编码风格	46
1.3.8. 行为准则	46
第二章 快速入门	47
2.1. 安装	47
2.1.1. 安装	47
2.1.1.1. 服务器要求	47
2.1.1.2. 安装 Laravel	47
2.1.1.3. 配置	48
2.1.2. Web 服务器配置	48
2.1.2.1. 目录配置	48
2.1.2.2. 优雅链接	48
2.2. 配置	49
2.2.1. 介绍	49

2.2.2. 环境配置	49
2.2.2.1. 环境变量类型	49
2.2.2.2. 检索环境配置	50
2.2.2.3. 确定当前环境	50
2.2.2.4. 在调试页面隐藏环境变量	50
2.2.3. 访问配置值	50
2.2.4. 配置缓存	50
2.2.5. 维护模式	51
2.3. 目录结构	51
2.3.1. 简介	51
2.3.2. 根目录	51
2.3.2.1. app 目录	51
2.3.2.2. 启动目录	51
2.3.2.3. 配置目录	51
2.3.2.4. 数据库目录	51
2.3.2.5. Public 公开目录	52
2.3.2.6. 资源目录	52
2.3.2.7. 路由目录	52
2.3.2.8. 文件存储目录	52
2.3.2.9. 测试目录	52
2.3.2.10. Vendor 目录	52
2.3.3. 应用目录	52
2.3.3.1. Broadcasting 目录	52
2.3.3.2. Console 目录	52

2.3.3.3. Events 目录	53
2.3.3.4. Exceptions 目录	53
2.3.3.5. Http 目录	53
2.3.3.6. Job 目录	53
2.3.3.7. Listeners 目录	53
2.3.3.8. Mail 目录	53
2.3.3.9. Notifications 目录	53
2.3.3.10. Policies 目录	53
2.3.3.11. Providers 目录	53
2.3.3.12. Rules 目录	53
2.4. 部署	54
2.4.1. 简介	54
2.4.2. 服务器配置	54
2.4.2.1. Nginx	54
2.4.3. 改进	54
2.4.3.1. 自动加载改进	54
第三章 核心架构	54
3.1. 生命周期	54
3.1.1. 简介	54
3.1.2. 生命周期概览	54
3.1.3. 聚焦服务提供者	56
3.2. 服务容器	56
3.2.1. 简介	56
3.2.2. 绑定	56

3.2.2.1. 绑定基础	56
3.2.2.2. 绑定接口到实现	57
3.2.2.3. 上下文绑定	58
3.2.2.4. 标记	58
3.2.2.5. 扩展绑定	59
3.2.3. 解析	60
3.2.3.1. Make 方法	60
3.2.3.2. 自动注入	60
3.2.4. 容器事件	60
3.3. 服务提供者	61
3.3.1. 简介	61
3.3.2. 编写服务提供者	61
3.3.2.1. Register 方法	61
3.3.2.2. boot 方法	62
3.3.3. 注册服务提供者	63
3.3.4. 延迟加载服务提供者	64
3.4. 门面(Facades)	64
3.4.1. 简介	64
3.4.2. 何时使用门面	65
3.4.2.1. 门面 vs.依赖注入	65
3.4.2.2. 门面 vs.辅助函数	65
3.4.3. 门面工作原理	66
3.4.4. 实时门面	66
3.4.5. 门面类列表	66

3.5. 契约 (Contracts)	67
3.5.1. 简介	67
3.5.1.1. 契约 Vs. 门面	67
3.5.2. 何时使用契约	68
3.5.3. 如何使用契约	68
3.5.4. 契约列表	68
第四章 基本概念	69
4.1. 路由	69
4.1.1. 路由入门	69
4.1.1.1. 路由重定向	69
4.1.1.2. 路由视图	70
4.1.2. 路由参数	70
4.1.2.1. 必选参数	70
4.1.2.2. 可选参数	70
4.1.2.3. 正则约束	70
4.1.2.4. 命名路由	71
4.1.3. 路由分组	72
4.1.3.1. 中间件	72
4.1.3.2. 命名空间	72
4.1.3.3. 子域名路由	73
4.1.3.4. 路由前缀	73
4.1.3.5. 路由名称前缀	73
4.1.4. 路由模型绑定	74
4.1.4.1. 隐式绑定	74

4.1.4.2. 显式绑定	74
4.1.5. 兜底路由	75
4.1.6. 限流	75
4.1.7. 表单方法伪造	75
4.1.8. 访问当前路由	75
4.1.9. 跨域资源共享(CORS)	76
4.2. 中间件	76
4.2.1. 简介	76
4.2.2. 定义中间件	76
4.2.2.1. 请求之前/之后的中间件	76
4.2.3. 注册中间件	76
4.2.3.1. 全局中间件	76
4.2.3.2. 指定路由中间件	77
4.2.3.3. 中间件组	77
4.2.4. 中间件排序	77
4.2.5. 中间件参数	77
4.2.6. 终端中间件	77
4.3. CSRF 保护	77
4.3.1. 简介	77
4.3.2. 排除指定 URL 不做 CSRF 安全校验	78
4.4. 控制器	78
4.4.1. 简介	78
4.4.2. 控制器入门	78
4.4.2.1. 定义控制器	78

4.4.2.2. 命名空间	79
4.4.2.3. 单一动作控制器	79
4.4.3. 控制器中间件	80
4.4.4. 资源控制器	80
4.4.4.1. 部分资源路由	80
4.4.4.2. 嵌套资源	81
4.4.4.3. 命名资源路由	81
4.4.4.4. 命名资源路由参数	81
4.4.4.5. 自定义资源 URIs	81
4.4.4.6. 补充资源控制器	81
4.4.5. 依赖注入&控制器	81
4.4.5.1. 构造函数注入	81
4.4.5.2. 方法注入	82
4.4.6. 路由缓存	82
4.5. HTTP 请求	82
4.5.1. 接收请求	82
4.5.1.1. 请求路径 & 方法	82
4.5.1.2. PSR-7 请求 Requests	82
4.5.2. 输入预处理&规范化	82
4.5.3. 接收数据	83
4.5.3.1. 上一次请求输入	83
4.5.3.2. Cookies	83
4.5.4. 文件上传	83
4.5.4.1. 接收上传文件	83

4.5.4.2. 存储上传文件	83
4.6. HTTP 响应	84
4.6.1. 创建响应	84
4.6.1.1. 添加响应头	84
4.6.1.2. 添加 Cookies 到响应	85
4.6.1.3. Cookies & 加密	85
4.6.2. 重定向	85
4.6.2.1. 重定向到命名路由	85
4.6.2.2. 重定向到控制器行为	85
4.6.2.3. 重定向到外部域名	85
4.6.2.4. 重定向并使用闪存的 Session 数据	86
4.6.3. 其他响应类型	86
4.6.3.1. 视图响应	86
4.6.3.2. JSON 响应	86
4.6.3.3. 文件下载	86
4.6.3.4. 文件响应	86
4.6.4. 响应宏	86
4.7. 视图	87
4.7.1. 创建视图	87
4.7.2. 传递数据到视图	87
4.7.3. 视图 Composer	87
4.8. URL 生成	87
4.8.1. 简介	87
4.8.2. 快速入门	87

4.8.2.1. 生成基础 URL	87
4.8.2.2. 访问当前 URL	87
4.8.3. 命名路由 URL	88
4.8.4. 控制器行为 URL	88
4.8.5. 默认值	88
4.9. Session	89
4.9.1. 简介	89
4.9.1.1. 配置	89
4.9.1.2. 驱动程序先决条件	89
4.9.2. 使用 Session	90
4.9.2.1. 获取数据	90
4.9.2.2. 存储数据	90
4.9.2.3. 闪存数据	90
4.9.2.4. 删除数据	91
4.9.2.5. 重新生成 SessionID	91
4.9.3. 添加自定义 Session 驱动	91
4.9.3.1. 实现驱动	91
4.9.3.2. 注册驱动	92
4.10. 验证	92
4.10.1. 简介	92
4.10.2. 快速入门	92
4.10.2.1. 定义路由	92
4.10.2.2. 创建控制器	93
4.10.2.3. 编写验证逻辑	94

4.10.2.4. 显示验证错误信息	95
4.10.2.5. 可选字段注意事项	95
4.10.3. 表单请求验证	96
4.10.3.1. 创建表单请求验证	96
4.10.3.2. 表单请求认证授权	97
4.10.3.3. 自定义错误消息	97
4.10.3.4. 自定义验证属性	97
4.10.3.5. 准备验证接入	98
4.10.4. 手动创建验证器	99
4.10.4.1. 自动重定向	99
4.10.4.2. 命名错误包	100
4.10.4.3. 验证后钩子	100
4.10.5. 错误信息处理	101
4.10.5.1. 自定义验证错误信息	101
4.10.6. 可用验证规则	101
4.10.7. 按条件添加条件规则	101
4.10.8. 验证数组	101
4.10.9. 自定义验证规则	102
4.10.9.1. 使用验证规则对象	102
4.10.9.2. 使用闭包	102
4.10.9.3. 使用扩展	102
4.10.9.4. 隐式扩展	102
4.11. 错误处理	102
4.11.1. 简介	102

4.11.2. 配置	103
4.11.3. 异常处理器	103
4.11.3.1. Report 方法	103
4.11.3.2. Render 方法	104
4.11.3.3. Reportable & Renderable 异常	105
4.11.4. HTTP 异常	105
4.11.4.1. 自定义 HTTP 错误页面	106
4.12. 日志	106
4.12.1. 简介	106
4.12.2. 配置	106
4.12.2.1. 构建日志推栈	106
4.12.3. 写入日志信息	107
4.12.3.1. 写入指定的通道	108
4.12.4. 先进的 Monolog 日志通道定制	108
4.12.4.1. 高级 Monolog 通道自定义	108
4.12.4.2. 创建 Monolog 处理器通道	109
第五章 前端开发	109
5.1. Blade 模板引擎	109
5.1.1. 简介	109
5.1.2. 模板继承	109
5.1.2.1. 定义布局	109
5.1.2.2. 继承布局	110
5.1.3. 显示数据	110
5.1.3.1. Blade & JavaScript 框架	110

5.1.4. 控制结构	110
5.1.4.1. if 语句	110
5.1.4.2. Switch 语句	110
5.1.4.3. 循环	110
5.2. 本地化	111
5.2.1. 简介	111
5.2.1.1. 区域设置	111
5.2.2. 定义翻译字符串	111
5.2.2.1. 如何使用短键	111
5.2.2.2. 如何使用翻译字符串作为键	111
5.2.3. 检索翻译字符串	111
5.2.3.1. 替换翻译字符串中的参数	111
5.2.3.2. 复数	111
5.2.4. 重写扩展包的语言文件	112
5.3. 前端脚手架	112
5.3.1.1. 简介	112
5.3.1.2. 编写 CSS	113
5.3.1.3. 编写 JavaScript	113
5.3.1.3.1. 编写 Vue 组件	113
5.3.1.3.2. 编写 React 组件	113
5.3.1.4. 添加预设命令	113
5.4. 编译资源(Mix)	113
5.4.1.1. 简介	113
5.4.1.2. 安装 & 设置	114

5.4.1.3. 运行 Mix	114
5.4.1.4. 使用样式表	114
5.4.1.4.1. Less	114
5.4.1.4.2. Sass	114
5.4.1.5. 使用 JavaScript	114
5.4.1.5.1. Vendor 提取	115
5.4.1.6. 复制文件 & 目录	115
5.4.1.7. 版本控制/缓存清除	115
第六章 安全相关	115
6.1. 用户认证	115
6.1.1. 简介	115
6.1.1.1. 数据库注意事项	116
6.1.2. 快速入门	116
6.1.2.1. 路由	116
6.1.2.2. 视图	116
6.1.2.3. 认证	117
6.1.2.4. 获取已验证用户	117
6.1.2.5. 路由保护	117
6.1.2.6. 确认密码	117
6.1.2.7. 登录限流	118
6.1.3. 手动验证用户	118
6.1.3.1. 记住用户	118
6.1.3.2. 其他认证方法	118
6.1.4. HTTP 基本认证	118

6.1.4.1. 无状态 HTTP 基本认证	118
6.1.5. 退出	119
6.1.5.1. 其他设备的会话无效	119
6.1.6. 添加自定义保护	119
6.1.6.1. 关闭请求保护	119
6.1.7. 添加自定义用户提供者	119
6.1.7.1. 用户提供程序契约	119
6.1.7.2. 可认证的契约	119
6.1.8. 事件	120
6.2. 用户授权	120
6.2.1. 简介	120
6.2.2. Gates	120
6.2.2.1. 编写 Gates	120
6.2.2.2. 授权动作	121
6.2.2.3. Gate 响应	122
6.2.2.4. Gate 拦截检查	122
6.2.3. 创建策略	123
6.2.3.1. 生成策略	123
6.2.3.2. 注册策略	123
6.2.4. 编写策略	123
6.2.4.1. 策略方法	123
6.2.4.2. 策略响应	123
6.2.4.3. 不包含模型方法	123
6.2.4.4. Guest 用户	123

6.2.4.5. 策略过滤器	123
6.2.5. 使用策略授权动作	124
6.2.5.1. 通过用户模型	124
6.3. 邮箱验证	124
6.3.1. 简介	124
6.4. 加密解密	124
6.4.1. 简介	124
6.4.2. 设置	124
6.4.3. 基本用法	124
6.5. 哈希	126
6.5.1. 简介	126
6.5.2. 配置	126
6.5.3. 基本用法	126
6.6. 密码重置	128
6.6.1. 简介	128
6.6.2. 数据库注意事项	128
6.6.3. 路由	128
6.6.4. 视图	128
6.6.5. 重置密码	129
6.6.6. 自定义	129
第七章 进阶系列	129
7.1. Artisan 控制台	129
7.1.1. 简介	129
7.1.1.1. Tinker 命令(REPL)	129

7.1.2. 编写命令	130
7.1.2.1. 生成命令	130
7.1.2.2. 命令结构	130
7.1.2.3. 闭包命令	130
7.1.3. 定义期望输入	131
7.1.3.1. 参数	131
7.1.3.2. 选项	132
7.1.3.3. 输入数组	132
7.1.3.4. 输入说明	132
7.1.4. 命令 I/O	132
7.1.4.1. 检索输入	132
7.1.4.2. 交互式输入	133
7.1.4.3. 编写输出	133
7.1.5. 注册命令	133
7.1.6. 通过代码调用命令	133
7.2. 广播	133
7.2.1. 简介	133
7.2.1.1. 配置	134
7.2.1.2. 对驱动的要求	134
7.2.2. 概念概览	134
7.2.2.1. 使用示例程序	134
7.2.3. 定义广播事件	135
7.2.3.1. 广播名称	135
7.2.3.2. 广播数据	135

7.2.3.3. 广播队列	136
7.2.3.4. 广播条件	137
7.2.4. 授权频道	137
7.2.4.1. 定义授权路由	137
7.2.4.2. 定义授权回调	137
7.2.4.3. 定义频道类	138
7.2.5. 广播事件	138
7.2.5.1. 只广播给他人	138
7.2.6. 接收广播	138
7.2.6.1. 安装 Laravel Echo	138
7.2.6.2. 对事件进行监听	138
7.2.6.3. 退出频道	139
7.2.6.4. 命令空间	139
7.2.7. Presence 频道	139
7.2.7.1. 授权 Presence 频道	140
7.2.7.2. 加入 Presence 频道	140
7.2.7.3. 广播到 Presence 频道	140
7.2.8. 客户端事件	140
7.2.9. 消息通知	140
7.3. 缓存	141
7.3.1. 配置	141
7.3.1.1. 驱动的前提条件	141
7.3.2. 缓存使用	142
7.3.2.1. 获取缓存实例	142

7.3.2.2. 从缓存中获取数据	142
7.3.2.3. 在缓存中存储数据	142
7.3.2.4. 从缓存中移除数据	143
7.3.2.5. 原子锁	143
7.3.2.6. 缓存辅助函数	143
7.3.3. 缓存标签	144
7.3.3.1. 存储被打上标签的缓存数据	144
7.3.3.2. 访问被打上标签的缓存数据	144
7.3.3.3. 移除被打上标签的缓存数据	144
7.3.4. 添加自定义缓存驱动	145
7.3.4.1. 编写驱动	145
7.3.4.2. 注册驱动	146
7.3.5. 缓存事件	147
7.4. 集合	148
7.4.1. 简介	148
7.4.1.1. 创建集合	149
7.4.1.2. 扩展集合	149
7.4.2. 集合方法	149
7.4.3. 方法列表	149
7.4.4. 高阶消息传递	149
7.4.5. 懒集合	149
7.5. 事件	150
7.5.1. 简介	150
7.5.2. 注册事件/监听器	150

7.5.2.1. 生成事件和监听器	151
7.5.2.2. 手动注册事件	151
7.5.2.3. 事件发现	152
7.5.3. 定义事件	152
7.5.4. 定义监听器	153
7.5.5. 事件监听器队列	153
7.5.5.1. 手动访问队列	153
7.5.5.2. 处理失败任务	153
7.5.6. 分发事件	154
7.5.7. 事件订阅者	154
7.5.7.1. 编写事件订阅者	154
7.5.7.2. 注册事件订阅者	154
7.6. 文件存储	154
7.6.1. 简介	154
7.6.2. 配置	154
7.6.2.1. 公共磁盘	154
7.6.2.2. 本地驱动	155
7.6.2.3. 驱动程序先决条件	155
7.6.2.4. 缓存	155
7.6.3. 获取磁盘实例	155
7.6.4. 检索文件	155
7.6.4.1. 下载文件	155
7.6.4.2. 文件 URLs	156
7.6.4.3. 文件元数据	156

7.6.5. 存储文件	156
7.6.5.1. 文件上传	156
7.6.5.2. 文件可见性	158
7.6.6. 删除文件	158
7.6.7. 目录	158
7.6.8. 自定义文件系统	159
7.7. 辅助函数	159
7.7.1. 简介	159
7.7.2. 方法列表	159
7.8. HTTP 客户端	159
7.8.1. 简介	159
7.8.2. 创建请求	159
7.8.2.1. 请求数据	160
7.8.2.2. 请求头	160
7.8.2.3. 认证	161
7.8.2.4. 重试	161
7.8.2.5. 错误处理	161
7.8.3. 测试	162
7.8.3.1. 虚拟响应	162
7.8.3.2. 检查请求	162
7.9. 邮件	163
7.9.1. 简介	163
7.9.1.1. 配置	163
7.9.1.2. 驱动前提	163

7.9.2. 生成可邮寄类	163
7.9.3. 编写可邮寄类	163
7.9.3.1. 配置发件人	163
7.9.3.2. 配置视图	164
7.9.3.3. 视图数据	164
7.9.3.4. 附件	164
7.9.3.5. 内部附件	164
7.9.3.6. 自定义 SwiftMailer 消息	164
7.9.4. Markdonw 可邮寄类	164
7.9.5. 发送邮件	164
7.9.6. 渲染可邮寄类	164
7.9.7. 本地化可邮寄类	165
7.9.8. 邮件&本地开发	165
7.9.9. 事件	165
7.10. 通知	166
7.10.1. 简介	166
7.10.2. 创建通知	166
7.10.3. 发送通知	166
7.10.3.1. 使用 Notifiable Trait	166
7.10.3.2. 使用 Notification Facade	167
7.10.4. 邮件通知	167
7.10.4.1. 格式化邮件消息	167
7.10.4.2. 自定义发送者	168
7.10.4.3. 自定义接收者	169

7.10.4.4. 自定义主题	170
7.10.5. Markdown 邮件通知	170
7.10.5.1. 生成消息	171
7.10.5.2. 编写消息	171
7.10.5.3. 自定义组件	172
7.10.6. 数据库通知	172
7.10.6.1. 先决条件	172
7.10.6.2. 格式化数据库通知	172
7.10.6.3. 访问通知	173
7.10.6.4. 标记通知已读	173
7.10.7. 广播通知	174
7.10.7.1. 先决条件	174
7.10.7.2. 格式化广播通知	174
7.10.7.3. 监听通知	174
7.10.8. 短信	175
7.10.8.1. 先决条件	175
7.10.8.2. 格式化短信通知	175
7.10.8.3. 格式化简码通知	176
7.10.9. Slack 通知	176
7.10.9.1. 先决条件	176
7.10.9.2. 格式化 Slack 通知	177
7.10.9.3. Slack 附件	177
7.10.10. 本地化通知	177
7.10.10.1. 用户首先语言区域设置	177

7.10.11. 通知事件	177
7.10.12. 自定义通道	178
7.11. 扩展包开发	179
7.11.1. 简介	179
7.11.1.1. Facades 注解	180
7.11.2. 包自动发现	180
7.11.2.1. 选择性的发现扩展包	180
7.11.3. 服务提供者	181
7.11.4. 资源	182
7.11.4.1. 配置	182
7.11.4.2. 路由	182
7.11.4.3. 迁移	183
7.11.4.4. 语言包	184
7.11.4.5. 视图	184
7.11.4.6. 视图组件	185
7.11.5. 命令	186
7.11.6. 发布前端资源	187
7.11.7. 发布文件组	187
7.12. 队列	188
7.12.1. 简介	188
7.12.1.1. 连接 Vs 队列	188
7.12.1.2. 驱动程序说明和先决条件	189
7.12.2. 创建任务	190
7.12.2.1. 生成任务类	190

7.12.2.2. 任务类结构	190
7.12.2.3. 任务中间件	192
7.12.3. 分发任务	193
7.12.3.1. 延迟调度	195
7.12.3.2. 同步调度	196
7.12.3.3. 任务链接	197
7.12.3.4. 自定义队列和连接	198
7.12.3.5. 指定任务最大尝试次数/超时值	198
7.12.3.6. 速率限制	199
7.12.3.7. 错误处理	200
7.12.3.8. 任务批处理	200
7.12.4. 队列闭包	200
7.12.5. 运行队列处理器	201
7.12.5.1. 队列优先级	202
7.12.6. Supervisor 配置	202
7.12.7. 处理失败任务	203
7.12.7.1. 任务失败后的清理工作	203
7.13. 任务调度	203
7.13.1. 简介	203
7.13.1.1. 启动调度器	203
7.13.2. 定义调度	204
7.13.2.1. 调度 Artisan 命令	205
7.13.2.2. 调度队列任务	206
7.13.2.3. 调度 Shell 命令	206

7.13.2.4. 调度频率设置	206
7.13.2.5. 时区	207
7.13.2.6. 避免任务重叠	207
7.13.2.7. 在单台服务器上运行任务	207
7.13.2.8. 后台任务	208
7.13.2.9. 维护模式	208
7.13.3. 任务输出	208
7.13.4. 任务钩子	209
第八章 数据库操作	210
8.1. 快速入门	210
8.1.1. 简介	210
8.1.1.1. 配置	210
8.1.1.2. 读写分离	211
8.1.1.3. 使用多个数据库连接	212
8.1.2. 运行原生 SQL 查询	212
8.1.3. 监听查询事件	213
8.1.4. 数据库事务	215
8.1.4.1. 处理死锁	215
8.1.4.2. 手动处理事务	216
8.1.5. 连接到数据库 CLI	216
8.2. 查询构建器	216
8.2.1. 简介	216
8.2.2. 获取结果集	217
8.2.2.1. 分块结果	218

8.2.2.2. 聚合	218
8.2.3. 查询 (Select)	219
8.2.4. 原生表达式	220
8.2.4.1. 原生方法	220
8.2.4.1.1. selectRaw	220
8.2.5. 连接 (Join)	220
8.2.5.1. 内连接	220
8.2.5.2. 左连接/右连接	221
8.2.5.3. 交叉连接	221
8.2.5.4. 高级连接语句	222
8.2.5.5. 子查询连接	222
8.2.6. 联合 (Union)	223
8.2.7. Where 子句	223
8.2.7.1. 参数分组	223
8.2.7.2. Where exists 子句	224
8.2.7.3. 子查询 Where 子句	225
8.2.7.4. JSON Where 子句	225
8.2.8. 排序、分组	226
8.2.8.1. orderBy	226
8.2.9. 条件子句	226
8.2.10. 插入	227
8.2.11. 更新	228
8.2.11.1. 更新或新增	228
8.2.11.2. 更新 JSON	229

8.2.11.3. 自增自减	229
8.2.12. 删除	230
8.2.13. 悲观锁	230
8.2.14. 调试	230
8.3. 分页	231
8.3.1. 简介	231
8.3.2. 基本用法	231
8.3.2.1. 查询构造器分页	231
8.3.2.2. Eloquent 分页	232
8.3.2.3. 手动创建分页	233
8.3.3. 显示分页视图	233
8.3.3.1. 将结果转换为 JSON	233
8.3.4. 自定义分页视图	233
8.3.4.1. 使用 Bootstrap	234
8.3.5. 分页器实例方法	234
8.4. 迁移	234
8.4.1. 简介	234
8.4.2. 生成迁移	235
8.4.3. 迁移结构	235
8.4.4. 运行迁移	237
8.4.4.1. 回滚迁移	237
8.4.5. 数据表	237
8.4.5.1. 创建数据表	237
8.4.5.2. 重命名/删除数据表	237

8.4.6. 字段	237
8.4.6.1. 创建字段	237
8.4.6.2. 修饰字段	238
8.4.6.3. 修改字段	238
8.4.6.4. 删除字段	238
8.4.7. 索引	239
8.4.7.1. 创建索引	239
8.4.7.2. 重命名索引	239
8.4.7.3. 删除索引	239
8.4.7.4. 外键索引	239
8.5. 数据填充	240
8.5.1. 简介	240
8.5.2. 编写 Seeders	240
8.5.2.1. 使用模型工厂	240
8.5.2.2. 调用其他 Seeders	241
8.5.3. 运行 Seeders	242
8.6. Redis	242
8.6.1. 简介	242
8.6.1.1. 配置	242
8.6.1.2. Predis	243
8.6.1.3. PhpRedis	244
8.6.2. 与 Redis 交互	244
8.6.2.1. 管道命令	244
8.6.3. 发布/订阅	245

第九章 Eloquent ORM	247
9.1. 快速入门	247
9.1.1. 简介	247
9.1.2. 定义模型	247
9.1.2.1. Eloquent 模型约定	247
9.1.2.2. 默认属性值	251
9.1.3. 获取模型	252
9.1.3.1. 集合	252
9.1.3.2. 组块结果集	253
9.1.3.3. 高级子查询	253
9.1.4. 获取单个模型/聚合结果	254
9.1.4.1. 获取聚合结果	254
9.1.5. 插入/更新模型	254
9.1.5.1. 插入	254
9.1.5.2. 更新	255
9.1.5.3. 批量赋值	256
9.1.5.4. 其他创建方法	256
9.1.6. 删除模型	256
9.1.6.1. 软删除	256
9.1.6.2. 查询软删除模型	257
9.1.7. 复制模型	257
9.1.8. 查询作用域	258
9.1.8.1. 全局作用域	258
9.1.8.2. 本地作用域	258

9.1.9. 比较模型	258
9.1.10. 事件	258
9.1.10.1. 使用闭包	259
9.1.10.2. 观察者	260
9.1.10.3. 禁用事件	260
9.2. 关联关系	260
9.2.1. 简介	260
9.2.2. 定义关联关系	261
9.2.2.1. 一对一	261
9.2.2.2. 一对多	262
9.2.2.3. 一对多(反向)	263
9.2.2.4. 多对多	264
9.2.3. 多态关系	264
9.2.3.1. 一对一(多态)	264
9.2.3.2. 一对多(多态)	264
9.2.3.3. 多对多(多态)	264
9.2.4. 关联查询	264
9.2.4.1. 关联方法 Vs 动态属性	264
9.2.4.2. 查询已存在的关联	265
9.2.4.3. 查询不存在的关联	265
9.3. 集合	265
9.3.1. 简介	265
9.3.2. 可用方法	266
9.3.3. 自定义集合	266

9.4. 访问器和修改器	267
9.4.1. 简介	267
9.4.2. 访问器&修改器	267
9.4.2.1. 定义一个访问器	267
9.4.2.2. 定义一个修改器	268
9.4.3. 日期修改器	270
9.4.4. 属性类型转换	270
9.4.4.1. 自定义类型转换	271
9.4.4.2. 数组&JSON 转换	271
9.4.4.3. Date 类型转换	271
9.4.4.4. 查询时类型转换	271
9.5. API 资源类	271
9.5.1. 简介	271
9.5.2. 生成资源	272
9.5.3. 核心概念	272
9.5.3.1. 资源集合	272
9.5.4. 编写资源	272
9.5.4.1. 数据包裹	272
9.5.4.2. 包裹嵌套资源	273
9.5.4.3. 数据包裹和分页	274
9.5.4.4. 分页	274
9.5.4.5. 条件属性	275
9.5.4.6. 条件关联	276
9.5.4.7. 添加元数据	276

9.5.5. 资源响应	276
9.6. 序列化	277
9.6.1. 简介	277
9.6.2. 序列化模型&集合	277
9.6.2.1. 序列化为数组	277
9.6.2.2. 序列化为 JSON	277
9.6.3. 在 JSON 中隐藏属性	277
9.6.4. 追加值到 JSON	278
9.6.5. 日期序列化	279
第十章 测试系列	280
10.1. 快速入门	280
10.1.1. 简介	280
10.1.2. 环境	280
10.1.3. 创建并运行测试	281
10.1.3.1. Artisan 测试运行器	281
10.2. HTTP 测试	281
10.2.1. 简介	281
10.2.1.1. 自定义请求头	282
10.2.1.2. 测试响应	283
10.2.2. Session/认证	285
10.2.3. 测试 JSON APIs	285
10.2.3.1. 验证 JSON 完全匹配	286
10.2.3.2. 验证 JSON 路径	287
10.2.4. 测试文件上传	288

10.3. 控制台测试	290
10.3.1. 简介	290
10.3.2. 期望输入/输出	290
10.4. 数据库测试	291
10.4.1. 简介	291
10.4.2. 生成模型工厂	291
10.4.3. 每次测试后重置数据库	291
10.4.4. 编写模型工厂	292
10.4.4.1. 工厂扩展	292
10.4.4.2. 工厂状态	293
10.4.4.3. 工厂回调	293
10.5. 模拟器	294
10.5.1. 简介	294
10.5.2. 模拟对象	294
10.5.3. 任务模拟	295
10.5.4. 事件模拟	295
10.5.5. 邮件模拟	295
10.5.6. 通知模拟	295
第十一章 附录：官方扩展包	295
11.1. Horizon 队列管理	295
11.1.1. 简介	295
11.1.2. 安装	296
11.1.2.1. 配置	296
11.1.2.2. 控制面板授权	296

11.1.3. 升级 Horizon.....	296
11.1.4. 运行 Horizon.....	296
11.1.4.1. 部署 Horizon.....	296
11.1.5. 标记.....	297
11.1.6. 通知.....	297
11.1.7. 指标.....	297
11.2. Sanctum 认证.....	298
11.2.1. 简介.....	298
11.2.2. 安装.....	298
11.2.3. API 令牌认证.....	298
11.2.3.1. 发行 API 令牌.....	298
11.2.3.2. 令牌能力.....	299
11.2.3.3. 保护路由.....	299
11.2.3.4. 撤销令牌.....	299
第十二章 补充说明.....	300

第一章 序言

1.1. 发行说明

原文地址：<https://laravel.com/docs/8.x/releases>

1.1.1. 版本化方案

Laravel 和它的其他第一方软件包遵循语义版本控制。主要框架版本每六个月发布一次（~2 月和~8 月），而次要和补丁版本可能每周发布一次。小版本和补丁版本不应该包含破坏性的更改。

1.1.2. 支持策略

对于 LTS 版本，例如 Laravel 6，提供了 2 年的错误修复和 3 年的安全修复。这些版本提供了最长的支持和维护窗口。对于一般的发行版本，只提供了 6 个月的错误修复和 1 年的安全修复。对于包括 Lumen 在内的所有其他版本，只有最新版本才会修复错误。此外，请查阅 Laravel 支持的数据库版本。

版本	发布时间	Bug 修复截止时间	安全修复截止时间
10	2023 年 1 月 24 日	2024 年 7 月 30 日	2025 年 1 月 28 日
9 (LTS)	2022 年 1 月 25 日	2024 年 1 月 30 日	2025 年 1 月 28 日
8	2020 年 9 月 8 日	2022 年 7 月 26 日	2023 年 1 月 4 日
7	2020 年 3 月 3 日	2020 年 9 月 3 日	2021 年 3 月 3 日
6 (LTS)	2019 年 9 月 3 日	2021 年 9 月 3 日	2022 年 9 月 3 日
5.8	2019 年 2 月 26 日	2019 年 8 月 26 日	2020 年 2 月 26 日
5.7	2018 年 9 月 4 日	2019 年 3 月 4 日	2019 年 9 月 4 日

	日		
5.6	2018 年 2 月 7 日	2018 年 8 月 7 日	2019 年 2 月 7 日
5.5 (LTS)	2017 年 8 月 30 日	2019 年 8 月 30 日	2020 年 8 月 30 日

1.1.3. Laravel 8

Laravel 8 通过引入 Laravel Jetstream，模型工厂类，迁移压缩，队列批处理改善速率限制，队列改进，动态 Blade 组件，Tailwind 分页视图，时间测试助手，artisan serve 的改进，事件监听器的改进，以及各种其他错误修复和可用性改进，对 Laravel 7.x 继续进行了改善。

1.1.3.1. Laravel Jetstream

Laravel Jetstream 为 Laravel 设计的精美的应用程序脚手架。Jetstream 为你的下一个项目提供了完美的七点，包括登录、注册、电子邮件验证、双因子认证、会话管理、通过 Laravel Jetstream 提供的 API 支持以及可选的团队管理。

1.1.3.2. 模型类目录

由于压倒性的社区需求，现在框架默认包含一个 `app/Models` 目录。

1.1.3.3. Blade 组件标签和改进

现在的一个组件能从指定的类获取数据。所有的公开属性和方法都清晰地定义在组件类里，会自动组装成组件视图。任何附加的 HTML 属性都指定于一个可以被管理的自动包含 `$attribute` 变量的组件，它是一个属性包的实例。

1.1.3.4. 流畅的字符串操作

你可能对框架已有的 `Illuminate\Support\Str` 这个类比较熟悉，它提供了各种有用的字符串操作函数。基于这些函数，现在提供了一个更加面向对象的、更加流畅的字符串操作库。你可使用 `Str::of` 方法创建一个

`Illuminate\Support\Stringable` 对象。然后可以使用该对象的各种方法去操作字符串。

```
return (string) Str::of(' Laravel Framework 6.x ')
    ->trim()
    ->replace('6.x', '7.x')
    ->slug();
```

1.1.3.5. 自定义键名

有时你可能希望使用 `id` 以外的字段来解析 Eloquent 模型。为此， 框架允许你在路由参数中指定某个字段。

```
Route::get('api/posts/{post:slug}', function (App\Post $post) { return
$post; });
```

1.1.3.6. 自动作用域

有时，当在单个路由定义中隐式绑定多个 Eloquent 模型时，您可能希望将第二个 Eloquent 模型限定为第一个 Eloquent 模型的子模型。

1.1.3.7. 多邮件驱动程序

框架允许为单个应用配置多个邮件驱动。在 `mail` 配置文件中的每个邮件驱动都拥有它们自己的配置以及自己独特的 `[transport]`，这允许你的应用使用不同的邮件来发送某些邮件。例如，你的应用可以使用 `Postmark` 发送批量邮件，使用 `Amazon SES` 发送公务邮件。

默认情况下，框架将使用 `mail` 配置文件中的 `default` 选项指定的邮件驱动作为邮件驱动。然而，你可以通过 `mailer` 方法来使用特定的邮件驱动来发送邮件。

```
Mail::mailer('postmark')
```



```
->to($request->user())

->send(new OrderShipped($order));
```

1.1.3.8. 查询时间类型转换

有时候在查询执行过程中对特定属性进行类型转换，例如需要从数据库表中获取数据的时候。

```
use App\Post;

use App\User;

$users = User::select([

    'users.*',

    'last_posted_at' => Post::selectRaw('MAX(created_at)')

    ->whereColumn('user_id', 'users.id')

])->get();
```

在该查询取到的结果集中，`last_posted_at` 属性将会是一个字符串。假如我们在执行查询时进行 `date` 类型转换将更方便。你可以通过使用 `withCasts` 方法来完成上述操作

```
$users = User::select([

    'users.*',

    'last_posted_at' => Post::selectRaw('MAX(created_at)')

    ->whereColumn('user_id', 'users.id')

])->withCasts([

    'last_posted_at' => 'date'

])->get();
```

1.1.3.9. MySQL 8+ 数据库队列改进

在先前版本中，`database` 队列的健壮性被认为无法满足生产环境的需求。但是，Laravel 7 针对使用基于 MySQL 8+ 数据库队列的应用进行了改进。通过使用 `FOR UPDATE SKIP LOCKED` 语句进行 SQL 的优化，`database` 队列驱动可以安全地用于生产环境。

1.2. 升级说明

1.2.1. 升级指南

1.2.1.1. 从 6.x 升级到 7.0

预计升级时间：15 分钟

Symfony 5 必选

影响的可能性：高

Laravel 7 将其底层 Symfony 组件升级到 5.X 系列，现在也是新的最低兼容版本。

PHP 7.2.5 必选

影响可能性：低

新的最低 PHP 版本现在是 7.2.5。

更新依赖关系

在编写器中更新以下依赖项。json 文件：

Laravel /framework to ^7.0

ununaduro /collision to ^4.1

Phpunit /Phpunit to ^8.5

Laravel /tinker to ^2.0

facade/ignition to ^2.0

1.2.2. 高影响变化

1.2.2.1. 身份认证脚手架

影响程度：高

所有认证方式的脚手架都已移至 `laravel/ui` 仓库。如果您使用的是 Laravel 的认证脚手架，则应该安装此软件的 ^2.0 发行版，并且这个包应该安装在所有的环境中。如果您之前将此包包含在应用程序编写器的需求开发部分中。Json 文件，你应该把它移到 `require` 部分：

```
composer require laravel/ui "^2.0"
```

1.2.2.2. 日期序列化

受影响可能性：高

当在 Eloquent 模型上使用 `toArray` 或 `toJson` 方法时，Laravel 7 使用了一种新的日期序列化格式。为了格式化序列化的日期，框架现在使用 Carbon 的 `toIso8601String` 方法，该方法生成一个与 ISO-8601 兼容的日期，包括时区信息和小数秒。此外，此更改还提供了与客户端日期解析库的更好支持和集成。

1.2.2.3. Symfony 5 相关升级

受影响可能性：高

Laravel 7 利用了 5.x 系列的 Symfony 组件。为了适应此升级，需要对应用程序进行一些小的更改。

1.2.3. 中影响变化

1.2.3.1. Blade 组件 & Blade X

受影响可能性：中

Laravel 7 包括对 Blade “标签组件”的第一方支持。如果你想禁用 Blade 的内置标签组件功能，你可以从 `AppServiceProvider` 的 `boot` 方法中调用

withoutComponentTags 方法:

```
use Illuminate\Support\Facades\Blade;  
  
Blade::withoutComponentTags();
```

1.2.3.2. CORS 支持

受影响可能性: 中

现在默认集成了跨域资源共享 (CORS) 支持。如果您正在使用任何第三方 CORS 库, 现在建议您使用新的 CORS 配置文件。

1.2.3.3. 工厂类型

受影响可能性: 中

Laravel 7 删除了“工厂类型”特性。该功能自 2016 年 10 月以来一直未记录。如果您仍在使用此功能, 则应该升级到工厂状态, 这样可以提供更大的灵活性。

1.2.3.4. Markdown 邮件模板更新

受影响可能性: 中

默认的 Markdown 邮件模板已经用更专业和吸引人的设计进行了刷新。此外, 未归档的促销 Markdown 邮件组件已被删除。

1.3. 贡献导引

1.3.1. Bug 报告

为了鼓励积极协作, Laravel 官方强烈鼓励你拉取请求, 而不仅仅是提交 Bug 报告。

谨记, bug 报告的创建是希望和你有同样问题的他人能够与你协作解决问题, 不要指望 bug 报告能够自动查看任何活动或者其他跳转自此来修复它。创建 bug 报告有助于帮助自己和其他人开始着手解决问题。

Laravel 源码托管在 GitHub 上, 每个项目都有一些仓库:

Laravel Application

Laravel Art

Laravel Documentation

Laravel Cashier

Laravel Envoy

Laravel Framework

Laravel Homestead

Laravel Homestead Build Scripts

Laravel Horizon

Laravel Passport

Laravel Scout

Laravel Socialite

Laravel Telescope

Laravel Website

1.3.2. 支持问题

Laravel 的 GitHub 的 issue 功能不打算提供 Laravel 的帮助或支持。相反，使用以下渠道之一。

Laracasts Forums

Laravel.io Forums

StackOverflow

Discord

Larachat

IRC

1.3.3. 核心发展讨论

你可以在 [Laravel Ideas](#) 发行委员会 提出新功能或对现有 [Laravel](#) 行为进行改进。如果你提出一项新功能，你得愿意至少完成该功能所需的一些代码。

关于 Bug，新功能以及现有功能的实现的非正式讨论在 [Laravel Discord](#) 服务器的渠道中进行。[Laravel](#) 的维护者 [Taylor Otwell](#) 通常在工作日的上午 8 点至下午 5 点（UTC-06: 00 或美国 / 芝加哥）出现在频道中，偶尔也会在其他时间出现在频道中。

1.3.4. 哪个分支

所有的错误修复都应发送到最新的稳定分支或当前的 LTS 分支。除非将其修复的漏洞仅修复在即将发布的版本中，否则永远不应该将其发送给 `master` 分支。

与当前版本完全向后兼容的次要功能可能会发送到最新的稳定分支。

主要的新功能应始终发送到包含即将发布的版本的 `master` 分支。

1.3.5. 编译资源

如果你提交了修改，将会影响编译的文件。由于它们的文件很大，维护人员实际上无法对其进行检查。所有编译的文件将由 [Laravel](#) 的维护人员生成和提交。

1.3.6. 安全漏洞

如果您在 [Laravel](#) 中发现一个安全漏洞，请发送电子邮件至 [Taylor Otwell](#) 。所有安全漏洞将得到及时解决。

1.3.7. 编码风格

[Laravel](#) 遵循 [PSR-2](#) 编码标准和 [PSR-4](#) 自动加载标准。

1.3.8. 行为准则

[Laravel](#) 行为准则源自 [Ruby](#) 行为准则。任何违反行为准则的行为都可以报告给 [Taylor Otwell](#) (taylor@laravel.com)：

参与者容忍相反的观点。

参加者必须确保其语言和行动不受人身攻击和诋毁。

在解释他人的言行时，参与者应始终保持善意。

可以认为骚扰的行为是不能容忍的。

第二章 快速入门

2.1. 安装

2.1.1. 安装

2.1.1.1. 服务器要求

需要确认自己的环境满足以下要求：

PHP $\geq$ 7.3

BCMath PHP 拓展

Ctype PHP 拓展

Fileinfo PHP 拓展

JSON PHP 拓展

Mbstring PHP 拓展

OpenSSL PHP 拓展

PDO PHP 拓展

Tokenizer PHP 拓展

XML PHP 拓展

2.1.1.2. 安装 Laravel

Laravel 使用 Composer 来管理项目依赖。因此，在使用 Laravel 之前，请确保你的机器已经安装了 Composer。

```
composer create-project --prefer-dist laravel/laravel blog
```

2.1.1.3. 配置

公共目录

安装完 Laravel 之后，你必须将 web 服务器根目录指向 public 目录。该目录下的 index.php 文件将作为所有进入应用程序的 HTTP 请求的前端控制器。

配置文件

Laravel 框架的所有配置文件都放在 config 目录中。每个选项都有注释，方便你随时查看文件并熟悉可用的选项。

目录权限

安装完 Laravel 后，你可能需要给这两个文件配置读写权限：storage 目录和 bootstrap/cache。目录应该允许 Web 服务器写入，否则 Laravel 程序将无法运行。

应用密钥

安装 Laravel 之后下一件应该做的事就是将应用程序的密钥设置为随机字符串。通过 Composer 安装的，默认已经设置好了。

如果应用程序密钥没有被设置，就不能确保你的用户会话和其他加密数据的安全！

其他配置

除了以上的配置，Laravel 几乎就不需要再配置什么了。你随时就能开发。但是，可能的话还是希望你查看 config/app.php 文件及其注释。

2.1.2. Web 服务器配置

2.1.2.1. 目录配置

Laravel 应该始终在你的 Web 服务器配置的“Web 目录”的根目录中使用。

2.1.2.2. 优雅链接

Apache

Laravel 中包含了一个 public/.htaccess 文件，通常用于在资源路径中隐藏

index.php 的前端控制器。在用 Apache 为 Laravel 提供服务之前，确保启用了 mod_rewrite 模块，这样 .htaccess 文件才能被服务器解析。

2.2. 配置

2.2.1. 介绍

Laravel 框架的所有配置文件都保存在 config 目录中。每个选项都有说明，你可随时查看这些文件并熟悉都有哪些配置选项可供你使用。

2.2.2. 环境配置

对于应用程序运行的环境来说，不同的环境有不同的配置通常是很有用的。例如，你可能希望在本地使用的缓存驱动不同于生产服务器所使用的缓存驱动。

2.2.2.1. 环境变量类型

.env 文件中的所有变量都被解析为字符串，因此创建了一些保留值以允许你从 env() 函数中返回更多类型的变量。

.env 值	env() 值
true	(bool) true
(true)	(bool) true
false	(bool) false
(false)	(bool) false
empty	(string) ""
(empty)	(string) ""
null	(null) null
(null)	(null) null

2.2.2.2. 检索环境配置

当应用程序收到请求时，`.env` 文件中列出的所有变量将被加载到 PHP 的超级全局变量 `$_ENV` 中。你可以使用 `env` 函数检索这些变量的值。事实上，如果你查看 Laravel 的配置文件，你就能注意到有数个选项已经使用了这个函数。

```
'debug' => env('APP_DEBUG', false),
```

2.2.2.3. 确定当前环境

应用程序当前所处环境是通过 `.env` 文件中的 `APP_ENV` 变量确定的。你可以通过 `App facade` 中的 `environment` 方法来访问此值。

```
$environment = App::environment();
```

2.2.2.4. 在调试页面隐藏环境变量

当一个异常未被捕获并且 `APP_DEBUG` 环境变量为 `true` 时，调试页面会显示所有的环境变量和内容。在某些情况下你可能想隐藏某些变量。你可以通过设置 `config/app.php` 配置文件中的 `debug_blacklist` 选项来完成这个操作。

2.2.3. 访问配置值

你可以轻松地在应用程序的任何位置使用全局 `config` 函数来访问配置值。配置值的访问可以使用[点]语法，这其中包含了要访问的文件和选项的名称。还可以指定默认值，如果配置选项不存在，则返回默认值。

```
$value = config('app.timezone');
```

2.2.4. 配置缓存

为了给你的应用程序提升速度，你应该使用 `Artisan` 命令 `config:cache` 将所有的配置文件缓存到单个文件中。这会把你的应用程序中所有的配置选项合并成一个单一的文件，然后框架会快速加载这个文件。

2.2.5. 维护模式

当应用程序处于维护模式时，所有对应应用程序的请求都显示为一个自定义视图。这样可以在更新或执行维护时轻松地[关闭]你的应用程序。维护模式检查包含在应用程序的默认中间件栈中。

2.3. 目录结构

2.3.1. 简介

框架默认的目录结构试图为不管是大型应用还是小型应用提供一个良好的起点。当然，您也可以按照自己的喜好重新组织应用的目录结构，只要 Composer 可以自动加载入它们要即可。

当开始使用 Laravel 时，许多开发人员都因缺少 `models` 目录而感到困惑。然而，缺少这样的目录是故意的。我们发现模型含混不清，因为不同的人对模型有不同的理解。一些开发者把应用的模型称为所有业务逻辑的总体，而另一些人将模型称为与关系数据加交互的类。

出于这个原因，我们默认把 Eloquent 的模型放在 `app` 目录下，并允许开发人员将它们放在其他地方。

2.3.2. 根目录

2.3.2.1. `app` 目录

`app` 目录包含了应的核心代码。

2.3.2.2. 启动目录

`Bootstrap` 目录包含了少许文件。`app.php` 用于框架的启动和自动载入配置，还有一个 `cache` 文件夹，里面包含了框架为提升性能所生成的文件，如路由和服务缓存文件。

2.3.2.3. 配置目录

`Config` 目录包含了应用所有的配置文件。

2.3.2.4. 数据库目录

Database 目录包含了数据填充和迁移文件以及模型工厂类。

2.3.2.5. Public 公开目录

Public 目录包含了应用入口文件 index.php 和前端资源文件。

2.3.2.6. 资源目录

Resources 目录包含了应用视图文件和未编译的原生前端资源文件，以及本地化语言文件。

2.3.2.7. 路由目录

Routes 目录包含了应用定义的所有路由。

2.3.2.8. 文件存储目录

Storage 目录包含了编译后的 blade 模板、基于文件的 Session、文件缓存，以及其他由框架生成的文件，该目录细分为 app、frameworkd 和 logs 子目录。

2.3.2.9. 测试目录

Tests 目录包含自动化测试文件，其中默认已经提供了一个开箱即用的 PHPUnit 参考。

2.3.2.10. Vendor 目录

Vendor 目录包含了应用所有通过 Composer 加载的依赖。

2.3.3. 应用目录

App 目录包含额外的各种目录，如 Console、Http、Providers 等。Console 和 Http 目录提供了进入应用核心的 API，HTTP 协议和 CLI 是和应用进行交互的两种机制，但实际上并不包含应用逻辑。换句话说，它们只是两个向应用发送命令的方式。

2.3.3.1. Broadcasting 目录

Broadcasting 目录包含应用程序的所有广播通道类。这些类使用 make:channel 命令生成。此目录默认是不存在的，但是当你创建第一个通道时它将被创建。

2.3.3.2. Console 目录

Console 目录包含应用程序的所有自定义 Artisan 命令。这些命令可以使用 `make:command` 命令生成。

2.3.3.3. Events 目录

默认情况下这个目录是不存在的。

2.3.3.4. Exceptions 目录

Exceptions 目录包含应用程序的异常处理，并且也是一个放置应用程序抛出任何异常的好地方。

2.3.3.5. Http 目录

Http 目录包含你的控制器，中间件和表单请求。处理进入应用程序请求的所有逻辑几乎都放置在此目录。

2.3.3.6. Job 目录

默认情况下这个目录是不存在的。

2.3.3.7. Listeners 目录

默认情况下这个目录是不存在的。

2.3.3.8. Mail 目录

默认情况下这个目录是不存在的。

2.3.3.9. Notifications 目录

默认情况下这个目录是不存在的。

2.3.3.10. Policies 目录

默认情况下这个目录是不存在的。

2.3.3.11. Providers 目录

Providers 目录包含应用程序的服务提供者。服务提供者通过在服务容器中绑定服务引导应用程序，注册事件或者准备为应用程序即将到来的请求执行其它任何任务。

2.3.3.12. Rules 目录

默认情况下这个目录是不存在的，但如果执行 `make:rule Artisan` 命令，它将被创建出来。

2.4. 部署

2.4.1. 简介

当您准备部署应用到生产环境时，有一些重要的事情可以确保应用尽可能高效地运行。

2.4.2. 服务器配置

2.4.2.1. Nginx

如果你想要部署你的应用到 Nginx 服务器上，你可能会用到下面这个配置文件作为一个开始来配置你的 Web 服务器。

2.4.3. 改进

2.4.3.1. 自动加载改进

当你准备往生产环境部署应用时，确保你优化了你的 `Composer` 类的自动加载映射，这样可以使 `Composer` 可以很快的找到正确的加载文件去加载给定的类。

第三章 核心架构

3.1. 生命周期

3.1.1. 简介

当我们使用现实世界中的任何工具时，如果理解了该工具的工作原理，那么用起来就会得心应手，应用开发也是如此。

如果您不能马上理解所有的术语，也不用因此失去信心！只要多一点尝试、学着如何运用，随着你浏览文档的其他部分，你的知识一开视频会因此增长。

3.1.2. 生命周期概览

第一件事

请求入口是 `public/index.php` 文件，这里是加载框架其他部分的起点。

载入 Composer 生成的自动加载设置，然后从 `bootstrap/app.php` 脚本获得 Laravel 应用实例，Laravel 的第一个动作是创建服务容器实例。

HTTP/Console 内核

接下来，请求被发送到 HTTP 内核或 Console 内核（分别用于处理 Web 请求和 Artisan 命令），这取决于进入应用的请求类型。这两个内核是所有请求都要经过的中央处理器，现在，就让我们聚焦在位于 `app/Http/Kernel.php` 的 HTTP 的内核。

HTTP 内核继承自 `Illuminate\Foundation\Http\Kernel` 类，该类定义了一个 `bootstrappers` 数组，这个数组中的类在请求被执行前运行，这些 `bootstrappers` 配置了错误处理、日志、检测应用环境以及其它在请求被处理前需要执行的任务。

HTTP 内核还定义了一系列所有请求在处理前需要经过的 HTTP 中间件，这些中间件处理 HTTP 会话的读写、判断应用是否处于维护模式、验证 CSRF 令牌等等。

HTTP 内核的 `handle` 方法签名相当简单，获取一个 `Request`，返回一个 `Response`，可以把该内核想象作一个代表整个应用的大黑盒子，输入 HTTP 请求，返回 HTTP 响应。

内核启动过程中最重要的动作之一就是为应用载入服务提供者，应用的所有服务提供者都被配置在 `config/app.php` 配置文件的 `providers` 数组中。首先，所有提供者的 `register` 方法被调用，然后，所有提供被注册之后，`boot` 方法被调用。

服务提供者负责启动框架的所有各种各样的组件，比如数据库、队列、验证器，以及路由组件等，正是因为他们启动并配置了框架提供的所有特性，所以服务提供者是整个框架启动过程中最重要的部分。

一旦应用被启动并且所有的服务提供者被注册，`Request` 将会被交给路由器进行分发，路由器将会分发请求到路由或控制器，同时运行所有路由指定的中间件。

3.1.3. 聚焦服务提供者

应用实例被创建后，服务提供者被注册，请求被交给启动后的应用进行处理，整个过程就是这么简单。

对应用如何通过服务提供者构建和启动有一个牢固的掌握非常有价值，当然，应用默认的服务提供者存放在 `app/Providers` 目录下。

默认情况下，`AppServiceProvider` 是空的，这里是添加自定义启动和服务容器绑定的最佳位置，当然，对于大型应用，您可能希望创建多个服务提供者，每一个都有着更加细粒度的启动。

3.2. 服务容器

3.2.1. 简介

服务容器是一个用于管理类依赖和执行依赖注入的强大工具。依赖注入这个名词表面看起来很花哨，其实质是通过构造，函数或者某些情况下通过 `setter` 方法将类依赖注入到类中。

3.2.2. 绑定

3.2.2.1. 绑定基础

简单的绑定

在一个服务提供者中，可以通过 `$this->app` 变量访问容器，然后使用 `bind` 方法注册一个绑定，该方法需要两个参数，第一个参数是我们想要注册的类名或接口名称，第二个参数是返回类的实例的闭包。

```
$this->app->bind('HelpSpot\API', function ($app) {  
    return new HelpSpot\API($app->make('HttpClient'));  
});
```

绑定一个单例

`Singleton` 方法绑定一个只会解析一次的类或接口到容器，然后接下来对容器的调用将会返回一个对象实例。


```
$this->app->singleton('HelpSpot\API', function ($app)
{
return new HelpSpot\API($app->make('HttpClient'));
});
```

绑定实例

你还可以使用 **instance** 方法绑定一个已存在的对象实例到容器，随后调用容器将总是返回给定的实例。

```
$api = new HelpSpot\API(new HttpClient);
$this->app->instance('HelpSpot\API', $api);
```

绑定原始值

你可能有一个接收注入类的类，同时需要注入一个原生的数值比如整型，可以结合上下文轻松注入这个类需要的任何值。

```
$this->app->when('App\Http\Controllers\UserController ')
->needs('$variableName')
```

3.2.2.2. 绑定接口到实现

服务容器有一个非常强大的功能是其绑定接口到实现。我们假设有一个 **EventPusher** 接口及其实现类 **RedisEventPusher**，编写完该接口的 **RedisEventPusher** 实现后，就可以将其注册到服务容器。

```
$this->app->bind(
'App\Contracts\EventPusher',
'App\Services\RedisEventPusher'
);
```

这段代码告诉窗口当一个类需要 **EventPusher** 的实现时会注入 **RedisEventPusher**，现在我们可以构造器或者任何其他通过服务容器注入依赖

的地方进行 EventPusher 接口的依赖注入。

3.2.2.3. 上下文绑定

有时候我们可能有两个类使用同一个接口，但我们希望在每个类中注入不同实现，例如，两个控制器依赖 `Illuminate\Contracts\Filesystem\Filesystem` 契约的不同实现。

```
use Illuminate\Support\Facades\Storage;

use App\Http\Controllers\VideoController;

use App\Http\Controllers\PhotoControllers;

use Illuminate\Contracts\Filesystem\Filesystem;

$this->app->when(PhotoController::class)

    ->needs(Filesystem::class)

    ->give(function () {

        return Storage::disk('local');

    });

$this->app->when(VideoController::class)

    ->needs(Filesystem::class)

    ->give(function () {

        return Storage::disk('s3');

    });
```

3.2.2.4. 标记

少数情况下，我们需要解析特定分类下的所有绑定，例如，你下在构建一个

接收多个不同 **Report** 接口实现的报告聚合器，在注册完 **Report** 实现之后，可以通过 **tag** 方法给它们分配一个标签。

```
$this->app->bind('SpeedReport', function () {

    //

});

$this->app->bind('MemoryReport', function () {

    //

});

$this->app->tag(['SpeedReport', 'MemoryReport'], 'reports');
```

这些服务被打上标签后，可以通过 **tagged** 方法来轻松解析它们。

```
$this->app->bind('ReportAggregator', function ($app)

{

    return new ReportAggregator($app->tagged('reports'));

});
```

3.2.2.5. 扩展绑定

extend 方法允许对解析服务进行修改。例如，当服务被解析后，可以运行额外代码装饰或配置该服务。**extend** 方法接收一个闭包来返回修改后的服务。

```
$this->app->extend(Service::class, function($service)

{

    return new DecoratedService($service);

});
```

3.2.3. 解析

3.2.3.1. Make 方法

你可以使用 `make` 方法从容器中解析出类的实例。

```
$fooBar = $this->app->make('HelpSpot\API');
```

如果你所在的代码位置访问不了 `$app` 变量，可以使用辅助函数 `resolve`。

```
$api = resolve('HelpSpot\API');
```

如果类依赖不能通过容器来解析，你可以通过关联数组方式将其传递到 `makeWith` 方法来注入。

```
$api = $this->app->makeWith('HelpSpot\API', ['id' => 1]);
```

3.2.3.2. 自动注入

另外，并且更重要的是，你可以简单的通过在类的构造函数中对依赖进行类型提示来从容器中解析对象，控制器、事件监听器、中间件等都是通过这种方式。

3.2.4. 容器事件

服务容器在每次解析对象时都会触发一个事件，你可以使用 `resolving` 方法监听该事件。

```
$this->app->resolving(function ($object, $app) {  
    // Called when container resolves object of any type  
    ...  
});  
  
$this->app->resolving(HelpSpot\API::class, function ($api, $app)  
    // Called when container resolves objects of type "  
    HelpSpot\API"...  
});
```

正如你所看到的，被解析的对象将会传递给回调函数，这使得你能够在对象被传给调用者之前给它设置额外的属性。

3.3. 服务提供者

3.3.1. 简介

服务提供者是应用启动的中心，你自己的应用以及所有框架的核心服务都是通过服务提供者启动。

但是，引导是什么意思呢？通常，我们可以理解为注册，比如注册服务容器绑定、事件监听器、中间件，甚至路由。服务提供者是应用配置的中心。

3.3.2. 编写服务提供者

所有的服务提供者都继承自 `Illuminate\Support\ServiceProvider` 类。大部分服务提供者都包含两个方法：`register` 和 `boot`。在 `register` 方法中，您唯一要做的事就是绑定服务到服务容器，不要尝试在该方法中注册事件监听器、路由或者其他任何功能。

3.3.2.1. Register 方法

如上所述，在 `register` 方法中只绑定服务到服务容器，而不要做其他事情。否则，一不小心就可能用到一个尚未被加载的服务提供者提供的服务。

```
<?php

namespace App\Providers;

use Riak\Connection;

use Illuminate\Support\ServiceProvider;

class RiakServiceProvider extends ServiceProvider{

    /**
     * 注册应用服务
    */
}
```

```
*
* @return void
*/
public function register()
{
    $this->app->singleton(Connection::class, function ($app) {
        return new Connection(config('riak'));
    });
}
```

该服务只定义了一个 **register** 方法，并且使用该方法在服务容器中定义了一个 **Riak\Connection** 的接口。

Bindings 和 singletons 属性

如果你的服务提供者注册了很多简单的绑定，你可能希望使用 **bindings** 和 **singletons** 属性来替代手动注册每个容器绑定以简化代码。

3.3.2.2. boot 方法

如果我们想要在服务提供者中注册视图 **composer** 该怎么做？这就要用到 **boot** 方法了。该方法在所有服务提供者被注册以后才会被调用，这就是说我们可以在其中访问框架已注册的所有其他服务。

```
<?php
namespace App\Providers;

use Illuminate\Support\ServiceProvider;

class ComposerServiceProvider extends ServiceProvider
```

```
{

    /**

     * Perform post-registration booting of services.

     *

     * @return void

     */

    public function boot()

    {

        view()->composer('view', function () {

            //

        });

    }

}
```

Boot 方法的依赖注入

我们可以在 `boot` 方法中对依赖进行类型提示，服务容器会自动注入你所需要的依赖。

```
use Illuminate\Contracts\Routing\ResponseFactory;

public function boot(ResponseFactory $response) {

    $response->macro('caps', function ($value) {

        //

    });

}
```

3.3.3. 注册服务提供者

所有的服务提供者都通过配置文件 `config/app.php` 中进行注册，该文件包含

了一个列出所有服务提供者名字的 `providers` 数组，默认情况下，其中列出了所有核心服务提供者，这些服务提供者启动框架核心组件，比如邮件、队列、缓存等等。

要注册你自己的服务提供者，只需要将其追加到该数组中即可。

```
'providers' => [

    // 其它服务提供者

    App\Providers\ComposerServiceProvider::class,

],
```

3.3.4. 延迟加载服务提供者

如果你的提供者仅仅只是在服务容器中注册绑定，你可以选择延迟加载该绑定直到注册绑定的服务真的需要时再加载，延迟加载这样的一个提供者将会提升应用的性能，因为它不会在每次请求时都从文件系统加载。

Laravel 编译并保存所有延迟服务提供者提供的服务及服务提供者的类名。然后，只有当你尝试解析其中某个服务时框架才会加载其服务提供者。

3.4. 门面(Facades)

3.4.1. 简介

门面为应用的服务容器提供了一个静态接口，Laravel 内置了很多门面，可以访问绝大部分功能。Laravel 的门面实际是服务容器中底层类的静态代理，相比于传统静态方法，在维护时能够提供更加易于测试、更加灵活、简明优雅的语法。

框架所有门面都定义在 `Illuminate\Support\Facades` 命名空间下，所以我们可以轻松访问到门面。

```
use Illuminate\Support\Facades\Cache;

Route::get('/cache', function () {

    return Cache::get('key');

});
```


在整个框架中，很多示例使用了门面来演示框架的各种功能特性。

3.4.2. 何时使用门面

门面有诸多优点，其提供了简单、易记的语法，让我们无需记住长长的类名即可使用框架提供的功能特性，此外，由于他们对 PHP 动态方法的独到用法，使得它们很容易测试。

但是，使用门面也有需要注意的地方，一个最主要的危险就是类范围变化。由于门面如此好用并且不需要注入，在单个类中使用过多门面会让类很容易变得越来越大，使用依赖注入则会让此类问题缓解。因此，使用门面的时候要尤其注意类的大小，以便控制其有限职责。

3.4.2.1. 门面 vs. 依赖注入

依赖注入的最大优点是可以替换注入类的实现。这在测试时很有用。因为你可以注入一个模拟并且可在存根上断言不同的方法。

但是在静态类方法上进行模拟或存根却行不通，不过，由于门面使用了动态方法对服务容器中解析出来的对象方法调用进行了代理，我们也可以像测试注入类实例那样测试门面。例如，给定以下路由。

```
use Illuminate\Support\Facades\Cache;

Route::get('/cache', function () {

    return Cache::get('key');

});
```

3.4.2.2. 门面 vs. 辅助函数

除了门面外，Laravel 还内置了许多辅助函数用于执行常用任务，比如生成视图、触发事件、分配任务，以及发送 HTTP 响应等。很多辅助函数提供了和相应门面一样的功能，例如，下面这个门面调用和辅助函数调用是等价的：

```
return View::make('profile');
```

```
return view('profile');
```

门面和辅助函数并不存在实质性的差别，使用辅助函数的时候，可以像测试相应门面那样测试它们。

3.4.3. 门面工作原理

门面就是一个可以从容器访问对象的类。其中核心的部件就是 `Facade` 类。不管是框架自带的门面，还是自定义的门面，都会继承自 `Illuminate\Support\Facades\Facade` 类。

`Facade` 基类使用了 `__callStatic()` 魔术方法，直到对象从容器中被解析出来后，才会进行调用。在下面的例子中，调用了 `Laravel` 的缓存系统

3.4.4. 实时门面

使用实时门面，你可以将应用程序中的任何类视为门面。。

3.4.5. 门面类列表

序号	门面	类	处理方式
1	App	<code>Illuminate\Foundation\Application</code>	<code>app</code>
2	Artisan	<code>Illuminate\Contracts\Console\Kernel</code>	<code>artisan</code>
3	Auth	<code>Illuminate\Auth\AuthManager</code>	<code>auth</code>
4	Auth(实例)	<code>Illuminate\Contracts\Guard</code>	<code>Auth.driver</code>
5	Blade	<code>Illuminate\View\Compilers\BladeCompiler</code>	<code>Blade.compile</code>
6	Broadcast	<code>Illuminate\Contracts\Broadcasting\Factory</code>	

7	Broadcast(实例)	Illuminate\Contracts\Broadcasting\Broadcaster	
8	Bus	Illuminate\Contracts\Bus\Dispatcher	
9	Cache	Illuminate\Cache\CacheManager	cache
10	Cache(实例)	Illuminate\Cache\Repository	cache.store
11	Config	Illuminate\Config\Repository	config
12	Cookie	Illuminate\Cookie\CookieJar	cookie
13	Crypt	Illuminate\Encryption\Encrypter	encrypter
14	DB	Illuminate\Database\DatabaseManager	db

3.5. 契约 (Contracts)

3.5.1. 简介

契约是指框架提供的一系列定义的核心服务的接口。例如，`Illuminate\Contracts\Queue\Queue` 契约定义了队列任务需要实现的方法，`Illuminate\Contracts\Mail\Mailer` 契约定义了发送邮件所需要实现的方法。

每一个契约都有框架提供的相应实现。例如，框架为队列提供了多个驱动实现，邮件则由 `SwiftMailer` 驱动实现。

3.5.1.1. 契约 Vs. 门面

门面为服务提供了便捷方式，不再需要从服务容器中类型提示和契约解析即

可直接通过静态门面调用。

不同于门面不需要再构造器中进行类型提示，契约允许您在类中定义显式的依赖。有些开发者喜欢门面带来的便捷，也有些开发者倾向于使用契约，他们喜欢定义明确的依赖。

3.5.2. 何时使用契约

综上所述，大多数情况下使用契约还是门面取决于个人或团队的喜好。只要保持类的职责单一，你会发现使用契约和门面并没有什么实质性的差别。

但是，对契约您可能还有些疑问。例如，为什么要全部使用接口？使用接口是不是更复杂？

3.5.3. 如何使用契约

要实现一个契约，需要在解析类的构造函数中类型提示这个契约接口。

3.5.4. 契约列表

下面是契约列表，以及其对应的“门面”：

契约	对应门面
<code>Illuminate\Contracts\Auth\Access\Authorizable</code>	
<code>Illuminate\Contracts\Auth\Access\Gate</code>	<code>Gate</code>
<code>Illuminate\Contracts\Auth\Authenticatable</code>	
<code>Illuminate\Contracts\Auth\CanResetPassword</code>	
<code>Illuminate\Contracts\Auth\Factory</code>	<code>Auth</code>
<code>Illuminate\Contracts\Auth\Guard</code>	<code>Auth::guard()</code>
<code>Illuminate\Contracts\Auth\PasswordBroker</code>	<code>Password::broker()</code>
<code>Illuminate\Contracts\Auth\PasswordBrokerFactory</code>	<code>Password</code>

Illuminate\Contracts\Auth\StatefulGuard	
Illuminate\Contracts\Auth\SupportsBasicAuth	

第四章 基本概念

4.1. 路由

4.1.1. 路由入门

最基本的路由只接收一个 URL 和一个闭包。

```
Route::get('foo', function () {  
    return 'Hello World';  
});
```

路由器允许你注册能响应任何 HTTP 请求的路由。

```
Route::get($uri, $callback);  
Route::post($uri, $callback);  
Route::put($uri, $callback);  
Route::patch($uri, $callback);  
Route::delete($uri, $callback);  
Route::options($uri, $callback);
```

4.1.1.1. 路由重定向

如果您需要定义一个重定向到其他的 URI 的路由，可以使用 `Route::redirect` 方法，该方法非常方便。

```
Route::redirect('/here', '/there');
```

您还可以使用 `Route::permanentRedirect` 方法来返回 301 状态码。

```
Route::permanentRedirect('/here', '/there');
```

4.1.1.2. 路由视图

如果您的路由需要返回一个视图，可以使用 `Route::view` 方法。

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => '欢迎']);
```

4.1.2. 路由参数

4.1.2.1. 必选参数

有时我们需要在路由中捕获 URI 请求参数。

```
Route::get('user/{id}', function ($id) {
    return 'User ' . $id;
});
```

4.1.2.2. 可选参数

有必选参数就有可选参数，可以通过在参数名后加一表 `?` 标记来实现。

```
Route::get('user/{name?}', function ($name = null) {
    return $name;
});

Route::get('user/{name?}', function ($name = 'John')
{
    return $name;
});
```

4.1.2.3. 正则约束

可以通过路由实例上的 `where` 方法来约束路由参数的格式。

```
Route::get('user/{name}', function ($name) {  
    //  
})->where('name', '[A-Za-z]+');  
  
Route::get('user/{id}', function ($id) {  
    //  
})->where('id', '[0-9]+');  
  
Route::get('user/{id}/{name}', function ($id, $name) {  
    //  
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

使用正则还有一个好处就是避免了 `user/{id}` 和 `user/{name}` 的混淆。

4.1.2.4. 命名路由

命名中为生成 URL 或重定向提供了方便,在路由定义之后使用 `name` 方法链的方式来定义该路由的名称。

```
Route::get('user/profile', function () {  
    // 通过路由名称生成 URL  
  
    return 'my url: ' . route('profile');  
})->name('profile');
```

还可以为控制器动作指定路由名称。

```
Route::get('user/profile', 'UserController@showProfile')->name('profile');
```

4.1.3. 路由分组

路由分组的目的是让我们在多个路由中共享相同的路由属性，比如中间件和命名空间等，这样的话我们定义了大量的路由时就不必为每一个路由单独定义属性。共享属性以数组的形式作为第一个参数被传递给 `Route::group` 方法。

嵌套的分组会尝试智能地将属性合并到父分组中，中间件和 `where` 条件会直接被合并，而路由命名、命名空间、以及路由前缀会被附加到父组件对应属性之后。

4.1.3.1. 中间件

要给某个路由分组中定义的所有路由分配中间件，可以在定义分组之前使用 `middleware` 方法。

```
Route::middleware(['first', 'second'])->group(function () {  
  
    Route::get('/', function () {  
  
        // Uses first & second Middleware  
  
    });  
  
    Route::get('user/profile', function () {  
  
        // Uses first & second Middleware  
  
    });  
  
});
```

4.1.3.2. 命名空间

路由分组另一个通用例子就是使用 `namespace` 方法分配同一个 PHP 命名空间给该分组下的多个控制器。

```
Route::namespace('Admin')->group(function () {  
  
    // Controllers Within The "App\Http\Controllers\Admin" Namespace
```



```
});
```

4.1.3.3. 子域名路由

路由分组还可以被用于处理子域名路由，子域名可以像 URI 一样被分配给路由参数，从而允许捕获子域名的部分用于路由或者控制器，子域名可以在定义分组之前调用 `domain` 方法来指定。

```
Route::domain('{account}.blog.dev')->group(function ()
{
    Route::get('user/{id}', function ($account, $id) {

        return 'This is ' . $account . ' page of User ' . $id;

    });
});
```

4.1.3.4. 路由前缀

`prefix` 方法可以用来为分组中每个路由添加一个指定 URI 前缀。

例如，您可以为分组中所有路由 URI 添加 `admin` 前缀。

```
Route::prefix('admin')->group(function () {
    Route::get('users', function () {
        // Matches The "/admin/users" URL
    });
});
```

这样，我们就可以通过 `http://blog.test/admin/users` 访问路由了。

4.1.3.5. 路由名称前缀

`name` 方法可通过传入字符串为分组中的每个路由名称设置前缀。

例如，您可能想要在所有分组路由的名称前添加 `admin` 前缀。

4.1.4. 路由模型绑定

注入模块 ID 到路由或控制器动作时，通常需要查询数据库才能获取相应的模型数据。

4.1.4.1. 隐式绑定

Laravel 会自动解析定义在路由或控制器动作中的 Eloquent 模型类型声明。

```
Route::get('users/{user}', function (App\User $user)
{
    return $user->email;
});
```

4.1.4.2. 显式绑定

要注册显式绑定，可以使用路由器的 `model` 方法来为给定参数指定绑定类。

```
public function boot()
{
    parent::boot();

    Route::model('user_model', App\User::class);
}
```

接下来，在 `routes/api.php` 中定义一个包含 `{user}` 参数的路由。

```
$router->get('profile/{user_model}', function (App\User
r $user) {
    dd($user);
});
```

由于我们已经绑定 `{user_model}` 参数到 `App\User` 模型，`User` 实例会被注入以该路由。

4.1.5. 兜底路由

使用 `Route::fallback` 方法可以定义一个当所有其他路由都未能匹配请求 URL 时所执行的路由。

4.1.6. 限流

框架自带了一个中间件用于限制对应用路由的访问频率。`Throttle` 中间件接收两个参数用于判断给定时间内（单位：分钟）的最大请求次数。

```
Route::middleware('auth:api', 'throttle:60,1')->group(
(function () {
    Route::get('/user', function () {
        //
    });
});
```

超出访问次数后，会返回 429 状态码并提示 `Too many requests.`

4.1.7. 表单方法伪造

HTML 表单不支持 PUT、PATCH 或者 DELETE 请求方法，因此，需要添加一个隐藏的 `_method` 字段。

```
<form action="/foo/bar" method="POST">
    <input type="hidden" name="_method" value="PUT">
    <input type="hidden" name="_token" value="{{ csrf_token() }}">
</form>
```

4.1.8. 访问当前路由

您可以使用 `Route` 门面上的 `current`、`currentRouteName` 和 `currentRouteAction` 方法来访问处理当前输入请求的路由信息。

```
// 获取当前路由实例

$route = Route::current();

// 获取当前路由名称

$name = Route::currentRouteName();

// 获取当前路由 action 属性

$action = Route::currentRouteAction();
```

4.1.9. 跨域资源共享(CORS)

Laravel 可以使用您配置的值自动响应 CORS OPTIONS 请求。所有 CORS 设置都可以在 `./config/cors.php` 配置文件中配置，选项请求将由默认情况下包含在全局中间件中的 `HandleCors` 中间件自动处理。

4.2. 中间件

4.2.1. 简介

中间件提供了一种方便的机制来过滤进入应用程序的 HTTP 请求。

框架自带了一些中间件，包括认证、CSRF 保护中间件等等。所有的中间件都位于 `app/Http/Middleware` 目录下。

4.2.2. 定义中间件

要创建一个中间件，可以通过 Artisan 命令 `make:middleware`

```
php artisan make:middleware CheckAge
```

这个命令会在 `app/Http/Middleware` 目录下创建一个新的中间件。

4.2.2.1. 请求之前/之后的中间件

一个中间件是请求前还是请求后执行取决于中间件本身。

4.2.3. 注册中间件

中间件分为三类，分别是全局中间件、中间件组和指定路由中间件。

4.2.3.1. 全局中间件

如果您想要定义的中间件在每一个 HTTP 请求时都被执行，只需要将相应的中间件类添加到 `app/Http/Kernel.php` 的数组属性 `$middleware` 中即可。

4.2.3.2. 指定路由中间件

如果您想要分配中间件到指定路由，首先应该在 `app/Http/Kernel.php` 文件中分配该中间件一个 `key`。

4.2.3.3. 中间件组

有时候您可能想到通过指定一个键名的方式将相关中间件分到同一个组里面，这样可以更方便地将其分配到路由中。

4.2.4. 中间件排序

很少情况下，您可能需要中间件以特定的顺序执行，但是当它们被分配到路由时，你无法控制它们的顺序。

4.2.5. 中间件参数

中间件还可以接收额外的自定义参数，例如，如果应用需要在执行给定动作之前验证认证用户是否拥有指定的角色，可以创建一个 `CheckRole` 来接收角色名作为额外参数。

4.2.6. 终端中间件

终端中间件，可以理解为一个善后的后台处理中间件。有时候中间件可能需要在 HTTP 响应发送到浏览器之后做一些工作。比如，Laravel 内置的 `session` 中间件会在响应发送到浏览器之后将 `Session` 数据写到存储器中，为了实现这个功能，需要定义一个终止中间件并添加 `terminate` 方法到这个中间件。

4.3. CSRF 保护

4.3.1. 简介

跨站请求伪造（CSRF）是一种通过伪装授权用户的请求来攻击授信网站的恶意漏洞。

4.3.2. 排除指定 URL 不做 CSRF 安全校验

有时候我们需要从 CSRF 保护中间件中排除一些 URL。

4.4. 控制器

4.4.1. 简介

为了替代在路由文件中以闭包形式定义的所有的请求处理逻辑，你可能想要使用控制类来组织这些行为。控制器能将相关的请求处理逻辑组成一个单独的类。控制器被存放在 `app/Http/Controllers` 目录。

4.4.2. 控制器入门

4.4.2.1. 定义控制器

下面是一个基础控制器类的例子。需要注意的是，该控制器继承了 `Laravel` 的基类控制器。该基类控制器提供了一些便利的方法，比如 `middleware` 方法，该方法可以为控制器行为添加中间件。

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\User;

class UserController extends Controller
{
    /**
     * 显示指定用户的简介.
     *
     */
}
```

```
* @param int $id

* @return View

*/

public function show($id)

{

    return view('user.profile', ['user' => User::findOrFail($id)]);

}

}
```

我们可以像这样定义指向该控制器动作的路由：

```
Route::get('user/{id}', 'UserController@show');
```

现在，如果有一个请求匹配上面的路由 URI，UserController 的 show 方法就会被执行。

4.4.2.2. 命名空间

需要着重指出的是，在定义控制器路由时我们不需要指定完整的控制器命名空间。因为 RouteServiceProvider 会在一个包含命名空间的路由组中加载路由文件，我们只需要指定类名中 App\Http\Controllers 命名空间之后的部分就可以了。

4.4.2.3. 单一动作控制器

如果您想要定义一个只处理一个动作的控制器，可以在这个控制器中定义 `__invoke` 方法。

```
<?php

namespace App\Http\Controllers;

use App\User;

use App\Http\Controllers\Controller;

class ShowProfile extends Controller
```

```
{

    /**
     * 展示给定用户的简介
     *
     * @param int $id
     * @return Response
     */
    public function __invoke($id)
    {
        return view('user.profile', ['user' => User::findOrFail($id)]);
    }
}
```

当您为这个单动作控制器注册路由的时候，不需要指定方法。

```
Route::get('user/{id}', 'ShowProfile');
```

这背后的原理是__invoke()方法会被自动调用。

4.4.3. 控制器中间件

中间件可以像这样分配给控制器路由。

```
Route::get('profile', 'UserController@show')->middleware('auth');
```

4.4.4. 资源控制器

资源控制器可以让我们很便捷地构建基于资源的 RESTful 控制器，例如，您可能想要在应用中创建一个控制器，用于处理关于文章存储的 HTTP 请求。

4.4.4.1. 部分资源路由

当声明资源路由时，你可以指定控制器处理的部分行为，而不是所有默认的

行为。

```
Route::resource('photos', 'PhotoController')->only([
    'index', 'show'
]);

Route::resource('photos', 'PhotoController')->except([
    'create', 'store', 'update', 'destroy'
]);
```

4.4.4.2. 嵌套资源

有时可能需要定义一个嵌套的资源型路由。

4.4.4.3. 命名资源路由

默认情况下，所有的资源控制器行为都有一个路由名称。

4.4.4.4. 命名资源路由参数

默认情况下，`Route::resource` 会根据资源名称的「单数」形式创建资源路由的路由参数。你可以在选项数组中传入 `parameters` 参数来轻松地覆盖每个资源。

4.4.4.5. 自定义资源 URIs

默认情况下，`Route::resource` 将会用英文动词创建资源 URI。

4.4.4.6. 补充资源控制器

如果你想在默认的资源路由中增加额外的路由，你应该在 `Route::resource` 之前定义这些路由。

4.4.5. 依赖注入&控制器

4.4.5.1. 构造函数注入

框架使用服务容器来解析所有的控制器，因此，可以在控制器的构造函数中注入任何依赖，这些依赖会被自动解析并注入到控制器实例中。

4.4.5.2. 方法注入

除了构造函数注入之外，还可以在控制器的动作方法中进行依赖注入。

4.4.6. 路由缓存

如果您的应用完全基于控制器路由，可以使用路由缓存，使用路由缓存将会会极大降低注册所有应用路由所花费的时间

```
php artisan route:cache
```

4.5. HTTP 请求

4.5.1. 接收请求

要通过依赖注入获取当前 HTTP 请求实例，你应该在控制器上引入 `Illuminate\Http\Request` 类。传入的请求实例将会由服务容器自动注入。

4.5.1.1. 请求路径 & 方法

`Illuminate\Http\Request` 提供了各种方法来检查应用程序的 http 请求，并扩展了 `Symfony\Component\HttpFoundation\Request` 类。我们将在下面讨论一些最重要的方法。

4.5.1.2. PSR-7 请求 Requests

PSR-7 标准 指定了包括请求与响应在内的 HTTP 的消息接口。如果你想要获取 PSR-7 请求实例而不是 Laravel 请求， 那么你首先需要安装几个库。Laravel 使用 `Symfony HTTP Message Bridge` 组件将典型的 Laravel 请求和响应转换为 PSR-7 的兼容实现。

```
composer require symfony/psr-http-message-bridge  
  
composer require nyholm/psr7
```

4.5.2. 输入预处理&规范化

默认情况下，在 `App\Http\Kernel` 的全局中间件堆栈中引入了 `TrimStrings` 和 `ConvertEmptyStringsToNull` 中间件。这些中间件会自动对请求中的字符串字段进行处理，前者将字符串两端的空格清除，后者将空字符串转化为 `null`。

4.5.3. 接收数据

您可以使用 `all` 方法以数组格式获取所有输入值。

```
$input = $request->all();
```

4.5.3.1. 上一次请求输入

框架允许您在两次请求之间保存上一次输入数据，这个特性在检测校验数据失败后需要重新填充表单数据时很有用。

4.5.3.2. Cookies

为了安全起见，框架创建的 Cookie 都经过加密并使用一个认证码进行签名。

4.5.4. 文件上传

4.5.4.1. 接收上传文件

您可以使用 `file` 方法或使用动态属性从 `Illuminate\Http\Request` 实例中访问上传的文件。该 `file` 方法返回 `Illuminate\Http\UploadedFile` 类的实例，该类继承了 PHP 的 `SplFileInfo` 类的同时也提供了各种与文件交互的方法。

```
$file = $request->file('photo');  
  
$file = $request->photo;
```

您可以使用 `hasFile` 方法判断文件在请求中是否存在。

```
if ($request->hasFile('photo')) {  
  
    //  
  
}
```

4.5.4.2. 存储上传文件

要存储上传的文件，先配置好文件系统。您可以使用 `UploadedFile`

的 `store` 方法把上传文件移动到你的某个磁盘上，该文件可能是本地文件系统中的一个位置，甚至像 Amazon S3 这样的云存储位置。

4.6. HTTP 响应

4.6.1. 创建响应

所有路由和控制器处理完业务逻辑之后都会返回一个发送到用户浏览器的响应，框架提供了多种不同的方式来返回响应，最基本的响应是从路由或控制器返回一个简单的字符串，框架会自动将这个字符串转化为一个完整的 HTTP 响应。

```
Route::get('/', function () {  
    return 'Hello World';  
});
```

除了从路由或控制器返回字符串之外，还可以返回数组。框架会自动将数组转化为一个 JSON 响应。

```
Route::get('/', function () {  
    return [1, 2, 3];  
});
```

4.6.1.1. 添加响应头

大部分的响应方法都是可链式调用的，使得创建响应实例的过程更具可读性。例如，你可以在响应返回给用户前使用 `header` 方法为其添加一系列的头信息。

```
return response($content)  
    ->header('Content-Type', $type)  
    ->header('X-Header-One', 'Header Value')  
    ->header('X-Header-Two', 'Header Value');
```

4.6.1.2. 添加 Cookies 到响应

你可以使用响应上的 `cookie` 方法轻松地将为响应增加 Cookies。

4.6.1.3. Cookies & 加密

默认情况下，框架生成的所有 Cookie 都是经过加密和签名，因此不能被客户端修改或读取。

4.6.2. 重定向

重定向响应是 `Illuminate\Http\RedirectResponse` 类的实例，并且包含用户需要重定向至另一个 URL 所需的头信息。框架提供了几种方法用于生成 `RedirectResponse` 实例，最简单的方法就是使用全局辅助函数 `redirect`。

```
Route::get('dashboard', function () {  
    return redirect('home/dashboard');  
});
```

4.6.2.1. 重定向到命名路由

如果调用不带参数的辅助函数 `redirect` 时，会返回 `Illuminate\Routing\Redirector` 实例。

4.6.2.2. 重定向到控制器行为

还可以生成到 `controller action` 的重定向。要达到这个目的，只要把控制器和 `action` 的名称传递给 `action` 方法。记住，不需要传递控制器的全部命名空间，Laravel 的 `RouteServiceProvider` 会自动将其设置为基本控制器的命名空间：

```
return redirect()->action('HomeController@index');
```

4.6.2.3. 重定向到外部域名

有时候你需要重定向到应用外的域名。调用 `away` 方法可以达到此目的，它会创建一个不带有任何额外的 URL 编码、有效性校验和检查的

RedirectResponse 实例。

4.6.2.4. 重定向并使用闪存的 Session 数据

重定向到新的 URL 的同时 传送数据给 session 是很常见的。通常会在成功执行一个动作并传送消息给 session 之后这样做。为了方便起见，你可以创建一个 RedirectResponse 实例并在链式方法调用中将数据传送给 session。

4.6.3. 其他响应类型

response 助手可以用于生成其他类型的响应实例。

4.6.3.1. 视图响应

如果需要把 视图 作为响应内容返回的同时，控制响应状态和头信息，就需要调用 view 方法。

4.6.3.2. JSON 响应

json 自动将 Content-Type 头信息设置为 application/json,同时使用 PHP 的 json_encode 函数将给定的数组转换为 JSON。

4.6.3.3. 文件下载

download 方法可以用于生成强制用户浏览器下载给定路径文件的响应。download 方法文件名作为其第二个参数，它将作为用户下载文件的文件名。最后，你可以传递 HTTP 头信息数组作为其第三个参数。

```
return response()->download($pathToFile);

return response()->download($pathToFile, $name, $headers);

return response()->download($pathToFile)->deleteFileAfterSend();
```

4.6.3.4. 文件响应

file 方法用于直接在用户浏览器显示一个图片或 PDF 之类的文件，而不是下载。

4.6.4. 响应宏

如果您想要定义一个自定义的可以在多个路由和控制器中复用的响应，可以

使用 `Response` 门面上的 `macro` 方法。

4.7. 视图

4.7.1. 创建视图

视图包含应用的 HTML 代码，并将应用的控制器逻辑和表现逻辑进行分离。视图文件存放在 `resources/views` 目录。

4.7.2. 传递数据到视图

可能简单通过数组方式将数据传递到视图。

```
return view('greetings', ['name' => '测试']);
```

4.7.3. 视图 Composer

视图 Composer 是当视图被渲染时的回调函数或类方法。

4.8. URL 生成

4.8.1. 简介

框架提供了很多个辅助函数来帮助我们在应用中生成 URL。这些函数主要用于在视图模板和 API 响应中构建链接，或者生成重定向响应。

4.8.2. 快速入门

4.8.2.1. 生成基础 URL

`url` 辅助函数可用于为应用生成任意 URL，生成的 URL 将自动使用当前请求中的方案 (HTTP or HTTPS) 和主机。

```
$post = App\Post::find(1);  
  
echo url("/posts/{$post->id}");
```

4.8.2.2. 访问当前 URL

如果没有传递路径信息给 `url` 辅助函数，则会返回一个

Illuminate\Routing\UrlGenerator 实例，从页允许您访问当前 UR 的信息。

```
// Get the current URL without the query string...  
echo url()->current();  
  
// Get the current URL including the query string...  
echo url()->full();  
  
// Get the full URL for the previous request...  
echo url()->previous();
```

4.8.3. 命名路由 URL

辅助函数 `route` 可以用于为指定路由生成 URL。命名路由生成的 URL 不与路由上定义的 URL 相耦合。因此，就算路由的 URL 有任何改变，都不需要对 `route` 函数调用进行任何更改。

4.8.4. 控制器行为 URL

`action` 辅助函数用于为控制器动作生成 URL，和路由中定义的一样，您不需要传递完整的控制器命名空间。

4.8.5. 默认值

对于某些应用程序，您可能希望为某些 URL 参数的请求范围指定默认值。例如，假设有些路由定义了 `{locale}` 参数。

```
Route::get('/{locale}/posts', function () {  
    //  
})->name('post.index');
```


4.9. Session

4.9.1. 简介

由于 HTTP 协议本身是无状态的，Session 提供了一种在多个请求之间存储有关用户信息的方法，Laravel 通过同一个可读性强的 API 处理各种自带的后台驱动程序。支持诸如比较热门的 Memcached, Redis 和数据库。

4.9.1.1. 配置

Session 的配置文件存储在 `config/session.php` 文件中。请务必查看此文件中对于你而言可用的选项。默认情况下，Laravel 为绝大多数应用程序配置的 Session 驱动为 `file`。在生产环境中，你可以考虑使用 `memcached` 或 `redis` 驱动，让 Session 的性能更加出色。

Session driver 的配置预设了每个请求存储 Session 数据的位置。Laravel 自带了几个不错而且开箱即用的驱动：

`file` - 将 Session 存储在 `storage/framework/sessions` 中。

`cookie` - Sessions 被存储在安全加密的 `cookie` 中。

`database` - Sessions 被存储在关系型数据库中。

`memcached / redis` - Sessions 被存储在基于高速缓存的存储系统中。

`array` - Sessions 存储在 PHP 数组中，但不会被持久化。

4.9.1.2. 驱动程序先决条件

数据库

使用 `database` 作为 Session 驱动时，你需要创建一张包含 Session 各项数据的表。以下是使用 Schema 建表的例子。

```
Schema::create('sessions', function ($table) {  
    $table->string('id')->unique();  
    $table->unsignedInteger('user_id')->nullable();  
    $table->string('ip_address', 45)->nullable();  
});
```

```
$table->text('user_agent')->nullable();

$table->text('payload');

$table->integer('last_activity');

});
```

Redis

Laravel 在使用 Redis 作为 Session 驱动之前，需要通过 Composer 安装 predis/predis 扩展包 (~1.0)。然后在 database 配置文件中配置 Redis 连接信息。在 session 配置文件中，connection 选项可用于指定 Session 使用哪个 Redis 连接。

4.9.2. 使用 Session

4.9.2.1. 获取数据

主要有两种方式处理 Session 数据，全局的辅助函数 session，或者通过 Request 实例（启动过程中会将 Session 数据设置到请求实例的 session 属性中）。

4.9.2.2. 存储数据

要在 Session 中存储数据，通常可以通过 put 方法或 session 辅助函数。

```
//通过调用请求实例的 put 方法

$request->session()->put('key', 'value');

// 通过全局辅助函数 session

session(['key' => 'value']);
```

4.9.2.3. 闪存数据

有时候你可能想要在 Session 中存储有效的数据用于下个请求，这可以通过 flash 方法来实现。使用该方法存储的 Session 数据只在随后的 HTTP 请求中有效，然后将会被删除。闪存数据主要用于短期的状态消息。

```
$request->session()->flash('status', 'Task was successful!');
```

4.9.2.4. 删除数据

Forget 方法从 Session 中移除指定数据,如果您要从 Session 中移除所有数据,可以使用 flush 方法。

```
// Forget a single key...

$request->session()->forget('key');

// Forget multiple keys...

$request->session()->forget(['key1', 'key2']);

$request->session()->flush();
```

4.9.2.5. 重新生成 SessionID

重新生成 SessionID 经常用于阻止恶意用记对应用进行 session fixation 攻击。

如果您使用内置的 LoginController 的话,框架会在认证期间自动重新生成 session ID,如果您需要手动重新生成 sessionID,可以使用 regenerate 方法。

```
$request->session()->regenerate();
```

4.9.3. 添加自定义 Session 驱动

4.9.3.1. 实现驱动

自定义 Session 驱动需要实现 SessionHandlerInterface 接口,改接口包含少许我们需要实现的方法,比如一个基于 MongoDB 的 Session 驱动实现如下。

```
<?php

namespace App\Extensions;

class MongoSessionHandler implements SessionHandlerInterface
```

```
{

    public function open($savePath, $sessionName) {}

    public function close() {}

    public function read($sessionId) {}

    public function write($sessionId, $data) {}

    public function destroy($sessionId) {}

    public function gc($lifetime) {}

}
```

4.9.3.2. 注册驱动

Session 驱动被实现后，需要将其注册到框架，要添加额外驱动到后端，可以使用 Session 门面上的 extend 方法。

4.10. 验证

4.10.1. 简介

框架提供了多种方法来验证请求输入数据。默认情况下，控制器基类使用 ValidatesRequest trait，该 trait 提供了便捷方法通过各种功能强大的验证规则来验证输入的 HTTP 请求。

4.10.2. 快速入门

要了解框架强大的验证功能，让我们看一个验证表单并将错误消息显示回给用户的完整示例。

4.10.2.1. 定义路由

首先，我们假定在 routes/web.php 文件中包含如下路由。

```
Route::get('post/create', 'PostController@create');

Route::post('post', 'PostController@store');
```

GET 路由会用户显示了一个创建新的文章的表单, POST 路由将新的文章存储到数据库。

4.10.2.2. 创建控制器

下面让我们一起来看看处理这些路由的控制器 `store` 方法暂时留空。

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class PostController extends Controller
{
    /**
     * 显示创建博客文章的表单.
     *
     * @return Response
     */
    public function create()
    {
        return view('post.create');
    }

    /**
```

```
    * 保存一篇新的博客文章

    *

    * @param Request $request

    * @return Response

    */

    public function store(Request $request)

    {

        // 验证并存储博客文章...

    }

}
```

4.10.2.3.编写验证逻辑

现在我们开始在 `store` 方法中编写逻辑来验证新的博客文章。为此，我们将使用 `Illuminate\Http\Request` 对象提供的 `validate` 方法。如果验证通过，代码就可以正常的运行。如果验证失败，则会抛出异常，并自动将对应的错误响应返回给用户。在典型的 HTTP 请求的情况下，会生成一个重定向响应，而对于 AJAX 请求则会发送 JSON 响应。

让我们接着回到 `store` 方法来深入理解 `validate` 方法：

```
/**

 * 保存一篇新的博客文章

 *

 * @param Request $request

 * @return Response

 */
```

```
public function store(Request $request)
{
    $validatedData = $request->validate([
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ]);

    // 博客文章验证通过...
}
```

如你所见，我们将所需的验证规则传递至 `Validate` 方法中。另外再提醒一次，如果验证失败，会自动生成一个对应的响应。如果验证通过，那我们的控制器将会继续进行。

4.10.2.4. 显示验证错误信息

如果传入的请求参数未通过给定的验证规则呢？正如前面所提到的，框架会自动把用户重定向到之前的位置。另外，所有的验证错误信息会被自动存储到 `session`。

重申一次，我们不必在 `GET` 路由中将错误消息显示绑定到视图。因为框架会检查在 `Session` 数据中的错误信息，并自动将其绑定到视图。

4.10.2.5. 可选字段注意事项

默认情况下，在应用的全局中间件堆栈 `App\Http\Kernel` 类中包含了 `TrimStrings` 和 `ConvertEmptyStringsToNull` 中间件。因此，如果你不希望验证程序将 `null` 值视为无效的话，那就需要将可选的请求字段标记为 `mullable`，举个例子。

```
$request->validate([
    'title' => 'required|unique:posts|max:255',
```

```
'body' => 'required',  
  
'publish_at' => 'nullable|date',  
]);
```

4.10.3. 表单请求验证

4.10.3.1. 创建表单请求验证

面对更复杂的验证情境中，你可以创建一个 [表单请求] 来处理更为复杂的逻辑。表单请求是包含验证逻辑的自定义请求类。可使用 **Artisan** 命令 `:request` 来创建表单请求类。

```
php artisan make:request StoreBlogPost
```

新生成的类保存在 `app/Http/Reqeusts` 目录下。如果这个目录不存在，运行 `make:request` 命令时它会被创建出来。让我们添加一些验证规则到 `rules` 方法中。

```
/**  
  
 * 获取应用于请求的验证规则  
  
 *  
 * @return array  
  
 */  
public function rules()  
{  
  
    return [  
  
        'title' => 'required|unique:posts|max:255',  
  
        'body' => 'required',  
  
    ];  
  
}
```


所以，验证规则是如何运行的呢？你所需要做的就是 在控制器方法中类型提示传入的请求。在调用控制器方法之前验证传入的表单请求，这意味着你不需要在控制器中写任何验证逻辑。

4.10.3.2. 表单请求认证授权

表单请求类内也包含了 `authorize` 方法。在这个方法中，你可以检查经过身份验证的用户确定其是否具有更新给定资源的权限。比方说，你可以判断用户是否拥有更新文章评论的权限。

```
/**
 * 判断用户是否有请求权限
 *
 * @return bool
 */
public function authorize()
{
    $comment = Comment::find($this->route('comment'));

    return $comment && $this->user()->can('update', $comment);
}
```

由于所有的表单请求都是继承了框架中的请求基类，所以我们可以使用 `user` 方法去获取当前认证登录的用户。

4.10.3.3. 自定义错误消息

你可以通过重写表单请求的 `messages` 方法来自定义错误消息。

4.10.3.4. 自定义验证属性

如果你希望将验证消息中的 `:attribute` 部分替换为自定义的属性名，可以通过重写 `attributes` 方法来指定自定义的名称。该方法会返回属性名及对应自定义

名称键值对数组。

```
/**
 * Get custom attributes for validator errors.
 *
 * @return array
 */
public function attributes()
{
    return [
        'email' => 'email address',
    ];
}
```

4.10.3.5.准备验证接入

如果需要在应用验证规则之前清除请求中的任何数据，可以使用 `prepareForValidation` 方法。

```
use Illuminate\Support\Str;

/**
 * 准备验证数据
 *
 * @return void
 */
protected function prepareForValidation()
```

```
{

    $this->merge([

        'slug' => Str::slug($this->slug),

    ]);

}
```

4.10.4. 手动创建验证器

如果你不想在请求上使用 `validate` 方法，你可以通过 `Validator facade` 手动创建一个验证器实例。用 `Validator facade` 上的 `make` 方法创建一个验证器实例。

4.10.4.1. 自动重定向

如果你想手动创建验证器实例，又想使用 `validates` 方法提供的自动重定向，那么你可以在现有的验证器示例上调用 `validate` 方法。如果验证失败，用户将会自动重定向。在 `AJAX` 请求中，则会返回 `JSON` 格式的响应。

```
Validator::make($request->all(), [

    'title' => 'required|unique:posts|max:255',

    'body' => 'required',

])->validate();
```

如果验证失败，您可以使用 `validateWithBag` 方法将错误消息存储在命名错误包中。

```
Validator::make($request->all(), [

    'title' => 'required|unique:posts|max:255',

    'body' => 'required',

])->validateWithBag('post');
```

4.10.4.2.命名错误包

如果你一个页面中有多个表单，你可以通过命名包来检索特定表单的错误消息。只需给 `withErrors` 方法传递一个名字作为第二个参数。

```
return redirect('register')

    >withErrors($validator, 'login');
```

然后你就可以从 `$errors` 变量中获取指定表单的错误消息。

```
{{ $errors->login->first('email') }}
```

4.10.4.3.验证后钩子

验证器还允许附加回调并在验证完成后执行，以便你进行下一步的验证，甚至在消息集合中添加更多的错误消息。使用它只需要在验证实例上使用 `after` 方法。

```
$validator = Validator::make(...);

$validator->after(function ($validator) {

    if ($this->somethingElseIsInvalid()) {

        $validator->errors()->add('field', 'Something is wrong with
this field!');

    }

});

if ($validator->fails()) {

    //

}
```

4.10.5. 错误信息处理

通过 `Validator` 实例调用 `errors` 方法，会返回 `Illuminate\Support\MessageBag` 实例，它拥有各种方便的方法处理错误信息。自动提供给所有视图的 `$errors` 变量，也是 `MessageBag` 类的一个实例。

要查看特定字段的第一个错误消息，可以使用 `first` 方法。

```
$errors = $validator->errors();  
  
echo $errors->first('email');
```

4.10.5.1. 自定义验证错误信息

如果你有需要，你也可以使用自定义错误信息代替默认值并进行验证。有几种方法可以指定自定义信息。

4.10.6. 可用验证规则

以下是所有可用验证规则。

`accepted`

验证字段必须是 `yes`, `on`, `1`, 或 `true`。

4.10.7. 按条件添加条件规则

字段包含特定值跳过验证

如果另一个字段具有给定的值，你可能偶尔希望不验证给定字段。

4.10.8. 验证数组

验证表单的输入为数组的字段也不难。你可以使用 `[点]` 方法来验证数组中的属性。例如，如果传入的 HTTP 请求中包含 `photos[profile]` 字段，可以如相验证。

```
$validator = Validator::make($request->all(), [  
  
    'photos.profile' => 'required|image',
```

```
]);
```

4.10.9. 自定义验证规则

4.10.9.1. 使用验证规则对象

Laravel 提供了许多有用的验证规则；同时也支持自定义规则。注册自定义验证规则的方法之一，就是使用规则对象。可以使用 Artisan 命令 `make:rule` 来生成新的规则对象。接下来，让我们用这个命令生成一个验证字符串是否是大写的规则。Laravel 会将新的规则存放在 `app/Rules` 目录中。

```
php artisan make:rule Uppercase
```

4.10.9.2. 使用闭包

如果你在应用程序中只需要一次自定义规则的功能，则可以使用闭包代替规则对象。

4.10.9.3. 使用扩展

注册自定义的验证规则的另一种方法是使用 `Validator facade` 中的 `extend` 方法。

4.10.9.4. 隐式扩展

默认情况下，当所要验证的属性不存在或包含一个空字符串时，使用包含自定义扩展的正常的验证规则是不会执行的。

4.11. 错误处理

4.11.1. 简介

框架当你开始一个新的 Laravel 项目时，它已经为您配置了错误和异常处理。`app\Exceptions\Handler` 类用于记录应用程序触发的所有异常，然后将其呈现回用户。我们将在本文中深入讨论这个类。

4.11.2. 配置

你的 `config/app.php` 配置文件中的 `debug` 选项决定了对于一个错误实际上将显示多少信息给用户。默认情况下，该选项的设置将遵照存储在 `.env` 文件中的 `APP_DEBUG` 环境变量的值。

对于本地开发，你应该将 `APP_DEBUG` 环境变量的值设置为 `true`。在生产环境中，该值应始终为 `false`。如果在生产中将该值设置为 `true`，则可能会将敏感配置值暴露给应用程序的终端用户。

4.11.3. 异常处理器

所有异常都由类 `App\Exceptions\Handler` 处理，该类包含两个方法：`report` 和 `render`。

4.11.3.1. Report 方法

所有异常都是由 `App\Exceptions\Handler` 处理。这个类包含了两个方法：`report` 和 `render`。我们将详细的剖析这些方法。`report` 方法用于记录异常或将它们发送给如 `Flare`、`Bugsnag` 或 `Sentry` 等外部服务。默认情况下，`report` 方法将异常传递给记录异常的基类。不过，你可以用任何自己喜欢的方式来记录异常。

例如，如果你需要以不同方式报告不同类型的异常，则可以使用 `PHP` 的 `instanceof` 比较运算符。

```
/**
 * 报告或记录异常
 *
 * 这是一个向 Flare、Sentry、Bugsnag 等外部服务发送异常信息的好位置
 *
 * @param \Exception $exception
 *
 * @return void
 */
```

```
public function report(Exception $exception)
{
    if ($exception instanceof CustomException) {
        //
    }

    parent::report($exception);
}
```

在正常情况下，框架会自动将当前用户的 ID 作为数据添加到每一条异常日志中。你可以通过重写 `App\Exceptions\Handler` 类中的 `context` 方法来定义你定义你的全局环境变量。之后，这个变量将包含在每一条异常日志中。

```
/**
 * 定义默认的环境变量
 *
 * @return array
 */
protected function context()
{
    return array_merge(parent::context(), [
        'foo' => 'bar',
    ]);
}
```

4.11.3.2.Render 方法

`render` 方法负责将给定的异常转换为将被发送回浏览器的 HTTP 响应。默认情况下，异常将传递给为你生成响应的基类。不过，你可以按自己意愿检查异常类型或返回自己的自定义响应。

```
/**
 * 将异常转换为 HTTP 响应。
 *
 * @param \Illuminate\Http\Request $request
 * @param \Exception $exception
 * @return \Illuminate\Http\Response
 */
public function render($request, Exception $exception)
{
    if ($exception instanceof CustomException) {
        return response()->view('errors.custom', [], 500);
    }

    return parent::render($request, $exception);
}
```

4.11.3.3.Reportable & Renderable 异常

除了在处理器的 `report` 和 `render` 方法中进行异常类型检查外，还可以在自定义异常中直接定义 `report` 和 `render` 方法。

4.11.4. HTTP 异常

一些异常用于描述产生自服务器的 HTTP 错误代码。例如，「页面未找到」

错误 (404), 「未经授权的错误」(401) , 甚至可以是开发人员引起的 500 错误。你可以使用 `abort` 辅助函数从应用程序的任何地方生成这样的响应。

```
abort(404);
```

辅助函数 `abort` 会立即引发一个由异常处理器渲染的异常。你还可选择性地提供响应文本。

```
abort(403, 'Unauthorized action.');
```

4.11.4.1. 自定义 HTTP 错误页面

Laravel 可以轻松显示各种 HTTP 状态代码的自定义错误页面。例如, 如果你希望自定义 404 HTTP 状态码的错误页面, 可以创建一个 `resources/views/errors/404.blade.php` 视图文件。该文件将被用于你的应用程序产生的所有 404 错误。此目录中的视图文件的命名应匹配它们对应的 HTTP 状态码。

4.12. 日志

4.12.1. 简介

为了帮助您了解更多关于应用中所发生的事情, 框架提供了强大的日志服务来记录日志信息到文件、系统错误日志、甚至是 Slack 以便通知整个团队。

在日志引擎之下, 框架集成了 Monolog 日志库以便提供各种功能强大的日志处理器, 从而允许您来通过它们定制自己应用的日志处理。

4.12.2. 配置

应用日志系统的所有配置都存放在配置文件 `config/logging.php` 中。这个文件允许你配置你的应用程序日志通道, 所以务必查看每个可用的通道及它们的选项。当然, 我们将在下面回顾一些常用的选项。

默认情况下, 框架将使用 `stack` 去记录日志消息。`stack` 通道被用来将多个日志通道聚合到一个单一的通道中。

4.12.2.1. 构建日志推栈

示例配置。

```
'channels' => [  
    'stack' => [  
        'driver' => 'stack',  
        'channels' => ['syslog', 'slack'],  
    ],  
    'syslog' => [  
        'driver' => 'syslog',  
        'level' => 'debug',  
    ],  
    'slack' => [  
        'driver' => 'slack',  
        'url' => env('LOG_SLACK_WEBHOOK_URL'),  
        'username' => 'Laravel Log',  
        'emoji' => ':boom:',  
        'level' => 'critical',  
    ],  
],
```

4.12.3. 写入日志信息

您可以使用 Log 门面记录日志信息。

```
Log::emergency($error);  
  
Log::alert($error);  
  
Log::critical($error);
```

```
Log::error($error);  
  
Log::warning($error);  
  
Log::notice($error);  
  
Log::info($error);  
  
Log::debug($error);
```

4.12.3.1. 写入指定的通道

有时候你可能希望将消息写入到应用默认通道之外的通道中。可以使用 `Log` 门面的 `channel` 方法获取定义在配置文件中的人任一通道并将消息写入其中。

```
Log::channel('slack')->info('Something happened!');
```

4.12.4. 先进的 Monolog 日志通道定制

4.12.4.1. 高级 Monolog 通道自定义

有时需要完全控制已存在通道的 Monolog：比如，你可能想要为给定通道的日志处理配置自定义的 Monolog FormatterInterface 实现。

先在通道配置中定义一个 `tap` 数组。 `tap` 数组包含一个在通道创建后有机会用于自定义 Monolog 实例的类列表。

```
'single' => [  
    'driver' => 'single',  
    'tap' => [App\Logging\CustomizeFormatter::class],  
    'path' => storage_path('logs/laravel.log'),  
    'level' => 'debug',  
],
```

4.12.4.2. 创建 Monolog 处理器通道

Monolog 多种 可用处理器。在某些情况下，你会希望仅创建一个带有指定处理器的 Monolog 驱动的日志类型。这些通道可以使用 monolog 驱动创建。

在使用 monolog 驱动时，handler 配置项用于指定被实例化的处理器。如果该处理器的构造器需要参数，可以使用可选的 with 配置项来指定。

```
'logentries' => [  
    'driver' => 'monolog',  
    'handler' => Monolog\Handler\SyslogUdpHandler::class,  
    'with' => [  
        'host' => 'my.logentries.internal.datahubhost.company.com',  
        'port' => '10000',  
    ],  
],
```

第五章 前端开发

5.1. Blade 模板引擎

5.1.1. 简介

Blade 是由框架提供的非常简单但功能强大的模板引擎。和其他流行的 PHP 模板引擎不同，Blade 并不限制你在视图中使用原生 PHP 代码。所有 Blade 视图文件都将被编译成原生的 PHP 代码并缓存起来，除非它被修改，否则不会重新编译，这就意味着 Blade 基本上不会给你的应用增加任何负担。Blade 视图文件使用 .blade.php 作为文件扩展名，被存放在 resources/views 目录。

5.1.2. 模板继承

5.1.2.1. 定义布局

Blade 有两个主要优点是 模板继承 和 区块。为方便入门，让我们先通过一个简单的例子来上手。

5.1.2.2. 继承布局

在定义一个子视图时，使用 Blade 的 `@extends` 指令指定子视图要「继承」的视图。扩展自 Blade 布局的视图可以使用 `@section` 指令向布局片段注入内容。

5.1.3. 显示数据

可以将变量放在大括号中传递到 Blade 视图中显示。

5.1.3.1. Blade & JavaScript 框架

由于许多 JavaScript 框架也使用“大括号”来表示预设的表达式应该显示在浏览器中，因此可以使用 `@` 符号表示 Blade 渲染引擎表达式应保持不变。

5.1.4. 控制结构

除了模板继承和显示数据之外，Blade 还为常见的 PHP 控制结构提供了便捷的书写方式，例如条件和循环语句。这些快捷方式提供了一种简洁的 PHP 控制语句的写法，同时保持了与 PHP 中的相应结构的语法特性。

5.1.4.1. if 语句

你可以使用 `@if`, `@elseif`, `@else`, 和 `@endif` 指令来构造 if 语句。

5.1.4.2. Switch 语句

Switch 语句可以使用 `@switch`, `@case`, `@break`, `@default` 和 `@endswitch` 指令来构造。

5.1.4.3. 循环

除了条件语句外，Blade 还提供了用于处理 PHP 循环结构的简单指令。同样，这些指令中的每个函数的功能都与对应的 PHP 的函数功能相同。

5.2. 本地化

5.2.1. 简介

框架本地化特性提供了从不同语言文件中检索字符串的简单方法，让您的应用更好地支持多语言开发。

5.2.1.1. 区域设置

应用的默认语言设置保存在 `config/app.php` 配置文件中。

5.2.2. 定义翻译字符串

5.2.2.1. 如何使用短键

通常，翻译字符串都存放在 `resources/lang` 目录下的文件里。

5.2.2.2. 如何使用翻译字符串作为键

对于有大量翻译需求的应用，如果每条翻译语句都要一一使用『短键』来定义，那么当你在视图中尝试去引用这些『短键』的时候，很容易变得混乱。因此，Laravel 也支持使用字符串『默认』翻译作为关键字来定义翻译字符串。

使用翻译字符串作为键的翻译文件以 JSON 格式存储在 `resources/lang` 目录中。

5.2.3. 检索翻译字符串

你可以使用辅助函数 `__` 从语言文件中检索，`trans` 函数接受翻译字符串所在的文件名加键名作为其第一个参数。

5.2.3.1. 替换翻译字符串中的参数

如果需要，你可以在翻译字符串中定义占位符。所有的占位符都有一个 `:` 前缀。

5.2.3.2. 复数

复数是一个复杂的问题，因为不同的语言对复数有不同的规则，使用『管道符』`|`，可以区分字符串的单复数形式。

5.2.4. 重写扩展包的语言文件

部分扩展包可能会附带自己的语言文件。你可以通过在 `resources/lang/vendor/{package}/{locale}` 目录放置文件来重写它们，而不要直接修改扩展包的核心文件。

例如，当你需要重写 `skyrim/hearthfire` 扩展包的英语语言文件 `messages.php`，则需要把文件存放为 `resources/lang/vendor/hearthfire/en/messages.php`。在这个文件中，你只需要定义你想要修改的翻译字符串。任何没有被重写的翻译字符串仍将从扩展包的原始语言文件中加载。

5.3. 前端脚手架

5.3.1.1. 简介

框架并不强制您使用什么 JavaScript 框架或者 CSS 预处理器，不过它确实提供了适用于许多应用场景的 `Bootstrap`，`React`，和 `Vue` 这将对很多应用程序有帮助。默认情况下，Laravel 使用 `NPM` 安装这两个前端软件包。

框架提供的 `Bootstrap` 和 `Vue` 的脚手架位于 `laravel/ui` Composer 包中，可以使用 `Composer` 安装。

```
composer require laravel/ui:^2.4
```

安装完 `laravel/ui` 包后，你就可以使用 `ui Artisan` 命令安装前端脚手架。

```
// 生成基本脚手架...

php artisan ui bootstrap

php artisan ui vue

php artisan ui react


// 生成 登陆/注册 脚手架...

php artisan ui bootstrap --auth
```



```
php artisan ui vue --auth  
php artisan ui react --auth
```

5.3.1.2. 编写 CSS

在安装 `laravel/ui` 编写器包并生成前端脚手架之后，Laravel 的 `package.json` 文件将包含 `bootstrap` 包，以帮助你开始使用 `bootstrap` 对应用程序的前端进行原型设计。但是你可以根据自己应用的需要，在 `package.json` 文件中，随意添加或删除依赖包。不是一定要使用 `Bootstrap` 框架来构建您的 Laravel 应用程序，它只是为想使用它的人提供了一个易用的起点。

5.3.1.3. 编写 JavaScript

应用所依赖的所有 JavaScript 包必须能在项目根目录的 `package.json` 文件中找到。这个文件与 `composer.json` 文件类似，`composer.json` 文件解决 PHP 的依赖关系，`package.json` 文件则解决 JavaScript 的依赖关系。

5.3.1.3.1. 编写 Vue 组件

当使用 `laravel / ui` 包给你的前端搭脚手架时，`ExampleComponent.vue` Vue 组件将会放置在 `resources/js/components` 目录中。

5.3.1.3.2. 编写 React 组件

如果更喜欢使用 `React` 构建 JavaScript 应用，Laravel 使得在 `Vue` 脚手架和 `React` 脚手架间切换变得非常容易。

5.3.1.4. 添加预设命令

预设命令是 `macroable` 的，允许您在运行时添加额外方法到 `UiComand` 类。

5.4. 编译资源(Mix)

5.4.1.1. 简介

Laravel Mix 提供了定义 `Webpack` 构建步骤的 API，用于在应用中使用常见的 CSS 和 JavaScript 预处理器。通过链式调用这些简洁方法，可以流畅地定义

资源管道。例如：

```
mix.js('resources/js/app.js', 'public/js')

    .sass('resources/sass/app.scss', 'public/css');
```

如果对如何开始使用 **Webpack** 和资源编译感到困惑和不知所措，那么你一定喜欢 **Laravel Mix**。但是，你不一定非要使用它来开发应用；可以使用你喜欢的任何资源管道工具，甚至干脆不用。

5.4.1.2. 安装 & 设置

在触发 **Mix** 之前，要先确保机器上已经安装了 **Node.js** 和 **NPM**。

5.4.1.3. 运行 Mix

Mix 是 **Webpack** 的顶层配置，因此运行 **Mix** 任务，只需要执行一条包含在 **Laravel** 默认的 **package.json** 文件中的脚本。

5.4.1.4. 使用样式表

webpack.mix.js 文件所有资源编译的切入点。把它想像为 **Webpack** 的轻量级配置封装。**Mix** 任务可以与如何编译资源的配置一起被链式调用。

5.4.1.4.1. Less

less 方法可用于将 **Less** 编译成 **CSS**。下面的语句将 **app.less** 文件编译为 **public/css/app.css**：

```
mix.less('resources/less/app.less', 'public/css');
```

5.4.1.4.2. Sass

sass 方法能将 **Sass** 编译成 **CSS**。可以像下面这样使用这个方法。

```
mix.sass('resources/sass/app.scss', 'public/css');
```

5.4.1.5. 使用 JavaScript

Mix 提供了几个有助于使用 **JavaScript** 文件的特性，比如编译 **ECMAScript 2015**、模块打包、最小化和合并原生 **JavaScript** 文件。更棒的是，所有这些都

无需配置就可以很好地工作。

5.4.1.5.1. Vendor 提取

将应用自身的 JavaScript 和第三方库绑定在一起有个潜在的缺点：会使长期缓存变得困难。例如，应用代码的单独更新将强制浏览器重新下载全部第三方库，即使第三方库没有发生变化。

5.4.1.6. 复制文件 & 目录

`copy` 方法可以用于将文件和目录复制到新位置。当 `node_modules` 目录中特殊的资源需要重新定位到 `public` 文件夹时，这个方法很有用。

5.4.1.7. 版本控制/缓存清除

许多开发者在他们编译后的资源添加时间戳或唯一令牌作后缀，强制浏览器加载新的资源，以替换旧的代码副本。`Mix` 可以使用 `version` 方法替你处理它们。

第六章 安全相关

6.1. 用户认证

6.1.1. 简介

Laravel 使得实现身份验证非常简单。事实上，几乎所有的配置都是现成的。身份验证配置文件位于 `config/auth.php`，其中包含几个有良好文档记录的选项，用于调整身份验证服务的行为。

在其核心，Laravel 的认证设施由「守卫」和「提供者」组成。守卫决定如何对每个请求的用户进行身份验证。比如，Laravel 带有一个 `session` 保护，它使用会话存储和 `Cookies` 来维护状态。

提供者决定如何从持久储存中检索用户。Laravel 支持使用 `Eloquent` 和数据库查询生成器检索用户。但是，你可以根据应用程序的需要来自定义其他提供者。

6.1.1.1. 数据库注意事项

默认情况下，Laravel 包含一个 App\User Eloquent model 在你的 app 目录下。这个模型可与默认的 Eloquent 身份验证驱动程序一起使用。如果你的应用程序没有使用 Eloquent，你可以用 database 身份验证驱动程序，它用的是 Laravel 查询生成器。

当为 App\User 模型生成数据库架构时，确保密码的长度至少为 60 个字符。保持默认的字符串长度为 255 个字符是一个不错的选择。

另外，你应该验证 users（或等效）表是否包含一个可空的，含有 100 个字符的 remember_token 字符串。此列将用于存储用户登录应用程序时选择「记住我」选项的令牌。

6.1.2. 快速入门

Laravel 附带了几个预构建的身份验证控制器，它们位于 App\Http\Controllers\Auth 命名空间中。RegisterController 处理新的用户注册，LoginController 处理身份验证，ForgotPasswordController 处理用于重置密码的电子邮件链接，ResetPasswordController 包含重置密码的逻辑。这些控制器中的每一个都使用一个特性来包含它们的必要方法。对于许多应用程序，你根本不需要修改这些控制器。

6.1.2.1. 路由

框架的 laravel/ui 包提供了一种快速方法，可以使用一些简单的命令来支持你进行身份验证所需的所有路由和视图：

```
composer require laravel/ui

php artisan ui vue --auth
```

这个命令应该在新应用程序上使用，并将安装布局视图，注册和登录视图以及所有身份验证端点的路由。还将生成一个 HomeController 来处理应用程序仪表板的登录后请求。

6.1.2.2. 视图

如上一节所述，laravel/ui 包的 `php artisan ui vue --auth` 命令将创建验证所需的所有视图，并将它们放在 `resources/views/auth` 目录下。

`ui` 命令还将创建一个 `resources/views/layouts` 目录，其中包含应用程序的基本布局。这些视图都使用 Bootstrap CSS 框架，但是你可以自由地定制它们。

6.1.2.3. 认证

现在你已经给身份验证控制器设置了路由和视图，你可以为应用程序注册和验证新用户了！因为控制器已经默认包含了验证用户是否存在和保存用户到数据库中的认证逻辑（通过 `traits` 实现的），现在你已经可以在浏览器中访问应用了。

6.1.2.4. 获取已验证用户

你可以通过 `Auth facade` 来访问已认证的用户：

```
use Illuminate\Support\Facades\Auth;

// 获取当前通过认证的用户...

$user = Auth::user();

// 获取当前通过认证的用户 ID...

$id = Auth::id();
```

6.1.2.5. 路由保护

路由中间件 可以用于只允许通过认证的用户访问给定的路由。Laravel 自带了一个 `auth` 中间件，它定义在 `Illuminate\Auth\Middleware\Authenticate` 中。

6.1.2.6. 确认密码

在某些情况下，您可能希望要求用户在访问应用程序的特定区域之前确认其密码。例如，您可能需要先执行此操作，然后用户才能修改应用程序中的账单

设置。

为此，Laravel 提供了一个 `password.confirm` 中间件。

6.1.2.7. 登录限流

如果您使用了自带的 `LoginController` 类，就已经启用了内置的 `Illuminate\Foundation\Auth\ThrottlesLogins`。

默认情况下，如果用户多次尝试却无法提供正确的登录凭据，那么该用户在一分钟内将不能再次尝试登录。这种限流策略基于用户的用户名 / 邮箱地址及其 IP 地址的唯一性。

6.1.3. 手动验证用户

不一定非要在 Laravel 中使用验证控制器。如果选择删除这些控制器，就需要直接使用 Laravel 验证类。别担心，很容易！

可以借助 `Auth facade` 访问 Laravel 服务，因此需要在类的开头导入 `Auth`。

6.1.3.1. 记住用户

如果想在应用中提供「记住我」功能，可以给 `attempt` 方法传递一个布尔值作为其第二个参数，这会无限期保持用户身份验证，直到用户手动登出。

6.1.3.2. 其他认证方法

如果要将已经存在的用户登入应用，可以调用 `login` 方法，并以用户实例作为其参数。该对象必须实现 `Illuminate\Contracts\Auth\Authenticatedable` 契约。

6.1.4. HTTP 基本认证

HTTP 基础认证提供了一种快速方法来验证你应用程序中的用户，而无需设置专用的「登录」页面。开始之前，先把 `auth.basic` 中间件附加到你的路由中。

6.1.4.1. 无状态 HTTP 基本认证

你也可以使用 HTTP 基础身份验证，而无需在会话中设置用户标识符 `cookie`，这对 API 的身份验证特别有用。为此，请定义一个中间件 它将调用 `onceBasic` 方法。

6.1.5. 退出

要手动把用户从应用中退出登录,你可以使用 `Auth facade` 上的 `logout` 方法。

6.1.5.1. 其他设备的会话无效

框架还提供了一种机制,用于将其它设备上的用户 `Session` 失效和「注销」,而不会使当前设备上的 `Session` 失效。当你想要用户更改或更新密码时可以使其他设备上的会话无效并且保持当前设备的登录状态时,可以使用此功能。

6.1.6. 添加自定义保护

你可以使用 `Auth facade` 的 `extend` 方法来定义自己的身份验证看守器。你应该在 服务提供者 中调用 `extend` 方法。

6.1.6.1. 关闭请求保护

实现基于 `HTTP` 请求的自定义身份验证系统的最简单方法,是使用 `Auth::viaRequest` 方法。此方法允许您使用单个闭包来快速定义身份验证过程。

首先,在 `AuthServiceProvider` 的 `boot` 方法中调用 `Auth::viaRequest` 方法。`viaRequest` 方法接受一个看守器名称作为其第一个参数。此名称可以是描述你自定义看守器的任何字符串。传递给该方法的第二个参数应该是一个闭包函数,它接收传入的 `HTTP` 请求并返回一个用户实例,或者,如果验证失败,则为 `null`。

6.1.7. 添加自定义用户提供者

如果不使用传统的关系数据库存储用户,就需要使用自己的身份验证用户提供器扩展 `Laravel`。

6.1.7.1. 用户提供程序契约

`Illuminate\Contracts\Auth\UserProvider` 实现仅负责从 `MySQL`、`Riak` 等持久化存储系统中提取 `Illuminate\Contracts\Auth\Authenticatable` 实现。无论用户如何存储及用于表示它的类是什么类型,这两个接口都允许 `Laravel` 身份验证机制继续运行。

6.1.7.2. 可认证的契约

我们已经剖析了 `UserProvider` 的每个方法。下面再来看看 `Authenticatable`

契约。切记，用户提供器的 `retrieveById`、`retrieveByToken` 和 `retrieveByCredentials` 方法将返回此接口的实例。

6.1.8. 事件

框架支持在认证过程中触发多种事件，您可以在自己的 `EventServiceProvider` 中监听这些事件。

6.2. 用户授权

6.2.1. 简介

除了提供开箱即用的认证服务之外，框架还提供了一个简单的方式来管理授权逻辑以便控制对资源的访问权限。和认证一样，主要有两种方式：**Gate** 和 **Policy**。

可以把 `gates` 和策略比作路由和控制器。**Gates** 提供了一种简单的基于闭包的授权方法，而策略和控制器类似，围绕特定模型或资源对其逻辑进行分组来实现授权认证。我们先探索 `gates`，然后研究策略。

6.2.2. Gates

6.2.2.1. 编写 Gates

Gates 是用来决定用户是否授权执行给予动作的一个闭包函数，并且典型的做法就是在 `App\Providers\AuthServiceProvider` 中使用 **Gate** 来定义。**Gates** 总是接收一个用户实例作为第一个参数，并且可以接收可选参数，比如相关的 **Eloquent** 模型：

```
/**
 * 注册任意用户认证、用户授权服务。
 *
 * @return void
 */
public function boot()
{
```



```
$this->registerPolicies();

Gate::define('edit-settings', function ($user) {

    return $user->isAdmin;

});

Gate::define('update-post', function ($user, $post) {

    return $user->id === $post->user_id;

});
}
```

6.2.2.2. 授权动作

使用 **Gate** 来授权动作的时候，你应该使用 **allows** 或者 **denies** 方法。注意，不需要将当前已认证用户传递给这些方法。**Laravel** 会自动处理好已经认证通过的用户，然后传递给 **Gate** 闭包函数：

```
if (Gate::allows('edit-settings')) {

    // 指定当前用户可以编辑设置

}

if (Gate::allows('update-post', $post)) {

    // 指定当前用户可以进行更新...

}

if (Gate::denies('update-post', $post)) {
```

```
// 指定当前用户不能更新...  
}
```

6.2.2.3. Gate 响应

到目前为止，我们只检查了返回简单布尔值的 **Gate**。但是，有时你可能希望返回更详细的响应，包括错误消息。为此，你可以从你的 **Gate** 返回 `Illuminate\Auth\Access\Response`。

```
use Illuminate\Auth\Access\Response;  
  
use Illuminate\Support\Facades\Gate;  
  
Gate::define('edit-settings', function ($user) {  
  
    return $user->isAdmin  
  
        ? Response::allow()  
  
        : Response::deny('You must be a super administrator.');
```

```
});
```

6.2.2.4. Gate 拦截检查

有时，你可能希望将所有能力授予特定用户。所以你可以在所有其他授权检查之前使用 **before** 方法来定义运行的回调：

```
Gate::before(function ($user, $ability) {  
  
    if ($user->isSuperAdmin()) {  
  
        return true;  
  
    }  
  
});
```

6.2.3. 创建策略

6.2.3.1. 生成策略

策略是在特定模型或者资源中组织授权逻辑的类。

6.2.3.2. 注册策略

一旦策略存在，就需要注册它。

6.2.4. 编写策略

6.2.4.1. 策略方法

一旦注册了策略，你可以为它授权的每个操作添加方法。

6.2.4.2. 策略响应

到目前为止，我们只研究了返回简单布尔值的策略方法。然而，有时您可能希望返回更详细的响应，包括错误消息。为此，您可以从您的策略方法返回一个 `\Auth\Access\Response`。

6.2.4.3. 不包含模型方法

一些策略方法只接收当前经过身份验证的用户，而不接收它们授权的模型的实例。这种情况在授权 `create` 操作时最为常见。

6.2.4.4. Guest 用户

默认情况下，如果传入的 HTTP 请求不是由经过身份验证的用户发起的，所有的门和策略都会自动返回 `false`。但是，您可以通过声明一个「可选」类型提示或为用户参数定义提供一个 `null` 默认值，从而允许这些授权检查通过您的门和策略。

6.2.4.5. 策略过滤器

对于某些用户，您可能希望授权给定策略中的所有操作。为此，在策略上定义一个 `before` 方法。`before` 方法将在策略上的任何其他方法之前执行，从而使您有机会在实际调用预期的策略方法之前授权操作。

6.2.5. 使用策略授权动作

6.2.5.1. 通过用户模型

Laravel 应用程序中包含的 `User` 模型包括两个用于授权操作的有用方法：`can` 和 `cant`。`can` 方法接收您希望授权的操作和相关模型。

6.3. 邮箱验证

6.3.1. 简介

很多 Web 应用都要求用户注册之后验证邮箱地址才能登录，为些，框架也提供了便捷方法来发送和验证邮箱验证请求，但是这个功能是可选的，您可以启用也可以不启用。

6.4. 加密解密

6.4.1. 简介

框架的加密机制使用 OpenSSL 所提供的 AES-256 和 AES-128 加密。强烈建议使用自带的加密工具，而不是用其他的加密算法。所有加密之后的结果都会使用消息认证码(MAC)签名，使其底层值不能在加密后再次修改。

6.4.2. 设置

在使用框架的加密工具之前，你必须先设置 `config/app.php` 配置文件中的 `key` 选项。如果这个 `key` 值没有被正确设置，则所有加密值都会是不安全的。

6.4.3. 基本用法

加密一个值。你可以使用 `Crypt` 门面提供的 `encryptString` 来加密一个值。所有加密的值都使用 OpenSSL 的 `AES-256-CBC` 来进行加密。

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Http\Controllers\Controller;

use App\Models\User;

use Illuminate\Http\Request;

use Illuminate\Support\Facades\Crypt;

class UserController extends Controller
{
    /**
     * Store a secret message for the user.
     *
     * @param Request $request
     * @param int $id
     * @return Response
     */
    public function storeSecret(Request $request, $id)
    {
        $user = User::findOrFail($id);

        $user->fill([
            'secret' => Crypt::encryptString($request->secret),
        ])->save();
    }
}
```

解密一个值

你可以使用 `Crypt` 门面提供的 `decryptString` 来进行解密。

```
use Illuminate\Contracts\Encryption\DecryptException;

use Illuminate\Support\Facades\Crypt;

try {

    $decrypted = Crypt::decryptString($encryptedValue);

} catch (DecryptException $e) {

    //

}
```

6.5. 哈希

6.5.1. 简介

Laravel Hash facade 为存储用户密码提供了安全的 `Bcrypt` 和 `Argon2` 哈希加密方式。如果你在你的 Laravel 应用程序中使用了内置的 `LoginController` 和 `RegisterController` 类，那么它们默认使用 `Bcrypt` 进行注册和身份认证。

6.5.2. 配置

你可以在 `config/hashing.php` 配置文件中配置默认哈希驱动程序。目前支持三种驱动程序：`Bcrypt` 和 `Argon2` (`Argon2i` and `Argon2id` variants)。

6.5.3. 基本用法

你可以通过调用 Hash facade 的 `make` 方法来加密你的密码：

```
<?php

namespace App\Http\Controllers;
```

```
use App\Http\Controllers\Controller;

use Illuminate\Http\Request;

use Illuminate\Support\Facades\Hash;

class UpdatePasswordController extends Controller
{
    /**
     * 更新用户密码
     *
     * @param Request $request
     * @return Response
     */
    public function update(Request $request)
    {
        // 验证新密码的长度...

        $request->user()->fill([
            'password' => Hash::make($request->newPassword)
        ]->save();
    }
}
```

调整 Bcrypt 加密函数

如果使用 Bcrypt 算法, 你可以在 `make` 方法中使用 `rounds` 选项来配置该算法的加密系数。然而, 对大多数应用程序来说, 默认值就足够了:

```
$hashed = Hash::make('password', [  
    'rounds' => 12,  
]);
```

6.6. 密码重置

6.6.1. 简介

大多数 Web 应用都提供了为用户重置密码的功能。相较于强迫你在每个应用中都要重新实现一遍此功能, 框架提供了便捷的方法来发送密码提醒以及执行密码重置。

6.6.2. 数据库注意事项

首先, 验证你的 `App\Models\User` 模型是否实现了 `Illuminate\Contracts\Auth\CanResetPassword` 契约。当然, 框架中包含的 `App\Models\User` 模型已经实现了该接口, 并且使用了 `Illuminate\Auth\Passwords\CanResetPassword` trait 来包含实现该接口所需的方法。

6.6.3. 路由

执行密码重置需要的路由都可以通过 Composer 的 `laravel/jetstream` 软件包生成。

6.6.4. 视图

要生成重置密码必要的视图文件, 你可以使用 Composer 的 `laravel/jetstream` 拓展包。

```
composer require laravel/jetstream
```



```
php artisan jetstream:install livewire/inertia
```

这些视图在 `resources/views/auth/passwords` 中。你可以随意根据需要为您的应用程序自定义它们。

6.6.5. 重置密码

一旦你定义了重置用户密码的路由和视图，你可以在浏览器中访问 `/password/reset` 这个路由来重置密码。Laravel Jetstream 软件包中包含的控制器已经包含发送密码重置链接电子邮件以及重置用户密码的逻辑。

6.6.6. 自定义

在你的 `auth.php` 配置文件中，你可以配置多个「看守器」，可以用来定义多个用户表的身份验证行为。

第七章 进阶系列

7.1. Artisan 控制台

7.1.1. 简介

Artisan 是框架自带的命令行接口。它提供了相当多的命令来帮助你构建应用。

你可以通过 `list` 命令查看所有可用的 Artisan 命令：

```
php artisan list
```

7.1.1.1. Tinker 命令(REPL)

所有应用都包含了 Tinker，一个由 PsySH 包提供的 REPL。

安装

所有应用默认包含了 Tinker。不过，你也可以使用 Composer 在你需要时手动安装。

```
composer require laravel/tinker
```

用法

Tinker 让你可以在命令行与你的整个应用进行交互。包括但不限于 Eloquent ORM、任务、事件等。

```
php artisan tinker
```

7.1.2. 编写命令

除了 Artisan 提供的命令之外，你也可以编写自己的自定义命令。命令在多数情况下位于 `app/Console/Commands` 目录中；不过，只要命令可以由 Composer 加载，你就可以自己选择自己的存储位置。

7.1.2.1. 生成命令

你可以使用 Artisan 命令 `make:command` 来创建一个新的命令。`make:command` 命令会在 `app/Console/Commands` 目录中创建一个新的命令类。如果该目录不存在，它会在你第一次运行 `make:command` 命令时自动创建。生成的命令将包含所有命令中默认存在的属性和方法。

```
php artisan make:command SendEmails
```

7.1.2.2. 命令结构

创建一个新命令后，你应该先修改 `signature` 和 `description` 属性以便你在使用 Artisan 命令 `list` 时能够清楚知道该命令的用法。执行命令时会调用 `handle` 方法，你可以在此方法中放置命令逻辑。

7.1.2.3. 闭包命令

基于闭包的命令提供了一个用类代替定义控制台命令的方法。同理，闭包路由也是控制器的一种替代方法，而闭包命令可以说是命令类的替代方法。在 `app/Console/Kernel.php` 文件中的 `commands` 方法中，Laravel 加载了

routes/console.php 文件:

```
/**
 * 注册闭包命令
 *
 * @return void
 */
protected function commands()
{
    require base_path('routes/console.php');
}
```

7.1.3. 定义期望输入

在编写控制台命令时，通常是通过参数和选项来收集用户输入的。Laravel 让你可以非常方便地在 `signature` 属性中定义你期望用户输入的内容。`signature` 属性允许使用单一且可读性高，类似路由的语法来定义命令的名称、参数和选项。

7.1.3.1. 参数

所有用户提供的参数及选项都被包含在花括号中。在下面的例子中，这个命令定义了一个必须的参数：`user`。

```
/**
 * 命令行的名称及签名
 *
 * @var string
 */
protected $signature = 'email:send {user}';
```

7.1.3.2. 选项

选项类似于参数，是用户输入的另一种格式。在命令行输入选项时，它们以两个连字符（--）作为前缀。有两种类型的选项：接收值和不接收值。不接收值的选项就像是一个布尔值的「开关」。让我们看一下这种类型选项的例子：

```
/**
 * 命令名称及签名
 *
 * @var string
 */
protected $signature = 'email:send {user} {--queue}';
```

在这个例子中，在调用 `Artisan` 命令时可以指定 `--queue` 的开关。如果传递了 `--queue` 选项，该选项的值将会是 `true`。否则，其值将会是 `false`。

7.1.3.3. 输入数组

如果你想定义接收数组输入的参数或选项，你可以使用 `*` 符号。首先，我们先看一个数组参数的实例。

```
email:send {user*}
```

7.1.3.4. 输入说明

你可以通过使用冒号来为参数和选项添加说明，并使其将他们隔开。如果你需要一点额外的空间来定义你的命令，可以随意分开到多行。

7.1.4. 命令 I/O

7.1.4.1. 检索输入

在命令执行时，显然你需要获取命令接收到的参数和选项的值。

7.1.4.2. 交互式输入

除了显示输出外，你还可以要求用户在命令执行时提供输入。

7.1.4.3. 编写输出

可以使用 `line` 、 `info` 、 `comment` 、 `question` 和 `error` 方法来将输出发送到终端。每个方法都使用适当的 ANSI 颜色表明其目的。

7.1.5. 注册命令

`app/Console/Commands` 目录下的命令都会被注册，这是由于控制台内核的 `commands` 方法调用了 `load`。

7.1.6. 通过代码调用命令

有时候您可能希望在 CLI 之外执行 Artisan 命令，比如，您可能希望在路由或控制器中触发 Artisan 命令。

```
Route::get('/foo', function () {

    $exitCode = Artisan::call('email:send', [

        'user' => 1, '--queue' => 'default'

    ]);

    //

});
```

7.2. 广播

7.2.1. 简介

在现代 Web 应用中，WebSockets 被用来实现实时、即时更新的用户接口。当服务器上数据被更新后，更新信息会通过 Websocket 连接发送给客户端处理。

相当于不停地轮询应用程序，这是一种更加可靠和高效的选择。

为了帮助你构建这类应用，Laravel 将通过 WebSocket 连接来使「广播」事件变得更加轻松。广播 Laravel 事件允许你在服务端和客户端 JavaScript 应用程序间共享相同的事件名。

7.2.1.1. 配置

所有关于事件广播的配置都保存在 `config/broadcasting.php` 配置文件中。Laravel 自带了几个广播驱动：Pusher Channels、Redis，和一个用于本地开发与调试的 log 驱动。另外，还有一个 null 驱动允许你完全关闭广播系统。每一个驱动的示例配置都可以在 `config/broadcasting.php` 配置文件中找到。

7.2.1.2. 对驱动的要求

在开始介绍广播事件之前，你必须先注册 `App\Providers\BroadcastServiceProvider`。对于一个新建的 Laravel 应用程序，你只需要在 `config/app.php` 配置文件的 `providers` 数组中取消该提供者的注释即可。该提供者将允许你注册广播授权路由和回调。

7.2.2. 概念概览

Laravel 的事件广播允许你使用基于驱动的 WebSockets 将服务端的 Laravel 事件广播到客户端的 JavaScript 应用程序。当前的 Laravel 自带了 Pusher 和 Redis 驱动。通过使用 Laravel Echo 的 Javascript 包，我们可以很方便地在客户端消费事件。

7.2.2.1. 使用示例程序

深入了解事件广播的每个组件之前，让我们先用一个电子商务网站作为例子来概览一下。我们不会讨论配置 Pusher 或者 Laravel Echo 的细节，这些会在本文档的其它章节里详细讨论。

在我们的应用程序中，我们假设有一个允许用户查看订单配送状态的页面。一个 `ShippingStatusUpdated` 事件会在配送状态更新时被触发：

```
event(new ShippingStatusUpdated($update));
```

7.2.3. 定义广播事件

要通知 Laravel 应该广播给定事件，并在该事件上实现 `Illuminate\Contracts Broadcasting\ShouldBroadcast` 接口。此接口已导入到框架生成的所有事件类中，因此你可以轻松地将其添加到任何事件中。

7.2.3.1. 广播名称

通常，Laravel 将通过事件类的名称广播该事件，但是，你也可以通过在事件中定义一个 `broadcastAs` 方法，来自定义事件名称：

```
/**
 * 广播事件名称
 *
 * @return string
 */
public function broadcastAs()
{
    return 'server.created';
}
```

7.2.3.2. 广播数据

当一个事件被广播时，它的所有 `public` 属性都会被自动序列化并作为事件的有效负载进行广播，这允许你从 JavaScript 应用程序访问它的任何公共数据。所以，例如，如果你的事件有一个包含 Eloquent 模型的公共 `$user` 属性，该事件广播载入内容将如下所示：

```
{
    "user": {
```

```
        "id": 1,

        "name": "Patrick Stewart"

        ...

    }

}
```

7.2.3.3. 广播队列

默认情况下，每个广播事件都会被推送到配置文件 `queue.php` 中指定的默认队列连接对应的默认队列中，您可以通过在事件类上定义一个 `broadcastQueue` 属性来自定义广播使用的队列。该属性需要指定广播时您想要使用的队列名称。

```
/**
 * 事件被推送的队列名称.
 *
 * @var string
 */
public $broadcastQueue = 'your-queue-name';
```

如果您想要使用 `sync` 队列而不是默认的队列驱动来广播事件，可以实现 `ShouldBroadcastNow` 接口来取代 `ShouldBroadcast`:

```
<?php

use Illuminate\Contracts\Broadcasting\ShouldBroadcastNow;

class ShippingStatusUpdated implements ShouldBroadcastNow
{
    //
}
```


7.2.3.4. 广播条件

有时候你需要在给定条件为 `true` 的时候才广播事件，你可以在事件类中增加一个 `broadcastWhen` 方法去定义这些条件：

```
/**
 * 确定事件是否要被广播
 *
 * @return bool
 */
public function broadcastWhen()
{
    return $this->value > 100;
}
```

7.2.4. 授权频道

私有频道往往需要授权验证过的用户才可以实际收听该频道。这是通过使用频道名称向你的 `Laravel` 应用程序发出 `HTTP` 请求，并允许你的应用程序确定用户是否可以收听该频道来实现的。使用 `Laravel Echo` 时，将自动进行授权订阅私人频道的 `HTTP` 请求；但是，你需要定义响应这些请求的正确路由。

7.2.4.1. 定义授权路由

庆幸的是，`Laravel` 可以轻松定义响应频道授权请求的路由。在 `Laravel` 应用程序附带的 `BroadcastServiceProvider` 中，你将看到对 `Broadcast::routes` 方法的调用。

7.2.4.2. 定义授权回调

接下来，我们需要定义实际执行频道授权的逻辑。这是在你的应用程序附带的 `routes/channels.php` 文件中完成的。

7.2.4.3. 定义频道类

如果你的应用程序使用了很多不同的频道，那么 `routes/channels.php` 文件将会变得很臃肿。所以，你可以使用频道类，而不是使用闭包来授权频道。要生成频道类，使用 `make:channel` Artisan 命令。

7.2.5. 广播事件

一旦你定义了一个事件并用 `ShouldBroadcast` 接口标记它，你只需要使用 `event` 函数来触发事件。事件调度程序将注意到事件标记为 `ShouldBroadcast` 接口，并将事件排队以进行广播：

```
event(new ShippingStatusUpdated($update));
```

7.2.5.1. 只广播给他人

在构建利用事件广播的应用程序时，你可以使用 `broadcast` 函数替换 `event` 函数。与 `event` 函数一样，`broadcast` 函数将事件调度到服务器端侦听器：

```
broadcast(new ShippingStatusUpdated($update));
```

7.2.6. 接收广播

7.2.6.1. 安装 Laravel Echo

Laravel Echo 是一个 JavaScript 库，可以轻松订阅频道并收听 Laravel 广播的事件。你可以通过 NPM 包管理器安装 Echo。在这个例子中，我们还将安装 `pusher-js` 包，因为我们将使用 Pusher Channels 广播器：

```
npm install --save laravel-echo pusher-js
```

7.2.6.2. 对事件进行监听

一旦安装并实例化了 **Echo**，就可以开始监听事件广播了。首先，使用 **channel** 方法检索频道的实例，然后调用 **listen** 方法来监听指定的事件：

```
Echo.channel('orders')

    .listen('OrderShipped', (e) => {

        console.log(e.order.name);

    });
```

7.2.6.3. 退出频道

要退出频道，你可以在 **Echo** 实例上调用 **leaveChannel** 方法：

```
Echo.leaveChannel('orders');
```

7.2.6.4. 命名空间

你可能已经在上面的示例中，注意到我们没有为事件类指定完整的命名空间。这是因为 **Echo** 会自动假设事件位于 **App\Events** 命名空间中。但是，你可以通过传递 **namespace** 配置选项来实例化 **Echo** 时配置根命名空间：

```
window.Echo = new Echo({

    broadcaster: 'pusher',

    key: 'your-pusher-channels-key',

    namespace: 'App.Other.Namespace'

});
```

7.2.7. Presence 频道

Presence 频道建立在私有频道安全性的基础上，同时暴露了谁订阅频道的附加特征。这样可以轻松构建功能强大的协作应用程序功能，例如在其他用户查看同一页面时通知用户。

7.2.7.1. 授权 Presence 频道

所有在线频道也是私有频道；因此，用户必须 被授权才能访问。但是，在为 Presence 频道定义授权回调时，如果用户有权加入该频道，则不会返回 `true`。相反，你应该返回有关用户的数据数组。

7.2.7.2. 加入 Presence 频道

加入 Presence 频道，你可以使用 Echo 的 `join` 方法。 `join` 方法将返回一个 `PresenceChannel` 实现，它与 `listen` 方法一起展示，允许你订阅 `here`，`joining` 和 `leaving` 事件。

7.2.7.3. 广播到 Presence 频道

Presence 频道可以像公共或私有频道一样接收事件。使用聊天室的示例，我们可能希望将 `NewMessage` 事件广播到房间的 Presence 频道。

7.2.8. 客户端事件

有时，你可能希望将事件广播到其他连接的客户端，而根本不需要使用 Laravel 应用程序。这对于诸如键入通知之类的内容特别有用，在这种情况下，你希望提醒应用程序的用户另一个用户正在给定屏幕上键入消息。

7.2.9. 消息通知

通过将事件广播与通知配对，你的 JavaScript 应用程序可以在发生时收到新通知，而无需刷新页面。首先，请务必阅读有关使用广播通知频道的文档。

配置通知以使用广播频道后，你可以使用 Echo 的 `notification` 方法监听广播事件。请记住，频道名称应与接收通知的实体的类名相匹配：

```
Echo.private(`App.User.${userId}`)

    .notification((notification) => {

        console.log(notification.type);

    });
```

7.3. 缓存

7.3.1. 配置

框架为各种后端提供了丰富而统一的 API。缓存配置位于 `config/cache.php`。在该文件中你可以指定在应用中默认使用哪个缓存驱动。开箱支持主流的缓存后端如 Memcached 和 Redis 等。

缓存配置文件还包含各种其他选项，这些选项都记录在文件中，因此请确保阅读这些选项。默认情况下，Laravel 配置为使用 `file` 缓存驱动，它将序列化的缓存对象存储在文件系统中。对于较大型应用，建议使用更强大的驱动程序，例如 Memcached 或 Redis。你甚至可以为同一个驱动程序配置多个缓存配置。

7.3.1.1. 驱动的前提条件

数据库

当使用 `database` 缓存驱动时，你需要配置一个表来存放缓存数据。下面是构建缓存数据表结构的 Schema 声明示例：

```
Schema::create('cache', function ($table) {  
  
    $table->string('key')->unique();  
  
    $table->text('value');  
  
    $table->integer('expiration');  
  
});
```

Memcached

使用 Memcached 驱动需要安装 Memcached PECL 扩展包。你可以把所有的 Memcached 服务器都列在 `config/cache.php` 配置文件中：

```
'memcached' => [  
  
    [  
  
        'host' => '127.0.0.1',  
  
        'port' => 11211,
```

```
        'weight' => 100

    ],

],
```

7.3.2. 缓存使用

7.3.2.1. 获取缓存实例

`Illuminate\Contracts\Cache\Factory` 和 `Illuminate\Contracts\Cache\Repository` 契约提供了访问缓存服务的方法。

`Factory` 契约提供了所有访问应用定义的缓存驱动的方法。

`Repository` 契约通常是应用中 `cache` 配置文件中指定的默认缓存驱动的一个实现。

7.3.2.2. 从缓存中获取数据

`Cache` 门面的 `get` 方法用于从缓存中获取缓存项，如果缓存项不存在，则返回 `null`。如果需要的话您可以传递第二个参数到 `get` 方法指定不存在时的自定义默认值。

```
$value = Cache::get('key');

$value = Cache::get('key', 'default');
```

您甚至可以传递一个闭包作为默认值，如果缓存项不存在的话闭包的结果将会被返回。

```
$value = Cache::get('key', function() {
    return DB::table(...)->get();
});
```

`Has` 方法用于判断缓存项是否存在，如果值为 `null` 或 `false` 该方法会返回 `false`。

7.3.2.3. 在缓存中存储数据

您可以使用 `Cache` 门面上的 `put` 方法在缓存中存储数据。

```
Cache::put('key', 'value', $seconds);
```

如果没有传递缓存时间到 `put` 方法，则缓存永久有效。

7.3.2.4. 从缓存中移除数据

您可以使用 `Cache` 门面上的 `forget` 方法从缓存中移除项数据。

```
Cache::forget('key');
```

还可以通过设置缓存有效期为 0 或负数来移除缓存项。

7.3.2.5. 原子锁

原子锁允许你对分布式锁进行操作而不必担心竞争条件。例如，`Laravel Forge` 使用原子锁来确保在一台服务器上每次只有一个远程任务在执行。你可以使用 `Cache::lock` 方法来创建和管理锁：

```
use Illuminate\Support\Facades\Cache;

$lock = Cache::lock('foo', 10);

if ($lock->get()) {

    // 获取锁定 10 秒...

    $lock->release();

}
```

7.3.2.6. 缓存辅助函数

除了可以使用 `Cache` 的门面以及 `Cache` 契约 外，你也可以使用全局辅助函数 `cache` 来获取和保存缓存数据。当只使用一个字符串参数调用 `cache` 函数时，这将返回给定键对应的值。

```
$value = cache('key');
```

7.3.3. 缓存标签

缓存标签目前不支持 file 或 database 缓存驱动。

7.3.3.1. 存储被打上标签的缓存数据

缓存标签允许你给相关的缓存标签项打上同一个标签以便后续可以清除这些缓存值。你可以通过传入一个被排序的标签数组来访问这些缓存项。例如，我们可以在使用标签的同时使用 `put` 方法来设置缓存。

```
Cache::tags(['people', 'artists'])->put('John', $john, $seconds);

Cache::tags(['people', 'authors'])->put('Anne', $anne, $seconds);
```

7.3.3.2. 访问被打上标签的缓存数据

若要获取一个被打上标签的缓存数据，将相同标签的有序数组传递给 `tags` 方法，然后调用 `get` 方法检索你要获取的键：

```
$john = Cache::tags(['people', 'artists'])->get('John');

$anne = Cache::tags(['people', 'authors'])->get('Anne');
```

7.3.3.3. 移除被打上标签的缓存数据

你可能需要移除单个标签或者一组标签所标记的所有缓存数据，例如，下面这个例子将会移除带有 `people, authors` 或者两者都有的标签的所有缓存数据，所以 `Anne` 和 `John` 将会从缓存中删除：

```
Cache::tags(['people', 'authors'])->flush();
```


7.3.4. 添加自定义缓存驱动

7.3.4.1. 编写驱动

要创建自定义的缓存驱动，首先需要实现 `Illuminate\Contracts\Cache\Store` 契约，所以，我们的 MongoDB 缓存实现看起来会像这样子。

```
<?php

namespace App\Extensions;

use Illuminate\Contracts\Cache\Store;

class MongoStore implements Store
{
    public function get($key) {}

    public function many(array $keys);

    public function put($key, $value, $seconds) {}

    public function putMany(array $values, $seconds);

    public function increment($key, $value = 1) {}

    public function decrement($key, $value = 1) {}

    public function forever($key, $value) {}

    public function forget($key) {}

    public function flush() {}

    public function getPrefix() {}
}
```

我们只需要使用一个 MongoDB 连接来实现其中的每个方法，实现完成后，我们就可以完成自定义驱动注册。

```
Cache::extend('mongo', function($app) {

    return Cache::repository(new MongoStore);

});
```

```
});
```

注：如果您在担心将自定义缓存驱动代码放到哪，可以在 **app** 目录下创建一个 **Extensions** 命名空间。

7.3.4.2. 注册驱动

要通过框架注册自定义的缓存驱动，可以使用 **Cache** 门面上的 **extend** 方法。

```
<?php

namespace App\Providers;

use App\Extensions\MongoStore;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\ServiceProvider;

class CacheServiceProvider extends ServiceProvider
{
    /**
     * Perform post-registration booting of services.
     *
     * @return void
     * @translator laravelacademy.org
     */
    public function boot()
    {
        Cache::extend('mongo', function($app) {
            return Cache::repository(new MongoStore);
        });
    }
}
```

```
}

/**
 * Register bindings in the container.
 *
 * @return void
 */
public function register()
{
    //
}
}
```

传递给 `extend` 方法的第一个参数是驱动名称。

7.3.5. 缓存事件

要在每次缓存操作时执行代码，您可以监听缓存触发的事件，通常，您可以将这些缓存处理程序代码到 `EventServiceProvider` 中。

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Cache\Events\CacheHit' => [
        'App\Listeners\LogCacheHit',
    ],
]
```

```
'Illuminate\Cache\Events\CacheMissed' => [
    'App\Listeners\LogCacheMissed',
],
'Illuminate\Cache\Events\KeyForgotten' => [
    'App\Listeners\LogKeyForgotten',
],
'Illuminate\Cache\Events\KeyWritten' => [
    'App\Listeners\LogKeyWritten',
],
];
```

7.4. 集合

7.4.1. 简介

`Illuminate\Support\Collection` 类为处理数组数据提供了流式、方便的封装。例如，查看下面的源码，我们使用辅助函数 `collect` 创建一个新的集合实例，为每一个元素运行 `strtoupper` 函数，然后移除所有空元素。

```
$collection = collect(['taylor', 'abigail', null])->map(function ($name)
{
    return strtoupper($name);
})->reject(function ($name) {
    return empty($name);
});
```

正如您所看到的，`Collection` 类允许您使用方法链对底层数组执行匹配和移除操作。

7.4.1.1. 创建集合

```
$collection = collect([1, 2, 3]);
```

7.4.1.2. 扩展集合

```
use Illuminate\Support\Str;

Collection::macro('toUpper', function () {

    return $this->map(function ($value) {

        return Str::upper($value);

    });

});

$collection = collect(['first', 'second']);

$upper = $collection->toUpper();

// ['FIRST', 'SECOND']
```

7.4.2. 集合方法

所有这些方法都可以以方法链的方式流式操作底层数组。

7.4.3. 方法列表

All 方法返回集合表示的底层数组。

Avg 方法返回所有集合项的平均值。

7.4.4. 高阶消息传递

集合还支持“高阶消息传递”，也就是在集合上执行通用的功能，支持高阶消息传递的方法包括。

7.4.5. 懒集合

为了继续完善功能已经很强大的 Collection 类，LazyCollection 类使用了 PHP 的生成器，从而可以通过极低的内存处理极大的数据集。

7.5. 事件

7.5.1. 简介

框架事件提供了一个简单的观察者实现，允许你在应用中订阅和监听各种发生的事件。事件类通常放在 `app/Events` 目录下，而这些事件类的监听器则放在 `app/Listeners` 目录下。如果在你的应用中你没有看到这些目录，不用担心，它们会在你使用 `Artisan` 控制台命令生成事件与监听器的时候自动创建。

事件是分离应用程序各个方面的好方法，因为单个事件可以拥有多个互不依赖的监听器。例如，你可能希望每次订单发货时向用户推送一个 `Slack` 通知。你可以简单地发起一个 `OrderShipped` 事件，而不是将订单处理代码耦合到 `Slack` 通知代码，监听器可以接收该事件并将其转换为 `Slack` 通知。

7.5.2. 注册事件/监听器

应用中的 `EventServiceProvider` 为注册所有的事件监听器提供了一个便利的场所。其中，`listen` 属性包含了所有事件（键）以及事件对应的监听器（值）的数组。当然，你可以根据应用的需要，添加多个事件到这个数组中。举个例子，我们添加一个 `OrderShipped` 事件：

```
/**
 * 应用程序的事件监听器映射
 *
 * @var array
 */
protected $listen = [
    'App\Events\OrderShipped' => [
        'App\Listeners\SendShipmentNotification',
    ],
];
```

7.5.2.1. 生成事件和监听器

当然，手动创建事件和监听器的文件是件麻烦事。而在这里，你只需要将监听器和事件添加到 `EventServiceProvider` 中，并使用 `event:generate` 命令。这个命令会生成在 `EventServiceProvider` 中列出的所有事件和监听器。当然，已经存在的事件和监听器将保持不变：

```
php artisan event:generate
```

7.5.2.2. 手动注册事件

通常，事件应该通过 `EventServiceProvider` 的 `$listen` 数组来注册的，然而，你也可以在 `EventServiceProvider` 的 `boot` 方法中手动的注册基于闭包的事件：

```
/**
 * 注册应用的任何其他事件
 *
 * @return void
 */
public function boot()
{
    parent::boot();

    Event::listen('event.name', function ($foo, $bar) {

        //

    });
}
```

7.5.2.3. 事件发现

你可以启用自动事件发现，而不是在 `EventServiceProvider` 的 `$listen` 数组中手动的注册事件和监听器。事件发现启用后，`Laravel` 会通过扫描你应用的 `Listeners` 目录来自动的查找和注册事件和监听器。此外，`EventServiceProvider` 中列出的任何明确定义的事件仍将被注册。

7.5.3. 定义事件

事件类是一个保存与事件相关信息的容器。例如，假设我们生成的 `OrderShipped` 事件接收一个 `Eloquent ORM` 对象。

```
<?php

namespace App\Events;

use App\Models\Order;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class OrderShipped
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public $order;
```



```
/**
 * 创建一个事件实例
 *
 * @param  \App\Models\Order  $order
 * @return void
 */
public function __construct(Order $order)
{
    $this->order = $order;
}
}
```

如你所见，这个事件类没有包含其他逻辑，它只是一个已购买的 `Order` 的实例的容器。

7.5.4. 定义监听器

接下来，让我们看一下例子中事件的监听器。事件监听器在 `handle` 方法中接收实例。`event:generate` 命令会自动加载正确的事件类，并且在 `handle` 方法中加入事件的类型提示。

7.5.5. 事件监听器队列

如果你的监听器中要执行诸如发送电子邮件或发出 `HTTP` 请求之类的耗时任务，你可以将任务丢给队列处理。在开始使用队列监听器之前，请确保在你的服务器或者本地开发环境中能够 配置队列 并启动一个队列监听器。

7.5.5.1. 手动访问队列

如果你需要手动访问监听器下面队列任务的 `delete` 和 `release` 方法，你可以通过使用 `Illuminate\Queue\InteractsWithQueue` trait 来实现。

7.5.5.2. 处理失败任务

有时，你的事件监听器队列可能会失败。如果监听器的队列任务超过了队列中定义的最大尝试次数，则会在监听器上调用 `failed` 方法。

7.5.6. 分发事件

要分发事件，你可以将事件实例传递给 `event` 辅助函数。该辅助函数将会把事件分发到所有该事件已注册的监听器上。

7.5.7. 事件订阅者

7.5.7.1. 编写事件订阅者

事件订阅者是指那些在类本身中订阅多个事件的类，通过事件订阅者您可以在单个类中定义多个事件处理器。

7.5.7.2. 注册事件订阅者

编写好订阅者后，就可以通过事件分发器对订阅者进行注册，您可以使用 `EventServiceProvider` 提供的 `$subscribe` 属性来注册订阅者。

7.6. 文件存储

7.6.1. 简介

框架基于 Frank de Jonge 开发的 PHP 包 `Flysystem` 提供了强大的文件系统抽象层。Laravel 集成 `Flysystem` 以便使用不同驱动简化对文件系统的操作。此外，这些存储选项之间切换非常简单，因为对不同系统而言，API 是一致的。

7.6.2. 配置

文件系统配置文件位于 `config/filesystems.php`。在该文件中可以配置所有“磁盘”，每个磁盘描述了特定的存储驱动和存储路径。

7.6.2.1. 公共磁盘

`public` 磁盘适用于要公开访问的文件。默认情况下，`public` 磁盘使用 `local` 驱动，并且将这些文件存储在 `storage/app/public` 目录下。为了使它们能通过网络访问，你需要创建 `public/storage` 到 `storage/app/public` 的符号链接。这种方式能把可公开访问文件都保留在同一个目录下，以便在使用零停机时间部署系统如 `Envoyer` 的时候，就可以轻松地在不同的部署之间共享这些文件。

7.6.2.2. 本地驱动

使用 local 驱动时，所有文件操作都与 filesystems 配置文件中定义的 root 目录相关。默认情况下，此值设置为 storage/app 目录。

7.6.2.3. 驱动程序先决条件

Composer 包

在使用 SFTP、S3 驱动之前，你需要通过 Composer 安装相应的软件包：

SFTP: league/flysystem-sftp ~1.0

Amazon S3: league/flysystem-aws-s3-v3 ~1.0

使用缓存适配器是提高性能的一个绝对必要条件。你需要一个额外的包：

CachedAdapter: league/flysystem-cached-adapter ~1.0

7.6.2.4. 缓存

给指定磁盘开启缓存功能，需要在该磁盘的配置项中直接添加 cache 。

7.6.3. 获取磁盘实例

我们可以使用 Storage 门面和上面配置的任意磁盘进行交互。

7.6.4. 检索文件

get 方法可以用于检索文件的内容，此方法返回该文件的原始字符串内容。切记，所有文件路径的指定都应该相对于为磁盘配置的「root」目录：

```
$contents = Storage::get('file.jpg');
```

7.6.4.1. 下载文件

download 方法可用于生成一个响应，强制用户的浏览器在给定路径下载文件。download 方法接受一个文件名作为该方法的第二个参数，它将确定用户下载文件时看到的文件名。最后，你可以传递一个 HTTP 数组头作为该方法的第三个参数：

```
return Storage::download('file.jpg');
```

```
return Storage::download('file.jpg', $name, $headers);
```

7.6.4.2. 文件 URLs

你可以使用 `url` 方法来获取给定文件的 URL。如果你使用的是 `local` 驱动，一般只是在给定的路径上加上 `/storage` 并返回一个相对的 URL 到那个文件。如果使用的是 `s3` 或者是 `rackspace` 驱动，会返回完整的远程 URL：

```
use Illuminate\Support\Facades\Storage;

$url = Storage::url('file.jpg');
```

7.6.4.3. 文件元数据

除了读写文件外，**Laravel** 还可以提供有关文件本身的信息，例如，`size` 方法可用来获取文件的大小（以字节为单位）：

```
use Illuminate\Support\Facades\Storage;

$size = Storage::size('file.jpg');
```

7.6.5. 存储文件

`put` 方法可用于将原始文件内容保存到磁盘上。你也可以传递 **PHP** 的 `resource` 给 `put` 方法，它将使用文件系统下的底层流支持。强烈建议在处理大文件时使用此方法：

```
use Illuminate\Support\Facades\Storage;

Storage::put('file.jpg', $contents);

Storage::put('file.jpg', $resource);
```

7.6.5.1. 文件上传

在 web 应用程序中，最常用到的文件存储的场景的地方就是上传头像，照

片和文件。Laravel 上传文件的实例方法 `store` 可以轻松的处理文件上传存储问题。你只需要调用带有文件保存路径作为参数的 `store` 方法即可：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class UserAvatarController extends Controller
{
    /**
     * 更新用户头像
     *
     * @param Request $request
     * @return Response
     */
    public function update(Request $request)
    {
        $path = $request->file('avatar')->store('avatars');

        return $path;
    }
}
```

7.6.5.2. 文件可见性

在 Laravel 集成的文件系统中，「可见性」是一个对多个平台的文件权限的抽象概念。文件可以声明为 `public` 或 `private`。如果一个文件被声明为 `public`，意味着其他人可以访问。例如，使用 S3 驱动时，就可以检索声明为 `public` 的文件。

你可以通过 `put` 方法设置文件的可见性：

```
use Illuminate\Support\Facades\Storage;

Storage::put('file.jpg', $contents, 'public');
```

7.6.6. 删除文件

`delete` 方法接收一个文件名或数组形式的文件名来删除磁盘上的文件：

```
use Illuminate\Support\Facades\Storage;

Storage::delete('file.jpg');

Storage::delete(['file.jpg', 'file2.jpg']);
```

7.6.7. 目录

获取目录下的所有的文件

`files` 方法返回指定目录下的所有文件。如果你想检索指定目录（包括子目录）中所有的文件列表，可以使用 `allFiles` 方法：

```
use Illuminate\Support\Facades\Storage;

$files = Storage::files($directory);

$files = Storage::allFiles($directory);
```

7.6.8. 自定义文件系统

框架的 Flysystem 集成提供了多外“驱动”，不过 Flysystem 的功能并不止步于此，还为许多其他存储系统提供了适配器。

7.7. 辅助函数

7.7.1. 简介

框架自带了一系列 PHP 辅助函数，很多被框架自身使用，如果您觉得方便的话也可以在代码中使用它们。

7.7.2. 方法列表

`Arr::add` 方法添加给定键值对到数组。

7.8. HTTP 客户端

7.8.1. 简介

Laravel 为 Guzzle HTTP 客户端 提供了一种语义化且轻量的 API，让你可以快速地使用 HTTP 请求与其他 Web 应用进行通信。该 API 专注于其在常见用例中的快速实现以及良好的开发者体验。

7.8.2. 创建请求

你可以通过 `get`、`post`、`put`、`patch` 和 `delete` 方法来创建请求。首先，让我们先看一下如何发出一个基础的 GET 请求：

```
use Illuminate\Support\Facades\Http;

$response = Http::get('http://test.com');
```

`get` 方法返回一个 `Illuminate\Http\Client\Response` 的实例，该实例提供了大量的方法来检查请求的响应：

```
$response->body() : string;

$response->json() : array;
```

```
$response->status() : int;  
  
$response->ok() : bool;  
  
$response->successful() : bool;  
  
$response->serverError() : bool;  
  
$response->clientError() : bool;  
  
$response->header($header) : string;  
  
$response->headers() : array;
```

7.8.2.1. 请求数据

大多数情况下, POST、PUT 和 PATCH 携带着额外的请求数据是相当常见的。所以, 这些方法的第二个参数接受一个包含着请求数据的数组。默认情况下, 这些数据会使用 `application/json` 类型随请求发送。

```
$response = Http::post('http://test.com/users', [  
    'name' => 'Steve',  
    'role' => 'Network Administrator',  
]);
```

7.8.2.2. 请求头

你可以通过 `withHeaders` 方法添加请求头。该方法接受一个数组格式的键值对。

```
$response = Http::withHeaders([  
    'X-First' => 'foo',  
    'X-Second' => 'bar'  
])->post('http://test.com/users', [
```



```
        'name' => 'Taylor',  
    ]);
```

7.8.2.3. 认证

你可以使用 `withBasicAuth` 和 `withDigestAuth` 方法来分别指定使用 `basic` 或是 `digest` 认证方式：

```
// Basic 认证...  
  
$response = Http::withBasicAuth('taylor@laravel.com',  
'secret')->post(...);  
  
// Digest 认证...  
  
$response = Http::withDigestAuth('taylor@laravel.com',  
'secret')->post(...);
```

7.8.2.4. 重试

如果你希望你的 HTTP 客户端在发生错误时自动重新发送请求，你可以使用 `retry` 方法。该方法接受两个参数：重新尝试次数以及重试等待时间（毫秒）：

```
$response = Http::retry(3, 100)->post(...);
```

7.8.2.5. 错误处理

跟 `Guzzle` 的默认行为不同，`Laravel HTTP` 客户端并不会在客户端或服务端错误时抛出异常（400 及 500 状态码）。你可以通过 `successful`、`clientError` 或是 `serverError` 方法来判断是否发生错误：

```
// 确认状态码是否在 200 到 300 之间（包含 200）  
  
$response->successful();
```

```
// 确认是否发生了 400 级别的错误（以 4 开头的状态码）

$response->clientError();

// 确认是否发生了 500 级别的错误（以 5 开头的状态码）

$response->serverError();
```

7.8.3. 测试

许多 Laravel 服务都可以让你非常简单且语义化地编写测试，Laravel HTTP 客户端也不例外。Http Facade 的 Fake 方法允许你让 HTTP 客户端在请求创建时返回虚拟的响应。

7.8.3.1. 虚拟响应

如果你希望 HTTP 客户端为每个请求返回空的 200 响应，你可以不带任何参数地调用 fake 方法：

```
use Illuminate\Support\Facades\Http;

Http::fake();

$response = Http::post(...);
```

7.8.3.2. 检查请求

在伪造响应时，你可能希望检查客户端收到的请求，以确保你的应用发送了正确的数据或请求头。你可以在调用 Http::fake 方法后调用 Http::assertSent 来完成该操作。

7.9. 邮件

7.9.1. 简介

框架基于 SwiftMailer 库提供了一套干净、清爽的邮件 API，可以为 SMTP、Mailgun、Postmark、Amazon SES 和 sendmail 提供驱动，让你可以快速从本地或云端服务自由地发送邮件。

7.9.1.1. 配置

Laravel 的 email 服务可以通过 mail 配置文件进行配置。这个文件中配置地每个邮件程序都可能地配置项，甚至是独有的传输方式，这将允许你的应用程序使用不同的邮件服务来发送特定的邮件。例如，你的应用程序可能使用 Postmark 发送事务性邮件，而使用 Amazon SES 发送批量邮件。

7.9.1.2. 驱动前提

基于 API 的驱动，比如 Mailgun 和 Postmark 通常比 SMTP 服务器更简单快速。如果可以的话，你应该尽可能使用这些驱动。所有的 API 驱动都需要 Guzzle HTTP 函数库，这个函数库可以通过 Composer 包管理安装：

```
composer require guzzlehttp/guzzle
```

7.9.2. 生成可邮寄类

在框架中，应用发送的每一封邮件都可以表示为“可邮寄”类，这些类都存放在 app/Mail 目录。如果你的应用中没有该目录，别慌，当你使用 make:mail 命令生成你的首个 maiable 类时，应用将会自动创建它。

7.9.3. 编写可邮寄类

所有的可邮寄类配置都在 build 方法中完成，在这个方法中，您可以调用多个方法，例如 from、subject、view 和 attach 来配置邮件的内容和发送。

7.9.3.1. 配置发件人

首先，我们来看一下邮件发件人的配置。

7.9.3.2. 配置视图

您可以在可邮寄类的 `build` 方法中使用 `view` 方法来指定渲染邮件内容时使用哪个视图模板，由于每封邮件通常使用 `Blade` 模板来渲染内容，所以您可以在构建邮件 HTML 时使用 `Blade` 模板引擎提供的所有功能。

7.9.3.3. 视图数据

通常，我们需要传递一些数据到渲染邮件的 HTML 视图以供使用。有两种方式将数据传递到视图，第一种是可邮寄类的公共（`public`）属性在视图中自动生效。

7.9.3.4. 附件

要添加附件以邮件，可以在可邮寄类的 `build` 方法中使用 `attach` 方法。

7.9.3.5. 内部附件

要嵌入内联图片，在邮件视图中使用 `$message` 变量上的 `embed` 方法即可。

7.9.3.6. 自定义 `SwiftMailer` 消息

`Mailable` 基类上的 `withSwiftMessage` 方法允许您注册一个在发送消息之前可以被原生 `SwiftMailer` 消息实例调用的回调。

7.9.4. Markdown 可邮寄类

Markdown 邮件消息允许您在可邮寄类中利用预置模板和邮件通知组件。因为这些消息以 Markdown 格式编写，框架还可以为它们渲染出高颜值、响应式的 HTML 模板，同时自动生成纯文本的副本。

7.9.5. 发送邮件

要发送一条信息，可以使用 `Mail` 门面上的 `to` 方法。`to` 方法接收邮箱地址、用户实例或用户集合作为参数。如果传递的是对象或对象集合，在设置邮件收件人的时候邮件会自动使用它们的 `email` 和 `name` 属性。

7.9.6. 渲染可邮寄类

有时可能希望捕获 `mailable` 的 HTML 内容，而不发送它。可以调用 `mailable` 的 `render` 方法实现此目的。此方法返回 `mailable` 渲染计算后的字符串：

```
$invoice = App\Invoice::find(1);

return (new App\Mail\InvoicePaid($invoice))->render();
```

7.9.7. 本地化可邮寄类

Laravel 允许你使用有别于当前语言的区域设置发送 `mailable`，即使被加入到队列中也保留该区域设置。

7.9.8. 邮件&本地开发

在开发发送邮件的应用时，你也许不想真的向实时邮件地址发送邮件。Laravel 为本地开发期间提供了几个「禁用」真实发送的途径

7.9.9. 事件

框架会在发送邮件消息前触发两个事件，`MessageSending` 事件在消息发送前触发，`MessageSent` 事件在消息发送后触发。

切记，这些事件是在邮件被 发送 时触发，而不是在队列化的时候。可以在 `EventServiceProvider` 中注册此事件的侦听器：

```
/**
 * 为应用映射事件侦听器。
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Mail\Events\MessageSending' => [
        'App\Listeners\LogSendingMessage',
    ],
    'Illuminate\Mail\Events\MessageSent' => [
```



```
use Illuminate\Foundation\Auth\User as Authenticatable;

use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;
}
```

这个 trait 由预设的 `App\Models\User` 模型使用，它含有一个可以用来发送通知的方法：`notify`。`notify` 方法需要接收一个通知实例参数：

```
use App\Notifications\InvoicePaid;

$user->notify(new InvoicePaid($invoice));
```

7.10.3.2. 使用 Notification Facade

另外，你可以通过 `Notification facade` 来发送通知，它主要用在当你给多个可接收通知的实体发送的时候，比如给用户集合发送通知。使用 `Facade` 发送通知的话，要把可以接收通知和通知的实例传递给 `send` 方法：

```
Notification::send($users, new InvoicePaid($invoice));
```

7.10.4. 邮件通知

7.10.4.1. 格式化邮件消息

如果通知支持以邮件方式发送，你需要在通知类上定义一个 `toMail` 方法。该方法会接收一个 `$notifiable` 实体并返回 `Illuminate\Notifications\Messages\MailMessage` 实例。邮件消息可以包含多行文本

以及对「动作的调用」。让我们来看一个 `toMail` 方法的示例:

```
/**
 * 获取通知对应的邮件.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    $url = url('/invoice/'. $this->invoice->id);

    return (new MailMessage)
        ->greeting('Hello!')
        ->line('One of your invoices has been paid!')
        ->action('View Invoice', $url)
        ->line('Thank you for using our application!');
}
```

7.10.4.2. 自定义发送者

默认情况下, 电子邮件的发件人 / 发件人地址 在 `config/mail.php` 配置文件中定义。但是, 你可以使用 `from` 方法指定特定通知的发件人地址:

```
/**
 * Get the mail representation of the notification.
 *

```



```
* @param mixed $notifiable

* @return \Illuminate\Notifications\Messages\MailMessage

*/

public function toMail($notifiable)

{

    return (new MailMessage)

        ->from('test@example.com', 'Example')

        ->line('...');

}
```

7.10.4.3. 自定义接收者

通过 mail 通道发送通知时，通知系统会自动在被通知实体上查找 email 属性，你可以通过在该实体上定义一个 routeNotificationForMail 来自定义使用哪个邮箱地址发送通知。

```
<?php

namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable

{

    use Notifiable;
```

```
/**
 * 邮件通道通知的路由。
 *
 * @param  \Illuminate\Notifications\Notification
$notification
 * @return array|string
 */
public function routeNotificationForMail($notification)
{
    // 只返回邮件地址...

    return $this->email_address;

    // 返回名字和邮件地址...

    return [$this->email_address => $this->name];
}
}
```

7.10.4.4. 自定义主题

默认情况下，邮件的主题就是格式为「标题风格」的通知类名，因此，如果通知类被命名为 `InvoicePaid`，邮件的主题就是 `Invoice Paid`，如果你想要为消息指定明确的主题，可以在构建消息的时候调用 `subject` 方法。

7.10.5. Markdown 邮件通知

Markdown 邮件通知允许你利用邮件通知的预置模板，从而让你可以自由编

写更长、更具个性化的消息。因为这些消息以 Markdown 格式编写，Laravel 还可以为它们渲染出高颜值、响应式的 HTML 模板，同时自动生成纯文本的副本。

7.10.5.1.生成消息

要生成带有相应 Markdown 模板的通知，可以在使用 Artisan 命令 `make:notification` 时带上 `--markdown` 选项：

```
php artisan make:notification InvoicePaid
--markdown=mail.invoice.paid
```

7.10.5.2.编写消息

Markdown 邮件通知联合使用了 Blade 组件和 Markdown 语法，从而让你在不脱离 Laravel 预置组件的情况下轻松构建通知：

```
@component('mail::message')

# Invoice Paid


Your invoice has been paid!


@component('mail::button', ['url' => $url])

View Invoice

@endcomponent


Thanks, <br>

{{ config('app.name') }}

@endcomponent
```

7.10.5.3. 自定义组件

你可以导出所有 Markdown 通知组件到应用中进行自定义，要导出组件，使用 Artisan 命令 `vendor:publish` 来发布 `laravel-mail` 资源标签：

```
php artisan vendor:publish --tag=laravel-mail
```

该命令会发布 Markdown 邮件通知组件到 `resources/views/vendor/mail` 目录。mail 目录包含 `html` 和 `markdown` 目录，每个子目录中又包含各自的所有有效组件。你可以按照自己的喜好自由编辑这些组件。

7.10.6. 数据库通知

7.10.6.1. 先决条件

`database` 通知通道会在数据表中存储通知信息，该表包含诸如通知类型以及用于描述通知的自定义 JSON 数据之类的信息。

你可以在用户界面中查询这个数据表来展示通知，不过，在此之前，需要创建数据表来保存信息，你可以使用 `notifications:table` 命令来生成迁移文件然后以此生成相应数据表：

```
php artisan notifications:table

php artisan migrate
```

7.10.6.2. 格式化数据库通知

如果要将通知消息存入到数据库表中，就需要为通知类定义 `toDatabase` 或 `toArray` 方法。此方法接收 `$notifiable` 实体作为参数并返回原生 PHP 数组。返回的数据将被编码为 JSON 格式并存储到 `notifications` 表的 `data` 列。来看一个 `toArray` 方法示例：

```
/**
 * Get the array representation of the notification.
 *
 */
```

```
* @param mixed $notifiable
* @return array
*/
public function toArray($notifiable)
{
    return [
        'invoice_id' => $this->invoice->id,
        'amount' => $this->invoice->amount,
    ];
}
```

7.10.6.3.访问通知

一旦通知存入数据库，就需要用适当的方法在通知实体里访问它们。在 Laravel 的默认 `App\User` 模型带有的 `Illuminate\Notifications\Notifiable` trait 里，它的 `notifications Eloquent` 关联方法能返回实体通知。要获取通知，可以像其它 Eloquent 关联方法一样访问此方法。默认情况下，通知按照 `created_at` 时间戳排序：

```
$user = App\User::find(1);

foreach ($user->notifications as $notification) {
    echo $notification->type;
}
```

7.10.6.4.标记通知已读

通常，在用户阅览一条通知之后，你会想将其标识为「已读」。Illuminate\Notifications\Notifiable trait 提供了 markAsRead 方法，它更新数据库中通知记录的 read_at 列：

```
$user = App\User::find(1);

foreach ($user->unreadNotifications as $notification) {

    $notification->markAsRead();

}
```

7.10.7. 广播通知

7.10.7.1. 先决条件

广播通知前，你需要配置和熟悉 Laravel 的事件广播服务。事件广播提供了在 JavaScript 客户端响应服务端触发 Laravel 事件的方法。

7.10.7.2. 格式化广播通知

broadcast 通道使用 Laravel 的 event broadcasting 服务广播通知，它允许 JavaScript 客户端实时捕获通知。如果通知支持广播，你就需要在通知类上定义 toBroadcast 方法。此方法接收 \$notifiable 实体作为参数，并返回 BroadcastMessage 实例。如果不存在 toBroadcast 方法，则将使用 toArray 方法来收集广播的数据。返回的数据将被编码为 JSON 并广播给 JavaScript 客户端。

7.10.7.3. 监听通知

通知将会以格式为 {notifiable}.{id} 的形式在私有频道上广播，因此，如果你要发送通知到 ID 为 1 的 App\User 实例，那么该通知将会在私有频道 App.User.1 上进行广播，如果使用了 Laravel Echo，可以使用辅助函数 notification 轻松在某个频道上监听通知：

```
Echo.private('App.User.' + userId)

    .notification((notification) => {
```

```
        console.log(notification.type);  
    });
```

7.10.8. 短信

7.10.8.1. 先决条件

Laravel 基于 Nexmo 发送短信通知，在使用 Nexmo 发送通知前，需要安装对应 Composer 依赖包 `laravel/nexmo-notification-channel`：

```
composer require laravel/nexmo-notification-channel
```

7.10.8.2. 格式化短信通知

如果通知支持以短信方式发送，那么你需要在通知类上定义一个 `toNexmo` 方法。该方法接收一个 `$notifiable` 实体并返回 `Illuminate\Notifications\Messages\NexmoMessage` 实例：

```
/**  
 * Get the Nexmo / SMS representation of the notification.  
 *  
 * @param mixed $notifiable  
 * @return NexmoMessage  
 */  
public function toNexmo($notifiable)  
{  
    return (new NexmoMessage)  
        ->content('Your SMS message content');  
}
```

7.10.8.3. 格式化简码通知

Laravel 还支持发送短代码通知，这是你 Nexmo 帐户中的预定义消息模板。你可以指定通知的类型 (`alert`, `2fa`, or `marketing`) 以及将填充模板的自定义值：

```
/**
 * Get the Nexmo / Shortcode representation of the notification.
 *
 * @param mixed $notifiable
 * @return array
 */
public function toShortcode($notifiable)
{
    return [
        'type' => 'alert',
        'custom' => [
            'code' => 'ABC123',
        ],
    ];
}
```

7.10.9. Slack 通知

7.10.9.1. 先决条件

在通过 Slack 发送通知前，必须通过 Composer 安装 Slack 通知通道：

```
composer require laravel/slack-notification-channel
```


7.10.9.2. 格式化 Slack 通知

如果通知支持通过 Slack 消息发送，则需要在通知类上定义一个 `toSlack` 方法，该方法接收一个 `$notifiable` 实体并返回 `Illuminate\Notifications\Messages\SlackMessage` 实例，该实例包含文本内容以及格式化额外文本或数组字段的“附件”。

7.10.9.3. Slack 附件

你还可以添加“附件”到 Slack 消息。相对简单文本消息，附件可以提供更加丰富的格式选择。

7.10.10. 本地化通知

Laravel 允许您以当前语言环境之外的其他语言发送通知，并且会在通知队列时记住该语言环境。

要实现这一点，`Illuminate\Notifications\Notification` 类提供了一个 `locale` 方法来设置所需的语言。

7.10.10.1. 用户首先语言区域设置

有些情况下，应用程序保存了每个用户的首选语言区域设置。通过在模型上实现 `HasLocalePreference` 契约，可以指定 Laravel 在发送通知时使用用户保存的首选语言设置：

7.10.11. 通知事件

当通知被发送后，通知系统会触发 `Illuminate\Notifications\Events\NotificationSent` 事件，该事件实例包含被通知的实体（如用户）和通知实例本身。你可以在 `EventServiceProvider` 中为该事件注册监听器：

```
/**  
  
 * The event listener mappings for the application.
```

```
*  
  
* @var array  
  
*/  
  
protected $listen = [  
  
    'Illuminate\Notifications\Events\NotificationSent' => [  
  
        'App\Listeners\LogNotification',  
  
    ],  
  
];
```

7.10.12. 自定义通道

Laravel 为我们提供了多种通知通道，但是尝试编写自定义通道驱动以通过其他通道发送通知，也很简单。首先定义一个包含 `send` 方法的类，该方法接收两个参数：`$notifiable` 和 `$notification`：

```
<?php  
  
  
namespace App\Channels;  
  
  
use Illuminate\Notifications\Notification;  
  
  
class VoiceChannel  
{  
  
    /**  
  
     * 发送指定的通知.  
  
     */  
  
}
```

```
        * @param mixed $notifiable

        * @param \Illuminate\Notifications\Notification
$notification

        * @return void

        */

    public function send($notifiable, Notification $notification)
    {

        $message = $notification->toVoice($notifiable);

        // Send notification to the $notifiable instance...

    }

}
```

7.11. 扩展包开发

7.11.1. 简介

扩展包是向 Laravel 添加功能的主要方式。扩展包可以包含很多有用的功能，比如时间处理扩展包 **Carbon**，亦或者是提供完整 BDD 测试框架的扩展包 **Behat**。

扩展包有很多的类型。有些扩展包是独立的，这意味着它们可以和任何 PHP 框架一起使用。**Carbon** 和 **Behat** 就是这样的独立扩展包。要在 Laravel 中使用这些扩展包只需要在 **composer.json** 文件中引入它们即可。

另外，有些扩展包只能在 Laravel 中使用，这些扩展包可能包含专门用于增强 Laravel 应用的路由、控制器、视图和配置文件。本指南主要介绍 Laravel 扩展包的开发。

7.11.1.1.Facades 注解

当开发 Laravel 应用时，通常使用契约（ `contracts` ）或 `facades` 都可以，因为它们都提供基本相同的可测试能力。但是，在开发扩展包时，扩展包并不能访问 Laravel 提供的所有测试辅助函数。如果你希望能够像在 Laravel 应用中一样编写扩展包的测试用例，你可以使用扩展包 `Orchestral Testbench` 。

7.11.2. 包自动发现

在 Laravel 应用的 `config/app.php` 配置文件中， `providers` 选项定义了能够被 Laravel 加载的服务提供者列表。当有人安装你的扩展包时，通常会希望此列表中包含你的服务提供者。你可以在扩展包的 `composer.json` 文件中的 `extra` 部分定义服务提供者，而不是让用户手动将你的服务提供者添加到列表中。除了服务提供者，你还可以列出你想要注册的所有 `facades`：

```
"extra": {
    "laravel": {
        "providers": [
            "Barryvdh\\Debugbar\\ServiceProvider"
        ],
        "aliases": {
            "Debugbar": "Barryvdh\\Debugbar\\Facade"
        }
    }
},
```

将扩展包配置为可发现之后，框架将在安装时自动注册扩展包的服务提供者和 `facades`，从而为扩展包的用户提供便利的安装体验。

7.11.2.1.选择性的发现扩展包

如果你是扩展包的使用者，想要禁止一个扩展包的发现，你可以在应用的

`composer.json` 文件中的 `extra` 部分列出这个扩展包：

```
"extra": {
    "laravel": {
        "dont-discover": [
            "barryvdh/laravel-debugbar"
        ]
    }
},
```

你也可以在应用的 `dont-discover` 指令中使用 `*` 字符，禁用所有扩展包的发现功能：

```
"extra": {
    "laravel": {
        "dont-discover": [
            "*"
        ]
    }
},
```

7.11.3. 服务提供者

服务提供者 将你的扩展包和 `Laravel` 联系在一起。服务提供者负责将事物绑定到 `Laravel` 服务容器 中，并告诉 `Laravel` 从哪里加载扩展包的资源文件，例如视图、配置文件、语言包等。

服务提供者继承了 `Illuminate\Support\ServiceProviders` 类，并包含两个方法：`register` 和 `boot`。基类 `ServiceProvider` 位于 `Composer` 扩展包的 `illuminate/support` 中，你必须将它添加到你的扩展包依赖项中。要了解有关服务

提供者的结构和用途的更多信息，请查阅它的文档。

7.11.4. 资源

7.11.4.1. 配置

通常，需要发布的扩展包配置文件到应用根目录下的 `config` 目录。这样使用扩展包的用户就可以轻松的重写默认配置项。要发布配置文件，只需要在服务提供者的 `boot` 方法中调用 `publishes` 方法：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    $this->publishes([
        __DIR__.' /path/to/config/courier.php' =>
        config_path('courier.php'),
    ]);
}
```

7.11.4.2. 路由

如果扩展包包含路由文件，可以使用 `loadRoutesFrom` 方法加载它们，这个方法会自动判定应用的路由是否被缓存，如果路由已经被缓存将不会加载路由文件。

```
/**
 * Perform post-registration booting of services.
```

```
*  
  
* @return void  
  
*/  
  
public function boot()  
{  
  
$this->loadRoutesFrom(__DIR__.'/routes.php');  
  
}
```

7.11.4.3. 迁移

如果您的扩展包包含数据库迁移,可以使用 `LoadMigrationsFrom` 方法告知框架如何加载它们。`loadMigrationsFrom` 方法只需要扩展包迁移文件路径作为唯一参数:

```
/**  
  
* 在注册后启动服务  
  
*  
* @return void  
  
*/  
  
public function boot()  
{  
  
    $this->loadMigrationsFrom(__DIR__.'/path/to/migrations');  
  
}
```

7.11.4.4.语言包

如果您的扩展包包含翻译文件，您可以使用 `LoadTranslationsFrom` 方法告诉框架如何加载它们。例如，如果你的扩展包名为 `courier`，你需要将下面的内容加入到服务提供者的 `boot` 方法中：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    $this->loadTranslationsFrom(__DIR__.'/path/to/translations',
'courier');
}
```

7.11.4.5.视图

想要在 Laravel 中注册你的扩展包的 视图 ， 需要告知 Laravel 视图文件的位置。 你可以使用服务提供者的 `loadViewsFrom` 方法来实现。`loadViewsFrom` 方法接收两个参数：视图模板的路径和扩展包名。例如，如果你的扩展包名为 `courier`，你需要将下面的内容加入到服务提供者的 `boot` 方法中：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
```



```
public function boot()

{

    $this->loadViewsFrom(__DIR__.'/path/to/views', 'courier');

}
```

7.11.4.6. 视图组件

如果你的扩展包中包含视图组件，你需要使用 `loadViewComponentsAs` 方法告知 Laravel 如何加载它们。`loadViewComponentsAs` 方法接受两个参数：视图组件的标签前缀和视图组件类组成的数组。例如，如果包的前缀是 `courier`，并且具有 `Alert` 和 `Button` 视图组件，则可以将以下内容添加到服务提供者的 `boot` 方法中：

```
/**
 * 启动应用服务
 *
 * @return void
 */
public function boot()
{

    $this->loadViewComponentsAs('courier', [

        Alert::class,

        Button::class,

    ]);

}
```

一旦在服务提供者中注册了视图组件，就可以直接在视图中引用它们，如下所示：

```
<x-courier-alert />

<x-courier-button />
```

7.11.5. 命令

想要在 Laravel 中注册扩展包的 Artisan 命令，需要使用 `commands` 方法。此方法接收一个命令类的数组，一旦这些命令被注册成功，你可以使用 Artisan 命令行 执行他们：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    if ($this->app->runningInConsole()) {
        $this->commands([
            FooCommand::class,
            BarCommand::class,
        ]);
    }
}
```

7.11.6. 发布前端资源

你的扩展包可能包含 JavaScript、CSS 和图片之类的资源文件。要将这些资源发布到应用的 `public` 目录，可以使用服务提供者的 `publishes` 方法。在下面的例子中，我们也可以添加一个 `public` 资源组标签，该标签可用于发布相关资源组：

```
/**
 * 在注册后启动服务
 *
 * @return void
 */
public function boot()
{
    $this->publishes([
        __DIR__.' /path/to/assets' => public_path('vendor/courier'),
    ], 'public');
}
```

7.11.7. 发布文件组

你可能想要分别发布扩展包资源文件和资源。举个例子，你想要用户只发布扩展包的配置文件，而不是被强制发布扩展包中的资源文件。你可以通过调用服务提供者中 `publishes` 方法时对他们打上「标签」。例如，让我们使用扩展包服务提供者中的 `boot` 方法来定义两个发布群组：

```
/**
 * 在注册后启动服务
 *
```

```
* @return void

*/

public function boot()
{
    $this->publishes([
        __DIR__.'../config/package.php' => config_path('package.php')
    ], 'config');

    $this->publishes([
        __DIR__.'../database/migrations/' =>
database_path('migrations')
    ], 'migrations');
}
```

7.12. 队列

7.12.1. 简介

队列提供了可以跨不同队列后台的统一 API，例如 Beanstalk、Amazon SQS、Redis 甚至关系数据库。通过队列，你可以将耗时任务 (如发送电子邮件) 的处理往后推延。延迟这些耗时的任务可以极大地提升 web 请求响应速度。

队列配置文件存储在 `config/queue.php` 中。在这个文件中，你可以找到框架中包含的每个队列驱动程序连接配置，其中包括数据库、Beanstalkd，Amazon SQS，Redis，和一个同步驱动程序(供本地使用)。还包括一个用于丢弃排队任务的 `null` 队列驱动。

7.12.1.1. 连接 Vs 队列

在开始使用 Laravel 队列之前，理解「连接」和「队列」之间的区别非常重

要。在 `config/queue.php` 配置文件中，有一个 `connections` 配置选项。此选项定义到后端服务（如 Amazon SQS、Beanstalk 或 Redis）的特定连接。然而，任何给定的队列连接都可能有多队列，这些队列可能被认为是不同的堆栈或成堆的排队任务。

请注意，`queue` 配置文件中的每个连接配置示例都包含一个 `queue` 属性。这是将任务发送到给定连接时将被分配到的默认队列。换句话说，如果你没有显式地定义任务应该被发送到哪个队列，那么该任务将被放置在连接配置的 `queue` 属性中定义的队列上：

```
// 这个任务将被推送到默认队列...

Job::dispatch();

// 这个任务将被推送到 "emails" 队列...

Job::dispatch()->onQueue('emails');
```

有些应用程序可能不需要将任务推到多个队列中，而是倾向于使用一个简单的队列。然而，如果希望对任务的处理方式进行优先级排序或分段时，将任务推送到多个队列就显得特别有用，因为队列工作程序允许您指定哪些队列应该按优先级处理。

7.12.1.2. 驱动程序说明和先决条件

数据库

要使用 `database` 队列驱动程序，你需要一个数据库表来保存任务。要生成创建此表的迁移，请运行 `queue:table` Artisan 命令。一旦迁移已经创建，你可以使用 `migrate` 命令迁移你的数据库：

```
php artisan queue:table

php artisan migrate
```

Redis

要使用 Redis 队列驱动程序，需要在 `config/database.php` 配置文件中配置一个 redis 数据库连接。

Redis 集群

如果你的 Redis 队列连接使用一个 Redis 集群，那么你的队列名称就必须包含一个 `key hash tag`。这是为了确保一个给定列表的所有 Redis 键都被放在同一个哈希插槽：

```
'redis' => [
    'driver' => 'redis',
    'connection' => 'default',
    'queue' => '{default}',
    'retry_after' => 90,
],
```

7.12.2. 创建任务

7.12.2.1. 生成任务类

默认情况下，应用程序的所有的可排队任务都被保存在 `app/Jobs` 目录中。如果 `app/Jobs` 目录不存在，当您运行 `make:job Artisan` 命令时，将会自动创建它。您可以使用 Artisan CLI 来生成一个新的队列任务：

```
php artisan make:job ProcessPodcast
```

生成的类将会实现 `Illuminate\Contracts\Queue\ShouldQueue` 接口，告诉 Laravel，该任务应该推入队列以异步的方式运行。

7.12.2.2. 任务类结构

任务类非常简单，通常只包含一个 `handle` 方法，该方法在队列处理任务时调用。让我们从一个任务类的例子看起。在这个例子中，我们假设我们管理一个 `podcast` 服务，并且需要上传的 `podcast` 文件发布之前对其进行处理：

```
<?php

namespace App\Jobs;

use App\Models\Podcast;
use App\Services\AudioProcessor;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class ProcessPodcast implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $podcast;

    /**
     * 创建一个新的任务实例。
     *
     * @param Podcast $podcast
     * @return void
     */
}
```

```
public function __construct(Podcast $podcast)

{

    $this->podcast = $podcast;

}


/**

 * 运行任务

 *

 * @param  AudioProcessor  $processor

 * @return void

 */

public function handle(AudioProcessor $processor)

{

    // Process uploaded podcast...

}

}
```

在本例中, 请注意我们能够将一个 **Eloquent model** 直接传递到已排队任务的构造函数中。由于任务所使用的 **SerializesModels**, 在任务处理时, **Eloquent** 模型及其加载的关系被优雅地序列化和非序列化。如果你的队列任务在其构造函数中接受一个 **Eloquent** 模型, 那么只有模型的标识符才会被序列化到队列中。当实际处理任务时, 队列系统将自动重新从数据库获取完整的模型实例及其加载的关系。

7.12.2.3. 任务中间件

任务中间件允许你围绕排队任务的执行封装自定义逻辑, 从而减少了任务本身的样板代码。例如, 看下面的 **handle** 方法, 它利用了 **Laravel** 的 **Redis** 速率

限制特性，允许每 5 秒只处理一个任务。

```
/**
 * 执行任务
 *
 * @return void
 */
public function handle()
{
    Redis::throttle('key')->block(0)->allow(1)->every(5)->then(function ()
    {
        info('获取锁 ...');

        // 处理任务 ...
    }, function () {
        // 无法获取锁 ...

        return $this->release(5);
    });
}
```

7.12.3. 分发任务

一旦编写了任务类，就可以使用作业本身的 `dispatch` 方法来分派它。传递给 `dispatch` 方法的参数将被传递给任务的构造函数：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;

class PodcastController extends Controller
{
    /**
     * 存储一个新的广播
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Create podcast...

        ProcessPodcast::dispatch($podcast);
    }
}
```

如果你希望有条件的分派任务，可以使用 **dispatchIf** 和 **dispatchUnless** 方法：

```
ProcessPodcast::dispatchIf($accountActive === true, $podcast);

ProcessPodcast::dispatchUnless($accountSuspended === false, $podcast);
```

7.12.3.1. 延迟调度

如果你想延迟队列任务的执行，你可以在调度任务时使用 `delay` 方法。例如，让我们指定一个任务在发出 10 分钟后才能进行处理：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;

class PodcastController extends Controller
{
    /**
     * 存储一个新的 podcast
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
```

```
{  
  
    // 创建 podcast...  
  
    ProcessPodcast::dispatch($podcast)  
        ->delay(now()->addMinutes(10));  
  
}  
}
```

7.12.3.2. 同步调度

如果您想要立即 (同步地) 调度任务, 您可以使用 `dispatchSync` 方法。当使用此方法时, 任务将不会排队, 并将立即运行在当前进程:

```
<?php  
  
namespace App\Http\Controllers;  
  
use App\Http\Controllers\Controller;  
use App\Jobs\ProcessPodcast;  
use Illuminate\Http\Request;  
  
class PodcastController extends Controller  
{  
  
    /**  
     * 存储一个新的 podcast  
     *  
     */  
}
```

```
* @param Request $request

* @return Response

*/

public function store(Request $request)

{

    // 创建 podcast...

    ProcessPodcast::dispatchSync($podcast);

}

}
```

7.12.3.3.任务链接

任务链接允许你指定一组应在主任务成功执行后按顺序运行的排队任务。如果序列中的一个任务失败，其余的任务将不会运行。要执行一个排队任务链，你可以使用 **Bus facade** 提供的 **chain** 方法：

```
use Illuminate\Support\Facades\Bus;

Bus::chain([

    new ProcessPodcast,

    new OptimizePodcast,

    new ReleasePodcast,

])->dispatch();
```

除了链接作业任务实例，你还可以链接闭包：

```
Bus::chain([
```

```
new ProcessPodcast,

new OptimizePodcast,

function () {

    Podcast::update(...);

},

])->dispatch();
```

7.12.3.4.自定义队列和连接

通过将任务推到不同的队列，你可以对排队的任务进行分类，甚至可以对分配给不同队列的任务进行优先排序。

7.12.3.5.指定任务最大尝试次数/超时值

最大尝试次数

指定任务可尝试的最大次数的其中一个方法是，通过 `Artisan` 命令行上的 `--tries` 开关：

```
php artisan queue:work --tries=3
```

但是，可以采用更粒度的方法：定义任务本身的最在最大尝试次数。如果在任务上指定了最大尝试次数，它将优先于命令行上提供的值：

```
<?php

namespace App\Jobs;

class ProcessPodcast implements ShouldQueue
{
    /**
```

```
    * 任务尝试次数

    *

    * @var int

    */

    public $tries = 5;
}
```

基于时间的尝试

除了定义任务失败前尝试的次数之外，还可以定义任务应该超时的时间。这允许在给定的时间范围内尝试任意次数的任务。要定义任务超时的时间，请在任务类中添加 `retryUntil` 方法：

```
/**
 * 确定任务应该超时的时间
 *
 * @return \DateTime
 */
public function retryUntil()
{
    return now()->addSeconds(5);
}
```

7.12.3.6. 速率限制

如果你的应用程序与 **Redis** 交互，你可能会根据时间或并发性限制排队任务。当排队的任务与同样有速率限制的 **api** 交互时，此特性可以派上用场。

例如，使用 `throttle` 方法，你可以将给定类型的作业限制为每 60 秒只运行

10 次。如果无法获得锁，通常应将任务释放回队列，以便稍后重试：

```
Redis::throttle('key')->allow(10)->every(60)->then(function () {  
  
    // 任务逻辑...  
  
}, function () {  
  
    // 无法获得锁...  
  
    return $this->release(10);  
  
});
```

7.12.3.7. 错误处理

如果在处理任务时抛出异常，则任务将自动释放回队列，以便再次尝试。直到它被尝试的次数达到你的申请允许的最大次数，该任务才将继续被释放。最大尝试次数由 `queue:work Artisan` 命令上使用的 `--tries` 开关定义。或者，可以在任务类本身上定义尝试的最大次数。有关运行队列 `worker` 的更多信息可以在下面找到。

7.12.3.8. 任务批处理

任务批处理功能使你轻松地执行一些任务，然后在任务完成执行后执行一些操作。在开始之前，你应该创建数据库迁移以构建一个包含任务批处理元信息的表。可以使用 `Artisan` 命令 `queue: batches-table` 生成此迁移：

```
php artisan queue:batches-table  
  
php artisan migrate
```

7.12.4. 队列闭包

除了将作业类推送到队列之外，你还可以推送闭包到队列。这对于需要在当

前请求周期之外执行的简单快速的任務非常有用。在將閉包推送到隊列時，閉包的代碼內容是經過加密簽名的，因為在傳輸過程中沒有辦法進行修改：

```
$podcast = App\Podcast::find(1);

dispatch(function () use ($podcast) {

    $podcast->publish();

});
```

當隊列的閉包方法失敗重試次數達到上限後仍沒有成功運行時，會執行 `catch` 方法中的閉包：

```
use Throwable;

dispatch(function () use ($podcast) {

    $podcast->publish();

})->catch(function (Throwable $e) {

    // 任務失敗了

});
```

7.12.5. 運行隊列處理器

Laravel 包含一個隊列工作器對新推入隊列的任務被進行處理。您可以使用 `queue:work` Artisan 命令運行 `worker`。請注意，一旦 `queue:work` 命令已經啟動，它將一直運行，直到它被手動停止或你關閉你的終端：

```
php artisan queue:work
```

請記住，隊列處理器是長生命週期的進程，並將啟動的應用程式狀態存儲在內存中。因此，在啟動它們之後，代碼庫中的更改對其不起作用。因此，在部署過程中，一定要重新啟動你的隊列處理器。此外，請記住，應用程式創建或修改

的任务静态状态不会在任务之间自动重置。

或者，你可以运行 `queue:listen` 命令。在使用 `queue:listen` 命令时，当你想要重新加载更新的代码或重置应用程序状态时，你不必手动重新启动 `worker`；但是，这个命令的效率不如 `queue:work`：

```
php artisan queue:listen
```

7.12.5.1. 队列优先级

有时，你可能希望优先考虑如何处理队列。例如，在 `config/queue.php` 中，你可以将你的 `redis` 连接的默认 `queue` 设置为 `low`。然而，有时你可能希望将一个任务推到一个 `high` 优先级队列，就像这样：

```
dispatch((new Job)->onQueue('high'));
```

要启动一个 `worker`，它在继续执行 `low` 队列上的任何作业之前，验证所有的 `high` 队列任务被处理了，请将一个以逗号分隔的队列名称列表传递给 `work` 命令：

```
php artisan queue:work --queue=high,low
```

7.12.6. Supervisor 配置

安装 Supervisor

Supervisor 是一个用于 Linux 操作系统的进程监视器，如果 `queue:work` 进程失败，它将自动重启该进程。要在 Ubuntu 上安装 Supervisor，你可以使用以下命令：

```
sudo apt-get install supervisor
```

Supervisor 配置

Supervisor 配置文件通常存储在 `/etc/supervisor/conf.d` 目录。在此目录中，你可

以创建任意数量的配置文件，这些配置文件将指示 `supervisor` 如何监事你的进程。

7.12.7. 处理失败任务

有时你排队的任务会失败。别担心，事情并不总是按计划进行！Laravel 提供了一种方便的方法来指定一个任务应该尝试的最大次数。当任务超过这个尝试数量后，它将被插入到 `failed_jobs` 数据库表中。要为 `failed_jobs` 表创建一个迁移，你可以使用 `queue:failed-table` 命令：

```
php artisan queue:failed-table  
  
php artisan migrate
```

7.12.7.1. 任务失败后的清理工作

你可以直接在 `job` 类上定义一个 `failed` 方法，它允许你在发生故障时执行特定于任务的清理。这是向用户发送警报或还原任务执行的任何操作的最佳位置。

7.13. 任务调度

7.13.1. 简介

过去，你可能需要在服务器上为每一个调度任务去创建 `Cron` 列表。但是这种方式很快会变得不友好，因为这些任务调度不在源代码中，并且你每次都需要通过 `SSH` 链接登录到服务器中才能增加 `Cron` 列表。

Laravel 命令行调度器允许你在 Laravel 中清晰流畅的定义命令调度。在使用这个任务调度器时，你只需要在你的服务器上创建单个 `Cron` 入口。你的任务调度在 `app/Console/Kernel.php` 的 `schedule` 方法中进行定义。为了帮助你更好的入门，这个方法中有个简单的例子。

7.13.1.1. 启动调度器

当使用这个调度器时，你只需要把下面的 `Cron` 入口添加到你的服务器中。如果你不知道如何在服务器中添加 `Cron` 列表，可以考虑使用一些服务来管理

Cron 入口，比如 Laravel Forge：

```
***** cd /path-to-your-project && php artisan schedule:run >> /dev/null
2>&1
```

该 Cron 会每分钟调用一次命令行调度器。在执行 `schedule:run` 命令时，Laravel 会根据你的调度执行预定的程序。

7.13.2. 定义调度

你可以在 `App\Console\Kernel` 类的 `schedule` 方法中定义所有的调度任务。在开始之前，让我们来看一个例子。在此例中，我们计划每天午夜执行一个闭包。在闭包中，我们会执行一个数据库查询来清空一张表：

```
<?php

namespace App\Console;

use Illuminate\Console\Scheduling\Schedule;
use Illuminate\Foundation\Console\Kernel as ConsoleKernel;
use Illuminate\Support\Facades\DB;

class Kernel extends ConsoleKernel
{
    /**
     * 应用中自定义的 Artisan 命令
     *
     * @var array
     */
}
```

```

protected $commands = [

    //

];

/**
 * 定义应用中的命令调度
 *
 * @param  \Illuminate\Console\Scheduling\Schedule  $schedule
 * @return void
 */
protected function schedule(Schedule $schedule)
{
    $schedule->call(function () {

        DB::table('recent_users')->delete();

    })->daily();
}
}

```

除在使用闭包来定义任务调度外，你也可以使用可调用对象。可调用对象是简单的 PHP 类，包含一个 `__invoke` 方法：

```
$schedule->call(new DeleteRecentUsers)->daily();
```

7.13.2.1. 调度 Artisan 命令

除了调用闭包这种方式来调度外，你还可以调用 `Artisan commands` 和操作系统命令。例如，你可以给 `command` 方法传递命令名称或者类来调度一个 Artisan

命令：

```
$schedule->command('emails:send --force')->daily();

$schedule->command(EmailsCommand::class, ['--force'])->daily();
```

7.13.2.2. 调度队列任务

`Job` 方法可用于调度一个队列任务。此方法提供了一种快捷方式来调度任务，而无需使用 `call` 方法创建闭包来调度任务：

```
$schedule->job(new Heartbeat)->everyFiveMinutes();

// Dispatch the job to the "heartbeats" queue...

$schedule->job(new Heartbeat, 'heartbeats')->everyFiveMinutes();
```

7.13.2.3. 调度 Shell 命令

`exec` 方法可用于调用操作系统 命令。

```
$schedule->exec('node /home/forged/script.js')->daily();
```

7.13.2.4. 调度频率设置

你还可以给你的任务分配多种调度方式

方法	描述
<code>->cron('* * * * *');</code>	自定义 Cron 计划执行任务
<code>->everyMinute();</code>	每分钟执行一次任务
<code>->everyTwoMinute();</code>	每两分钟执行一次任务
<code>->hourly();</code>	每小时执行一次任务

<code>->everyTwoHours();</code>	每两小时执行一次任务
<code>->daily();</code>	每天 0 点执行一次任务
<code>->weekly();</code>	每周日 00: 00 点执行一次任务
<code>->monthly();</code>	每月第一天 00: 00 点执行一次任务
<code>->yearly();</code>	每年第一天 00: 00 点执行一次任务

7.13.2.5.时区

`timezone` 方法可指定任务在给定时区中的时间执行：

```
$schedule->command('report:generate')
    ->timezone('America/New_York')
    ->at('02:00')
```

7.13.2.6.避免任务重叠

默认情况下，即使之前的任务实例还在执行，调度内的任务也会执行。为避免这种情况的发生，你可以使用 `withoutOverlapping` 方法：

```
$schedule->command('emails:send')->withoutOverlapping();
```

在此例中，若 `emails:send Artisan` 命令还未执行，那它将会每分钟执行一次。当你的任务执行时间非常不确定，导致你无法准确预测任务的执行时间，那 `withoutOverlapping` 方法会特别有用。

7.13.2.7.在单台服务器上运行任务

若你的应用在台服务器上运行，你可能需要限制某个计划任务仅在单台服务器上运行。假如你有一个计划任务：在每周五晚生成一份新报告。若此任务调度器在三台服务器上运行，那么该任务就会在三台服务器上运行并生成三份报告。

这样不好。

当你定义计划任务时,可使用 `onOneServer` 方法表明该任务仅在单台服务器上运行。

7.13.2.8. 后台任务

默认情况下,计划同时运行的多个命令将会顺序执行。若你有长时间运行的命令,这可能导致后续命令的启动时间比预期的更晚。你可以使用 `runInBackground` 方法让命令在后台运行,如此,它们便可同时运行了:

```
$schedule->command('analytics:report')
    ->daily()
    ->runInBackground();
```

7.13.2.9. 维护模式

Laravel 的队列任务在 维护模式 下不会运行。因为我们不想你的调度任务干扰到你服务器上可能还未完成的项目。不过,如果你确实是想在维护模式下强制调度任务执行,你可以使用 `evenInMaintenanceMode` 方法:

```
$schedule->command('emails:send')->evenInMaintenanceMode();
```

7.13.3. 任务输出

调度器提供了多个方便的方法来处理调度任务输出。首先,你可以使用 `sendOutputTo` 方法来输出到文件以便于后续检查:

```
$schedule->command('emails:send')
    ->daily()
    ->sendOutputTo($filePath);
```

如果希望将输出 附加 到给定文件,可以使用 `appendOutputTo` 方法:

```
$schedule->command('emails:send')
```



```
->daily()

->appendOutputTo($filePath);
```

使用 **emailOutputTo** 方法，你可以将输出发送到指定邮箱。在使用邮件发送之前，你需要配置 **Laravel** 的邮件服务：

```
$schedule->command('foo')

    ->daily()

    ->sendOutputTo($filePath)

    ->emailOutputTo('foo@example.com');
```

7.13.4. 任务钩子

使用 **before** 和 **after** 方法，你可以指定在调度任务完成之前和之后要执行的特定代码。

```
$schedule->command('emails:send')

    ->daily()

    ->before(function () {

        // 任务即将开始...

    })

    ->after(function () {

        // 任务完成...

    });
```

onSuccess 和 **onFailure** 方法允许你指定计划成功或失败要执行的代码：

```
$schedule->command('emails:send')

    ->daily()

    ->onSuccess(function () {
```

```
        // 任务成功...

    })

    ->onFailure(function () {

        // 任务失败...

    });
```

第八章 数据库操作

8.1. 快速入门

8.1.1. 简介

框架支持原生 SQL 查询、流畅的查询构造器 和 Eloquent ORM 这些操作在各种数据库后台与数据库的交互变得非常简单。目前 Laravel 支持以下四种数据库：

MySQL 5.6+ (版本策略)

PostgreSQL 9.4+ (版本策略)

SQLite 3.8.8+

SQL Server 2017+ (版本策略)

8.1.1.1. 配置

数据库配置文件在 `config/database.php` 文件中，你可以在此定义所有的数据库连接，并指定默认使用的连接。此文件内提供了大部分 Laravel 能支持的数据库配置示例。

默认情况下，Laravel 的示例环境配置使用了 Laravel Homestead (它是一种小型的虚拟机，能够让你很方便地在本地使用 Laravel 进行开发)。你可以根据本地数据库的需要修改这个配置文件。

8.1.1.2. 读写分离

有时候你希望 SELECT 语句使用一个数据为空连接，而 INSERT、UPDATE 和 DELETE 语句使用另一个数据库连接。在 Laravel 中，无论你是使用原生查询，查询构造器，还是 Eloquent ORM，都能轻松的实现。

为了弄明白读写分离是如何配置的，我们来看下个例子：

```
'mysql' => [  
    'read' => [  
        'host' => [  
            '192.168.1.1',  
            '196.168.1.2',  
        ],  
    ],  
    'write' => [  
        'host' => [  
            '196.168.1.3',  
        ],  
    ],  
    'sticky' => true,  
    'driver' => 'mysql',  
    'database' => 'database',  
    'username' => 'root',  
    'password' => '',  
    'charset' => 'utf8mb4',
```

```
'collation' => 'utf8mb4_unicode_ci',

'prefix' => '',

],
```

注意在以上的例子中，配置数组中增加了三个键，分别是 `read`、`write` 和 `sticky`。`read` 和 `write` 都包含一个键为 `host` 的数组。而 `read` 和 `write` 的其他数据库选项都在键为 `mysql` 的数组中。

如果你想重写主数组中的配置，只需要修改 `read` 和 `write` 数组即可。所以，这个例子中：192.168.1.1 和 192.168.1.2 将作为读连接主机，而 192.168.1.3 将作为写连接主机。

8.1.1.3. 使用多个数据库连接

使用多个数据库连接的时候，可以通过 DB 门面上的 `connection` 方法访问不同连接。传递给 `connecton` 方法的 `name` 对应配置文件 `config/database.php` 中设置文件中 `connections` 数组中的一个值。

```
$users = DB::connection('read')->select(...);
```

你也可以使用一个连接实例上的 `getPdo` 方法访问底层的 PDO 实例。

```
$pdo = DB::connection()->getPdo();
```

8.1.2. 运行原生 SQL 查询

配置好数据库连接后，就可以使用 DB 门面来运行查询。DB 门面为每种操作提供了相应方法。`select`, `update`, `insert`, `delete` 和 `statement`。

执行 Select 查询

你可以使用 DB 门面的 `select` 方法来运行基础的查询语句：

```
<?php

namespace App\Http\Controllers;
```

```
use App\Http\Controllers\Controller;

use Illuminate\Support\Facades\DB;

class UserController extends Controller
{
    /**
     * 显示应用程序中所有用户的列表
     *
     * @return Response
     */
    public function index()
    {
        $users = DB::select('select * from users where active = ?', [1]);

        return view('user.index', ['users' => $users]);
    }
}
```

传递给 `select` 方法的第一个参数就是一个原生的 SQL 查询，而第二个参数则是需要绑定到查询中的参数值。

8.1.3. 监听查询事件

如果你想监控程序执行的每一个 SQL 查询，你可以使用 `listen` 方法。这个方法对于记录查询或调试非常有用。你可以在服务提供者中注册你的查询监听器。

```
<?php
```

```
namespace App\Providers;

use Illuminate\Support\Facades\DB;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * 注册所有应用的服务
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * 引导所有应用的服务
     *
     * @return void
     */
    public function boot()
```

```
{

    DB::listen(function ($query) {

        // $query->sql

        // $query->bindings

        // $query->time

    });

}
```

8.1.4. 数据库事务

可以使用 DB 门面的 **transaction** 方法在数据库事务中运行一连串操作。如果事务闭包 Closure 中抛出异常，事务将会自动回滚。如果闭包执行成功，事务将会自动提交。一旦你使用了 **transaction**，就不必担心手动滚动或提交的问题：

```
DB::transaction(function () {

    DB::table('users')->update(['votes' => 1]);

    DB::table('posts')->delete();

});
```

8.1.4.1. 处理死锁

transaction 方法接受一个可选的第二次参数，该参数用来表示事务发生死锁重复执行的次数。一旦定义的次数尝试完毕，就会抛出一个异常：

```
DB::transaction(function () {

    DB::table('users')->update(['votes' => 1]);

    DB::table('posts')->delete();

});
```

```
}, 5);
```

8.1.4.2. 手动处理事务

如果您想要手动开启事务从而对回滚和提交有更好的控制，可以使用 DB 门面的 `beginTransaction` 方法。

```
DB::beginTransaction();
```

您可以通过 `rollBack` 方法回滚事务。

```
DB::rollBack();
```

最后，您可以通过 `commit` 方法提交事务。

```
DB::commit();
```

8.1.5. 连接到数据库 CLI

如果要连接到数据库的 CLI，可以使用 `db Artisan` 命令。

```
php artisan db
```

如果需要，可以指定数据库连接名称以连接到不是默认连接的数据库连接。

```
php artisan db mysql
```

8.2. 查询构建器

8.2.1. 简介

Laravel 的数据库查询构建器为创建和执行数据库查询提供了一个方便的接口。它可以用于支持大部分数据库操作，并且在所有支持的数据库系统上统一运行。

Laravel 的查询构造器使用 PDO 参数绑定的形式，保护你的应用程序免受 SQL 注入攻击。因此不必清理因参数绑定而传入的字符串。

8.2.2. 获取结果集

我们可以从 DB 门面的 `table` 方法开始，`table` 方法为给定表返回一个流式查询构建器实例，该实例允许你在查询上链接多个约束条件并返回最终查询结果。

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\DB;

class UserController extends Controller
{
    /**
     * 显示所有应用程序的用户列表。
     *
     * @return Response
     */
    public function index()
    {
        $users = DB::table('users')->get();

        return view('user.index', ['users' => $users]);
    }
}
```

`get` 方法返回一个包含 `Illuminate\Support\Collection` 实例的结果，其中每一条记录都是 `PHP stdClass` 对象的一个实例。你可以通过对象属性的方式来获取每个字段的值。

```
foreach ($users as $user) {  
  
    echo $user->name;  
  
}
```

8.2.2.1. 分块结果

如果你需要处理上千条数据库记录，你可以考虑使用 `chunk` 方法。该方法一次获取结果集的一小块，并将其传递给 `Closure` 函数进行处理。该方法在 `Artisan` 命令 编写数千条处理数据的时候非常有用。例如，我们可以将全部 `users` 表数据切割成一次处理 100 条记录的一小块：

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {  
  
    foreach ($users as $user) {  
  
        //  
  
    }  
  
});
```

也可以通过 `Closure` 中返回 `false` 来终止继续获取分块结果。

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {  
  
    // Process the records...  
  
    return false;  
  
});
```

8.2.2.2. 聚合

查询构造器还提供了各种聚合方法, 比如 `count`, `max`, `min`, `avg`, 还有 `sum`。你可以在构造查询后调用任何方法:

```
$users = DB::table('users')->count();

$price = DB::table('orders')->max('price');
```

当然, 你也可以将这些聚合方法与其他的查询语句相结合:

```
$price = DB::table('orders')
    ->where('finalized', 1)
    ->avg('price');
```

判断记录是否存在

除了通过 `count` 方法可以确定查询条件的结果是否存在之外, 还可以使用 `exists` 和 `doesntExist` 方法。

```
return DB::table('orders')->where('finalized', 1)->exists();

return DB::table('orders')->where('finalized', 1)->doesntExist();
```

8.2.3. 查询 (Select)

指定一个 Select 语句

我们可能不是总是想要获取数据表的所有有列。使用 `select` 方法, 你可以自定义一个 `select` 查询语句来查询指定的字段。

```
$users = DB::table('users')->select('name', 'email as
user_email')->get();
```

`distinct` 方法会强制让查询返回的结果不重复。

```
$users = DB::table('users')->distinct()->get();
```

8.2.4. 原生表达式

有时候你希望在查询中使用原生表达式，这些表达式将会以字符串的形式注入到查询中，所以要格外小心避免 SQL 注入。创建原生表达式，可以使用 `DB::raw` 方法。

```
$users = DB::table('users')

->select(DB::raw('count(*) as user_count, status'))

->where('status', '<>', 1)

->groupBy('status')

->get();
```

8.2.4.1. 原生方法

除了使用 `DB::raw` 外，您还可以使用以下方法来插入原生表达式到查询的不同部分。

8.2.4.1.1. selectRaw

`selectRaw` 方法可以代替 `select(DB::raw(...))`。该方法的第二个参数是可选项，值是一个绑定参数的数组：

```
$orders = DB::table('orders')

->selectRaw('price * ? as price_with_tax', [1.0825])

->get();
```

8.2.5. 连接 (Join)

查询构建器还可以用于编写连接语句，关于 SQL 的几种连接类型，通过下图可以一目了然。

8.2.5.1. 内连接

要实现一个简单的“内连接”，您可以使用查询构建器实例上的 `join` 方法，

传递给 `join` 方法的第一个参数是您需要连接到的表名，剩余的其他参数则是为连接指定的列约束，当然，正如您所看到的，您可以在单个查询中连接多张表。

```
$users = DB::table('users')

->join('contacts', 'users.id', '=', 'contacts.user_id')

->join('orders', 'users.id', '=', 'orders.user_id')

->select('users.*', 'contacts.phone', 'orders.price')

->get();
```

8.2.5.2. 左连接/右连接

如果您想要执行左连接或右连接，而不是内连接，可以使用 `leftJoin` 或 `rightJoin` 方法。这些方法和 `join` 方法的用法一样。

```
$users = DB::table('users')

->leftJoin('posts', 'users.id', '=', 'posts.user_id')

->get();

$users = DB::table('users')

->rightJoin('posts', 'users.id', '=', 'posts.user_id')

->get();
```

8.2.5.3. 交叉连接

要执行交叉连接可以使用 `crossJoin` 方法，传递您想要交叉连接的表名到该方法即可。交叉连连接在第一张表和被连接表之间生成一个笛卡尔积。

```
$users = DB::table('sizes')

->crossJoin('colours')

->get();
```

8.2.5.4. 高级连接语句

您还可以指定更多的高级连接子句, 传递一个闭包到 `join` 方法作为第二个参数, 该闭包将会接收一个 `JoinClause` 对象用于指定 `join` 子句约束。

```
DB::table('users')

->join('contacts', function ($join) {

    $join->on('users.id', '=', 'contacts.user_id')

    ->orOn(...);

})

->get();
```

8.2.5.5. 子查询连接

您可以使用 `joinSub`、`leftJoinSub` 和 `rightJoinSub` 方法将查询和一个子查询进行连接, 每个方法都接收三个参数——子查询、表别名和定义关联字段的闭包。

```
$latestPosts = DB::table('posts')

->select('user_id', DB::raw('MAX(created_at) as
last_post_created_at'))

->where('is_published', true)

->groupBy('user_id');

$users = DB::table('users')

->joinSub($latestPosts, 'latest_posts', function($join) {

    $join->on('users.id', '=', 'latest_posts.user_id');

})->get();
```

8.2.6. 联合 (Union)

查询构建器还提供了联合两个查询的快捷方式，比如，您可以先创建一个查询，然后使用 `union` 方法将其和第二个查询进行联合。

```
$first = DB::table('users')
    ->whereNull('first_name');

$users = DB::table('users')
    ->whereNull('last_name')
    ->union($first)
    ->get();
```

注：`unionAll` 方法也是有效的，并且和 `union` 使用方式相同。

8.2.7. Where 子句

简单的 Where 语句

在构造 `where` 查询实例中，你可以使用 `where` 方法。调用 `where` 最基本的方式是需要传递三个参数：第一个参数列名，第二个参数是任意一个数据库系统支持的运算符，第三个是该列要比较的值。

例如，下面是一个要验证 `votes` 字段的值等于 100 的查询：

```
$users = DB::table('users')->where('votes', '=', 100)->get();
```

8.2.7.1. 参数分组

有时候你需要创建更高级的 `where` 子句，例如「`where exists`」或者嵌套的参数分组。Laravel 的查询构造器也能够处理这些。下面，让我们看一个在括号中进行分组约束的例子：

```
$users = DB::table('users')
    ->where('name', '=', 'John')
    ->where(function ($query) {
```

```

$query->where('votes', '>', 100)

        ->orWhere('title', '=', 'Admin');

    })

    ->get();

```

你可以看到，通过使用闭包写入 **where** 的方法来创建一个查询构造器，用来约束一个分组。这个闭包接收一个查询实例，你可以使用这个实例来设置应该包含的约束。上面的例子将生成以下 SQL 语句。

```
select * from users where name = 'John' and (votes > 100 or title = 'Admin')
```

8.2.7.2. Where exists 子句

whereExists 方法允许你使用 **where exists** SQL 语句。**whereExists** 方法接收一个 **Closure** 参数，该闭包获取一个查询构建器实例从而允许你定义放置在 **exists** 子句中查询：

```

$users = DB::table('users')

    ->whereExists(function ($query) {

        $query->select(DB::raw(1))

            ->from('orders')

            ->whereRaw('orders.user_id = users.id');

    })

    ->get();

```

上述查询将产生如下的 SQL 语句：

```

select * from users

where exists (

    select 1 from orders where orders.user_id = users.id

```



```
)
```

8.2.7.3. 子查询 Where 子句

有时你可能需要构造一个 `where` 语句，将子查询的结果与一个给定值进行比较。你可以通过给 `where` 方法传递一个闭包以及给定值实现。举例来说，下面的查询语句将会获取到所有属于最近一个会员类型的所有用户。

```
use App\User;

$users = User::where(function ($query) {

    $query->select(' type')

        ->from(' membership')

        ->whereColumn(' user_id', ' users.id')

        ->orderByDesc(' start_date')

        ->limit(1);

}, 'Pro')->get();
```

8.2.7.4. JSON Where 子句

Laravel 也支持查询 JSON 类型的字段（仅在对 JSON 类型支持的数据库上）。目前，本特性支持包括 MySQL 5.7、PostgreSQL、SQL Server 2016、和 SQLite 3.9.0 (with the JSON1 extension)。

```
$users = DB::table(' users')

        ->where(' options->language', ' en')

        ->get();

$users = DB::table(' users')
```

```
->where('preferences->dining->meal', 'salad')

->get();
```

8.2.8. 排序、分组

8.2.8.1. orderBy

`orderBy` 方法允许你通过给定字段对结果集进行排序。`orderBy` 的第一个参数应该你希望排序的字段，第二个参数控制排序的方向，可以是 `asc` 或 `desc`:

```
$users = DB::table('users')

    ->orderBy('name', 'desc')

    ->get();
```

如果你需要使用多个字段进行排序，你可以多次引用 `orderBy`:

```
$users = DB::table('users')

    ->orderBy('name', 'desc')

    ->orderBy('email', 'asc')

    ->get();
```

8.2.9. 条件子句

有时候你可能想要子句只适用于某个情况为真时才执行查询。例如你可能只想给定值在请求中存在的情况下才应用 `where` 语句。你可以通过使用 `when` 方法来实现。

```
$role = $request->input('role');

$users = DB::table('users')

    ->when($role, function ($query, $role) {
```

```
        return $query->where('role_id', $role);

    })

    ->get();
```

when 方法只有在每一个参数为 **true** 的时候才执行给的闭包。如果第一个参数为 **false**，那么这个闭包将不会被执行。

你可以传递另一个闭包作为 **when** 方法的第三个参数。该闭包会在第一个参数为 **false** 的情况下执行。为了说明如何使用这个特征，我们来配置一个查询的默认排序。

```
$sortBy = null;

$users = DB::table('users')

    ->when($sortBy, function ($query, $sortBy) {

        return $query->orderBy($sortBy);

    }, function ($query) {

        return $query->orderBy('name');

    })

    ->get();
```

8.2.10. 插入

查询构造器还提供了 **insert** 方法用来插入记录到数据库中。**insert** 方法接收数组形式的字段名和字段值进行插入操作。

```
DB::table('users')->insert(

    ['email' => 'john@example.com', 'votes' => 0]

);
```

你甚至可以将二维数组传递给 `insert` 方法，依次将多个记录插入到表中。

```
DB::table('users')->insert([

    ['email' => 'taylor@example.com', 'votes' => 0],

    ['email' => 'dayle@example.com', 'votes' => 0],

]);
```

自增 IDs

如果数据表有自增 ID，使用 `insertGetId` 方法来插入记录可以返回 ID 值。

```
$id = DB::table('users')->insertGetId(

    ['email' => 'john@example.com', 'votes' => 0]

);
```

8.2.11. 更新

当然，除了插入记录到数据库中，查询构造器也可以通过 `update` 方法更新已有的记录。`update` 方法和 `insert` 方法一样，接受包含要更新的字段及值的数组。你可以通过 `where` 子句对 `update` 查询进行约束。

```
$affected = DB::table('users')

    ->where('id', 1)

    ->update(['votes' => 1]);
```

8.2.11.1. 更新或新增

有时你可能希望更新数据库的现有记录，或者如果不存在匹配记录则创建它。在这种情况下，可以使用 `updateOrCreate` 方法。`updateOrCreate` 方法接受两个参数：一个用于查找记录的条件数组，以及一个包含要更改记录的键值对数组。

`updateOrCreate` 方法将首先尝试使用第一个参数的键和值来查询匹配的数据库记录。如果记录存在，则使用第二个参数中的值去更新记录。如果找不到记

录，将插入一个新记录，新增的数据是两个数组的集合。

```
DB::table('users')

    ->updateOrCreate(

        ['email' => 'john@example.com', 'name' => 'John'],

        ['votes' => '2']

    );
```

8.2.11.2.更新 JSON

更新 JSON 字段时，你可以使用 `->` 语法访问 JSON 对象中相应的值。注意，此操作只能支持 MySQL 5.7+ 和 PostgreSQL 9.5+

```
$affected = DB::table('users')

    ->where('id', 1)

    ->update(['options->enabled' => true]);
```

8.2.11.3.自增自减

查询构造器还为给定字段的递增或递减提供了便捷的方式。此方法提供了一个比手动编写 `update` 语句更具表达力且更精炼的接口。

这两种方法都至少接收一个参数：需要修改的列。可选的第二个参数用于控制列递增或递减的量：

```
DB::table('users')->increment('votes');

DB::table('users')->increment('votes', 5);

DB::table('users')->decrement('votes');
```

```
DB::table('users')->decrement('votes', 5);
```

8.2.12. 删除

查询构造器也可以使用 `delete` 方法从表中删除记录。在使用 `delete` 前，可以添加 `where` 子句来约束 `delete` 语法。

```
DB::table('users')->delete();
```

```
DB::table('users')->where('votes', '>', 100)->delete();
```

你可以使用 `truncate` 方法来清空整个表，这将移除所有的数据并重置所有的自增 ID 为 0。

```
DB::table('users')->truncate();
```

8.2.13. 悲观锁

查询构造器也包含了一些能够帮助你在 `select` 语句中实现「悲观锁」的函数。要执行一个含有「共享锁」的语句，你可以在查询中使用 `sharedLock` 方法。共享锁可防止指定的数据列被篡改，直到事务被提交为止。

```
DB::table('users')->where('votes', '>', 100)->sharedLock()->get();
```

8.2.14. 调试

在绑定查询的时候，你可以使用 `dd` 或 `dump` 方法来输出查询绑定和 SQL。`dd` 方法将会显示调试信息并终止执行请求，而 `dump` 方法则会显示调试信息并允许请求继续执行。

```
DB::table('users')->where('votes', '>', 100)->dd();
```

```
DB::table('users')->where('votes', '>', 100)->dump();
```

8.3. 分页

8.3.1. 简介

在其他框架中，分页可能是件非常痛苦的事。Laravel 的分页器将 查询构造器 和 Eloquent ORM 结合起来，提供了方便、易用的数据库结果集分页。通过分页器生成的 HTML 兼容 Bootstrap CSS 框架。

8.3.2. 基本用法

8.3.2.1. 查询构造器分页

数据分页有多种方法。最简单的是使用 查询构造器 或 Eloquent query 的 `paginate` 方法。`paginate` 方法根据用户浏览的当前页码，自动设置恰当的偏移量 `offset` 和限制数 `limit`。默认情况下，HTTP 请求中的 `page` 查询参数值被当作当前页的页码。Laravel 会自动检测该值，并自动将其插入到分页器生成的链接中。

在下面的例子中，传递给 `paginate` 方法的唯一参数是你想要在每页显示的记录数。在此例中，我们指定要在每页显示 15 条数据：

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\DB;

class UserController extends Controller
{
```

```
/**
 * Show all of the users for the application.
 *
 * @return Response
 */
public function index()
{
    $users = DB::table('users')->paginate(15);

    return view('user.index', ['users' => $users]);
}
```

如果你只需要在分页视图中简单地显示「下一页」和「上一页」的链接，你可以使用 `simplePaginate` 方法来执行更高效地查询。这在数据量很大且不需要在渲染视图时显示每页的页码时非常有用。

```
$users = DB::table('users')->simplePaginate(15);
```

8.3.2.2. Eloquent 分页

你还可以对 Eloquent 查询结果进行分页。在下面的例子中，我们将 `User` 模型按每页 15 条分页。如你所见，其语法与查询构造器分页基本相同：

```
$users = App\User::paginate(15);
```

你还可以在调用 `paginate` 之前，在查询中设置诸如 `where` 从句的其他约束：

```
$users = User::where('votes', '>', 100)->paginate(15);
```


8.3.2.3. 手动创建分页

有时你可能希望手动创建分页，并传递一个数组集给它。可以通过创建 `Illuminate\Pagination\Paginator` 或 `Illuminate\Pagination\LengthAwarePaginator` 实例来实现，这取决于你的需要。

`Paginator` 类不需要知道结果集的总数；然而，这样一来，这个类就没办法获知最后一页的索引。`LengthAwarePaginator` 接受和 `Paginator` 几乎相同的参数；不过，它会计算结果集的总数。

换句话说，`Paginator` 相当于查询构造器或 Eloquent 的 `simplePaginate` 方法，而 `LengthAwarePaginator` 相当于 `paginate` 方法。

8.3.3. 显示分页视图

调用 `paginate` 方法时，你会得到 `Illuminate\Pagination\LengthAwarePaginator` 实例，调用 `simplePaginate` 方法时，你会得到 `Illuminate\Pagination\Paginator` 实例。这些对象提供了几个描述结果集的方法。除了这些辅助方法，分页器的实例是迭代器，可以像数组一样循环。

8.3.3.1. 将结果转换为 JSON

Laravel 分页器类实现了 `Illuminate\Contracts\Support\Jsonable` 接口契约，提供了 `toJson` 方法，所以可以方便的将分页结果转换为 JSON。你也可以通过来自路由或控制器的操作返回分页实例来将其转换为 JSON：

```
Route::get('users', function () {  
    return App\User::paginate();  
});
```

8.3.4. 自定义分页视图

默认情况下，渲染视图来显示分页链接是与 Tailwind CSS 框架 兼容的。然而，如果你不想使用 Tailwind，你可以自由的选择你自己定制的视图来渲染这些链接。

8.3.4.1. 使用 Bootstrap

默认情况下，渲染视图来显示分页链接是与 Bootstrap CSS 框架兼容的。然而，如果你不想使用 Bootstrap，你可以自由的选择你自己定制的视图来渲染这些链接。在调用分页器实例的 `links` 方法时，将视图名称作为传递给它的第一个参数：

```
{{ $paginator->links('view.name', ['foo' => 'bar']) }}
```

```
use Illuminate\Pagination\Paginator;
```

```
public function boot()
{
    Paginator::useBootstrap();
}
```

8.3.5. 分页器实例方法

每个分页器实例都可以通过以下方法提供更多分页信息。

方法	描述
<code>\$results->count()</code>	获取当前页面数据的数据量
<code>\$results->currentPage()</code>	获取当前页页码
<code>\$results->firstItem()</code>	获取结果集中第一条数据的结果编号

8.4. 迁移

8.4.1. 简介

迁移就像是数据库的版本控制器，让你的团队更容易修改和共享程序的数据库结构。迁移通常配合 Laravel 的结构生成器，能更容易的生成应用程序的数据库结构。如果你曾经让一个团队成员在他本地的数据库结构中手动的添加了字段，

那么你将面对解决数据库迁移的问题。

Laravel 的 Schema facade 提供数据库相关的支持，用于在所有 Laravel 支持的数据库系统中创建和操作表。

8.4.2. 生成迁移

使用 Artisan 命令 `make:migration` 来创建迁移：

```
php artisan make:migration create_users_table
```

8.4.3. 迁移结构

一个迁移类包含两个方法：`up` 和 `down`。`up` 方法是用于新增数据库的数据表、字段或者索引的，而 `down` 方法应该与 `up` 方法的执行操作相反。

在这两种方法中，你可以使用 Laravel 的结构生成器以表达式方式创建和修改表。要了解 Schema 生成器上可用的所有方法，请查看其文档。例如，这是一个创建 `flights` 数据表的例子：

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateFlightsTable extends Migration
{
    /**
     * Run the migrations.
     *
     */
}
```

```
* @return void

*/

public function up()

{

    Schema::create('flights', function (Blueprint $table) {

        $table->id();

        $table->string('name');

        $table->string('airline');

        $table->timestamps();

    });

}

/**

 * Reverse the migrations.

 *

 * @return void

 */

public function down()

{

    Schema::drop('flights');

}

}
```

8.4.4. 运行迁移

请执行 Artisan 的 `migrate` 命令来执行所有未执行的迁移：

```
php artisan migrate
```

8.4.4.1. 回滚迁移

要回滚到最后一次操作，你可以用下 `rollback` 命令。此命令会回滚到最后一批的迁移，这可能会包含多个迁移文件：

```
php artisan migrate:rollback
```

8.4.5. 数据表

8.4.5.1. 创建数据表

请使用 Schema facade 中的 `create` 方法来在数据库中创建一个新表。`create` 可以接受两个参数。一个是数据表的名字，另一个是 Closure 可以在定义一个新数据表的时候接受 Blueprint 对象：

```
Schema::create('users', function (Blueprint $table) {  
    $table->id();  
});
```

8.4.5.2. 重命名/删除数据表

重命名一个存在的数据库表，请使用 `rename` 方法：

```
Schema::rename($from, $to);
```

8.4.6. 字段

8.4.6.1. 创建字段

在 Schema 门面中可以使用 `table` 方法来更新已存在的数据表。和 `create` 方法一样, `table` 方法接受两个参数: 一个是数据表的名字, 另一个是 Closure 可以在新增一个数据表中字段的时候接受 Blueprint 对象:

```
Schema::table('users', function (Blueprint $table) {  
  
    $table->string('email');  
  
});
```

8.4.6.2. 修饰字段

除了上述列出的字段类型之外, 还有几个可以在添加字段到数据库表时使用的「修饰符」。例如, 如果要把字段设置为「可空」, 你可以使用 `nullable` 方法:

```
Schema::table('users', function (Blueprint $table) {  
  
    $table->string('email')->nullable();  
  
});
```

8.4.6.3. 修改字段

在修改字段之前, 请确保将 `doctrine/dbal` 依赖添加到 `composer.json` 文件中。Doctrine DBAL 库用于确定字段的当前状态, 并创建对该字段进行指定调整所需的 SQL 查询:

```
composer require doctrine/dbal
```

8.4.6.4. 删除字段

可以使用结构生成器上的 `dropColumn` 方法来删除字段。在从 SQLite 数据库删除字段前, 你需要在 `composer.json` 文件中加入 `doctrine/dbal` 依赖并在终端执行 `composer update` 来安装该依赖:

```
Schema::table('users', function (Blueprint $table) {  
  
    $table->dropColumn('votes');  
  
});
```

8.4.7. 索引

8.4.7.1. 创建索引

结构生成器支持多种类型的索引。下面的例子中新建了一个值唯一的 `email` 字段。我们可以将 `unique` 方法链式地添加到字段定义上来创建索引：

```
$table->string('email')->unique();
```

8.4.7.2. 重命名索引

若要重命名索引，你需要调用 `renameIndex` 方法。此方法接受当前索引名称作为其第一个参数，并将所需名称作为其第二个参数：

```
$table->renameIndex('from', 'to');
```

8.4.7.3. 删除索引

若要删除索引，则必须指定索引的名称。`Laravel` 默认会自动将数据表名称、索引的字段名及索引类型简单地连接在一起作为名称。

8.4.7.4. 外键索引

`Laravel` 还支持创建用于在数据库层中的强制引用完整性的外键约束。例如，让我们在 `posts` 表上定义一个引用 `users` 表的 `id` 字段的 `user_id` 字段：

```
Schema::table('posts', function (Blueprint $table) {  
  
    $table->unsignedBigInteger('user_id');
```

```
$table->foreignId('user_id')->references('id')->on('users');  
});
```

8.5. 数据填充

8.5.1. 简介

Laravel 包含一个填充类可以为你的数据库填充测试数据，所有的填充类都放在 `database/seeds` 目录下。你可以随意为填充类命名，但是更建议您遵守类似 `UserSeeder` 的命名规范。通常，Laravel 默认定义了一个 `DatabaseSeeder` 类。通过这个类，你可以用 `call` 方法来运行其它的 `seed` 类从而控制数据填充的顺序。

8.5.2. 编写 Seeders

运行 Artisan 命令 `make:seeder` 生成 `Seeder`，框架生成的 `seeders` 都放在 `database/seeds` 目录下：

```
php artisan make:seeder UserSeeder
```

8.5.2.1. 使用模型工厂

手动为每个模型填充指定属性很麻烦，你可以使用 `model` 工厂 轻松地生成大量数据库数据。首先，阅读 `model` 工厂文档 来学习如何定义工厂文件，一旦定义好了你的工厂文件，然后就可以使用 `factory` 这个辅助函数来向数据库中插入数据。

例如，创建 50 个用户并为每个用户创建关联：

```
/**  
  
 * Run the database seeds.  
  
 */
```



```
* @return void

*/

public function run()
{
    factory(App\User::class, 50)->create()->each(function ($user) {
        $user->posts()->save(factory(App\Post::class)->make());
    });
}
```

8.5.2.2. 调用其他 Seeders

在 `DatabaseSeeder` 类中，你可以使用 `call` 方法来运行其它的 `seed` 类。使用 `call` 方法可以将数据填充拆分成多个文件，这样就不会使单个 `seeder` 变得非常大。只需简单传递要运行的 `seeder` 类名称即可：

```
/**

 * Run the database seeds.

 *

 * @return void

 */

public function run()
{
    $this->call([
        UserSeeder::class,
        PostSeeder::class,
        CommentSeeder::class,
```

```
]);  
}
```

8.5.3. 运行 Seeders

完成 seeder 类的编写之后，你可能需要使用 `dump-autoload` 命令重新生成 Composer 的自动加载器：

```
composer dump-autoload
```

8.6. Redis

8.6.1. 简介

Redis 是一个开源的，高级键值对存储数据库。由于它包含 字符串，哈希，列表，集合，和 有序集合 这些数据类型，所以它通常被称为数据结构服务器。

在将 Redis 与 Laravel 一起使用前，我们鼓励你通过 PECL 安装并使用 PhpRedis PHP 扩展。尽管扩展安装起来更复杂，但对于大量使用 Redis 的应用程序可能会产生更好的性能。

8.6.1.1. 配置

您的 Laravel 应用的 Redis 配置文件都在 `config/database.php` 中。在这个文件里，你可以看到一个 `redis` 数组里包含了应用程序所使用的 Redis 服务器信息。

```
'redis' => [  
  
    'client' => env('REDIS_CLIENT', 'phpredis'),  
  
    'default' => [  

```

```
'host' => env('REDIS_HOST', '127.0.0.1'),

'password' => env('REDIS_PASSWORD', null),

'port' => env('REDIS_PORT', 6379),

'database' => env('REDIS_DB', 0),

],

'cache' => [

    'host' => env('REDIS_HOST', '127.0.0.1'),

    'password' => env('REDIS_PASSWORD', null),

    'port' => env('REDIS_PORT', 6379),

    'database' => env('REDIS_CACHE_DB', 1),

],

],
```

默认的服务器配置应该足以进行开发。当然，你也可以根据使用的环境来随意更改这个数组。只需在配置文件中给每个 **Redis** 服务器指定名称、**host**（主机）和 **port**（端口）即可。

8.6.1.2. Predis

要使用 **Predis** 扩展，需要将环境变量从 **phpredis** 选项修改为 **predis**：

```
'redis' => [

    'client' => env('REDIS_CLIENT', 'predis'),

    // Rest of Redis configuration...
```

```
],
```

8.6.1.3. PhpRedis

要使用 PhpRedis 扩展，需要将配置文件 `config/database.php` 中 Redis 配置的 `REDIS_CLIENT` 选项修改为 `phpredis`：

```
'redis' => [

    'client' => env('REDIS_CLIENT', 'phpredis'),

    // 其余 Redis 配置...

],
```

8.6.2. 与 Redis 交互

你可以通过调用 Redis facade 上的各种方法来与 Redis 交互。Redis facade 支持动态方法，意味着你可以在 facade 上调用各种 Redis 命令，命令将直接传递给 Redis。

8.6.2.1. 管道命令

当你需要在一个操作中给服务器发送很多命令时，推荐你使用管道命令。`pipeline` 方法接受一个 Redis 实例的闭包。你可以将所有的命令发送给 Redis 实例，它们都会在一个操作中执行完成：

```
Redis::pipeline(function ($pipe) {

    for ($i = 0; $i < 1000; $i++) {

        $pipe->set("key:$i", $i);

    }

});
```

8.6.3. 发布/订阅

Laravel 为 Redis 的 `publish` 及 `subscribe` 提供了方便的接口。这些 Redis 命令让你可以监听指定「频道」上的消息。你可以从另一个应用程序发布消息给另一个应用程序，甚至使用其它编程语言，让应用程序和进程之间能够轻松进行通信。

首先，我们使用 `subscribe` 方法设置频道监听器。我们将这个方法调用放在 `Artisan` 命令 中，因为调用 `subscribe` 方法会启动一个长时间运行的进程：

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\Redis;

class RedisSubscribe extends Command
{
    /**
     * 控制台命令的名称和签名。
     *
     * @var string
     */
    protected $signature = 'redis:subscribe';
```

```
/**
 * 控制台命令说明。
 *
 * @var string
 */
protected $description = 'Subscribe to a Redis channel';

/**
 * 执行控制台命令。
 *
 * @return mixed
 */
public function handle()
{
    Redis::subscribe(['test-channel'], function ($message) {
        echo $message;
    });
}
```

第九章 Eloquent ORM

9.1. 快速入门

9.1.1. 简介

框架内置的 Eloquent ORM 提供了一个美观、简洁的与数据库打交道的 ActiveRecord 实现，每个数据表都对应一个与该表进行交互的模型（Model）。你可以通过模型查询数据表中的数据，以及在数据表中插入新记录。

在开始之前，请确保在 `config/database.php` 中配置数据库连接。

9.1.2. 定义模型

我们从创建一个 Eloquent 模型开始，模型通常位于 `app\Models` 目录下。所有 Eloquent 模型都继承自 `Illuminate\Database\Eloquent\Model` 类。

```
php artisan make:model Flight
```

如果要在生成模型的时候生成数据库迁移，可以使用 `--migration` 或 `-m` 选项。

```
php artisan make:model Flight --migration
```

```
php artisan make:model Flight -m
```

9.1.2.1. Eloquent 模型约定

现在，让我们来看一个 `Flight` 模型的例子，我们将用该类获取和存取数据表 `flights` 中的信息。

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
```

```
//  
}
```

注意我们并没有告诉 **Eloquent** 我们的 **Flight** 模型使用哪张表，默认规则是小写的模型类名复数格式作为与其对应的表名。所以，在本例中，**Eloquent** 认为 **Flight** 模型存储记录在 **flights** 表中。您也可以在模型中定义 **table** 属性来指定自定义的类名。

```
<?php  
  
namespace App;  
  
use Illuminate\Database\Eloquent\Model;  
  
class Flight extends Model  
{  
  
    /**  
     * 关联到模型的数据表  
     *  
     * @var string  
     */  
  
    protected $table = 'my_flights';  
}
```

Eloquent 默认每张表的主键名为 **id**，您可以在模型类中定义一个 **\$primaryKey** 属性来覆盖该约定。

```
<?php  
  
namespace App;  
  
use Illuminate\Database\Eloquent\Model;  
  
class Flight extends Model
```



```
{

    /**
     * The primary key associated with the table.
     *
     * @var string
     */
    protected $primaryKey = 'flight_id';
}
```

此外，Eloquent 默认主键字段是自增的整形数据，这意味着主键将会被自动转化为 `int` 类型，如果您想要使用非自增或非数字类型主键，必须在对应模型中设置 `$incrementing` 属性为 `false`。

```
<?php

class Flight extends Model
{

    /**
     * Indicates if the IDs are auto-incrementing.
     *
     * @var bool
     */
    public $incrementing = false;
}
```

如果主键不是整形，还要设置 `$keyType` 属性值为 `string`。

```
<?php

class Flight extends Model
```

```
{  
  
    /**  
     * The "type" of the auto-incrementing ID.  
     *  
     * @var string  
     */  
  
    protected $keyType = 'string';  
}
```

默认情况下，Eloquent 期望 `created_at` 和 `updated_at` 已经存在于数据表中，如果您不想要这些框架自动管理的数据列，在模型类中设置 `$timestamps` 属性为 `false`。

```
<?php  
  
namespace App;  
  
use Illuminate\Database\Eloquent\Model;  
  
class Flight extends Model  
{  
  
    /**  
     * 表明模型是否应该被打上时间戳  
     *  
     * @var bool  
     */  
  
    public $timestamps = false;  
}
```

如果您需要自定义用于存储时间戳的字段名称，可以在模型中设置

CREATED_AT 和 UPDATED_AT 常量。

```
<?php

class Flight extends Model
{
    const CREATED_AT = 'creation_date';

    const UPDATED_AT = 'last_update';
}
```

9.1.2.2. 默认属性值

如果您想要定义某些属性的默认值，可以在模型上定义 `$attributes` 属性：

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The model's default values for attributes.
     *
     * @var array
     */
    protected $attributes = [

        'delayed' => false,

    ];
}
```

```
}
```

9.1.3. 获取模型

创建完模型及其关联的数据表后，就可以从数据库中获取数据了。将 Eloquent 模型看作功能强大的查询构建器，您可以使用它来流畅地查询与其关联的数据表。例如：

```
<?php

use App\Flight;

$flights = App\Flight::all();

foreach ($flights as $flight) {

    echo $flight->name;

}
```

Eloquent 的 `all` 方法返回模型表的所有结果，由于每一个 Eloquent 模型都是一个查询构建器，您还可以添加约束条件到查询，然后使用 `get` 方法获取对应结果。

```
$flights = App\Flight::where('active', 1)

    ->orderBy('name', 'desc')

    ->take(10)

    ->get();
```

9.1.3.1. 集合

对 Eloquent 中获取多个结果的方法（比如 `all` 和 `get`）而言，其返回值是 `Illuminate\Database\Eloquent\Collection` 的一个实例，`Collection` 类提供了多外有用的函数来处理 Eloquent 结果集。

```
$flights = $flights->reject(function ($flight) {
```

```
        return $flight->cancelled;
    });
```

当然，您也可以像数组一样循环遍历该集合。

9.1.3.2. 组块结果集

如果您需要处理数据量很大的 Eloquent 结果集，可以全用 `chunk` 方法。`Chunk` 方法会获取一个指定数量的 Eloquent 模型组块，并将其填充到给定闭包进行处理。使用 `chunk` 方法在处理大量数据集时能够有效减少内存消耗。

```
Flight::chunk(200, function ($flights) {

    foreach ($flights as $flight) {

        //

    }

});
```

传递给该方法的第一个参数是你想要获取的“组块”数目，闭包作为第二个参数被传入用于处理每个从数据库获取的组块数据。

9.1.3.3. 高级子查询

Eloquent 还提供了高级子查询支持，该特征允许你从关联数据表中拉取信息到单个查询。例如，假设我们有一个航班目的地表 `destinations` 和一个飞向这些目的地的航班表 `flights`，`flights` 包含用于记录航班到达目的地时间的 `arrived_at` 字段。

```
use App\Destination;

use App\Flight;

return Destination::addSelect(['last_flight' => Flight::select('name')
->whereColumn('destination_id', 'destinations.id')
]);
```

```
->orderBy('arrived_at', 'desc')

->limit(1)

])->get();
```

9.1.4. 获取单个模型/聚合结果

当然，除了从给定表中获取所有记录之外，还可以使用 `find` 和 `first` 获取单个记录。这些方法返回单个模型实例而不是模型集合。

```
// 通过主键获取模型...

$flight = App\Flight::find(1);

// 获取匹配查询条件的第一个模型...

$flight = App\Flight::where('active', 1)->first();
```

还可以通用传递主键数组还调用 `find` 方法。

```
$flights = App\Flight::find([1, 2, 3]);
```

9.1.4.1. 获取聚合结果

当然，您还可以使用查询构建器提供的聚合方法，例如 `count`、`sum`、`max`，以及其他查询构建器提供的聚合函数。这些方法返回计算后的结果而不是整个模型实例。

```
$count = App\Flight::where('active', 1)->count();

$max = App\Flight::where('active', 1)->max('price');
```

9.1.5. 插入/更新模型

9.1.5.1. 插入

想要在数据库中插入新的记录，只需创建一个新的模型实例，设置模型的属

性，然后调用 `save` 方法。

```
<?php

namespace App\Http\Controllers;

use App\Flight;
use Illuminate\Http\Request;
use App\Http\Controllers\Controller;

class FlightController extends Controller{

    /**
     * 创建一个新的航班实例
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // 验证请求...

        $flight = new Flight;

        $flight->name = $request->name;

        $flight->save();
    }
}
```

在这个例子中，我们只是简单分配 HTTP 请求中的 `name` 参数值给 `App\Flight` 模型实例的 `name` 属性，当我们调用 `save` 方法时，一条记录将会被插入数据库。

9.1.5.2. 更新

`save` 方法还可以用于更新数据库中已存在的模型。要更新一个模型，应该先获取它，设置你想要更新的属性，然后调用 `save` 方法。

```
$flight = App\Flight::find(1);  
  
$flight->name = 'New Flight Name';  
  
$flight->save();
```

9.1.5.3. 批量赋值

9.1.5.4. 其他创建方法

还有其它两种可以用来创建模型的方法：`firstOrCreate` 和 `firstOrCreateNew`。

9.1.6. 删除模型

要删除一个模型，调用模型实例上的 `delete` 方法。

```
$flight = App\Flight::find(1);  
  
$flight->delete();
```

如果您知道模型的主键，可以调用 `destroy` 方法直接删除而不需要获取它。

9.1.6.1. 软删除

除了真实删除数据库记录，Eloquent 也可以「软删除」模型。软删除的模型并不是真的从数据库中删除了。事实上，是在模型上设置了 `deleted_at` 属性并将其值写入数据库。如果 `deleted_at` 值非空，代表这个模型已被软删除。如果要开启模型软删除功能，你需要在模型上使用

```
Illuminate\Database\Eloquent\SoftDeletes trait:
```

```
<?php
```

```
namespace App;
```



```
use Illuminate\Database\Eloquent\Model;

use Illuminate\Database\Eloquent\SoftDeletes;

class Flight extends Model
{
    use SoftDeletes;
}
```

9.1.6.2. 查询软删除模型

前面提到，查询结果会自动剔除已被软删除的结果。当然，你可以使用 `withTrashed` 方法来获取包括软删除模型在内的模型：

```
$flights = App\Flight::withTrashed()
    ->where('account_id', 1)
    ->get();
```

9.1.7. 复制模型

您可以使用 `replicate` 方法复制一个新的未保存到数据库的实例，当模型实例共享许多相同的属性时，这个方法非常好用。

```
$shipping = App\Address::create([
    'type' => 'shipping',
    'line_1' => '123 Example Street',
    'city' => 'Victorville',
    'state' => 'CA',
```

```
        'postcode' => '90001',
    ]);

    $billing = $shipping->replicate()->fill([
        'type' => 'billing'
    ]);

    $billing->save();
```

9.1.8. 查询作用域

9.1.8.1. 全局作用域

全局作用域允许我们为给定模型的所有查询添加条件约束。

9.1.8.2. 本地作用域

本地作用域允许我们定义通用的约束集合以便在应用中复用。

9.1.9. 比较模型

有时候您可能需要确定两个模型是否是一样的。

```
if ($post->is($anotherPost)) {
    //
}
```

9.1.10. 事件

Eloquent 模型触发几个事件，允许你挂接到模型生命周期的如下节点：
retrieved、creating、created、updating、updated、saving、saved、deleting、deleted、
restoring 和 restored。事件允许你每当特定模型保存或更新数据库时执行代码。
每个事件通过其构造器接受模型实例。

`retrieved` 事件在现有模型从数据库中查找数据时触发。当新模型每一次保存时，`creating` 和 `created` 事件被触发。如果数据库中已经存在模型并且调用了 `save` 方法，`updating` / `updated` 事件被触发。这些情况下，`saving` / `saved` 事件也被触发。

9.1.10.1.使用闭包

你可以注册在触发各种模型事件时执行的闭包，而不使用自定义事件类。通常，你应该在模型的 `booted` 方法中注册这些闭包：

```
<?php

namespace App;

use App\Scopes\AgeScope;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The "booted" method of the model.
     *
     * @return void
     */
    protected static function booted()
    {
        static::created(function ($user) {
```

```
        //

    });

}

}
```

9.1.10.2. 观察者

定义观察者

如果在一个模型上监听了多个事件，可以使用观察者来将这些监听器组织到一个单独的类中。观察者类的方法名映射到你希望监听的 **Eloquent** 事件。这些方法都以模型作为其唯一参数。`make:observer` Artisan 命令可以快速建立新的观察者类：

```
php artisan make:observer UserObserver --model=User
```

9.1.10.3. 禁用事件

9.2. 关联关系

9.2.1. 简介

数据表经常要与其它表做关联，比如一篇文章可能有很多评论，或者一个订单会被关联到下单用户，**Eloquent** 让组织和处理这些关联关系变得简单，并且支持多种不同类型的关联关系。

一对一

一对多

多对多

远程一对一

远程一对多

一对一（多态关联）

一对多（多态关联）

多对多（多态关联）

9.2.2. 定义关联关系

Eloquent 关联在 Eloquent 模型类中以方法的形式呈现。如同 Eloquent 模型本身，关联也可以作为强大的查询语句构造器使用，提供了强大的链式调用和查询功能。例如，我们可以在 `posts` 关联的链式调用中附加一个约束条件：

```
$user->posts()->where('active', 1)->get();
```

9.2.2.1. 一对一

一对一是最基本的关联关系。例如，一个 `User` 模型可能关联一个 `Phone` 模型。为了定义这个关联，我们要在 `User` 模型中写一个 `phone` 方法。在 `phone` 方法内部调用 `hasOne` 方法并返回其结果：

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * 获取与用户相关的电话记录。
     */
}
```

```
*/  
  
public function phone()  
{  
  
    return $this->hasOne('App\Phone');  
  
}  
  
}
```

9.2.2.2. 一对多

一对多关联用于定义单个模型拥有任意数量的其它关联模型。例如，一篇博客文章可能会有无限多条评论。正如其它所有的 Eloquent 关联一样，一对多关联的定义也是在 Eloquent 模型中写一个方法：

```
<?php  
  
namespace App;  
  
use Illuminate\Database\Eloquent\Model;  
  
class Post extends Model  
{  
  
    /**  
     * 获取博客文章的评论  
     */  
  
    public function comments()  
  
    {
```

```
        return $this->hasMany('App\Comment');
    }
}
```

9.2.2.3. 一对多(反向)

现在，我们已经能获得一篇文章的所有评论，接着再定义一个通过评论获得所属文章的关联关系。这个关联是 `hasMany` 关联的反向关联，需要在子级模型中使用 `belongsTo` 方法定义它：

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    /**
     * 获取此评论所属文章
     */
    public function post()
    {
        return $this->belongsTo('App\Post');
    }
}
```

9.2.2.4. 多对多

多对多关联比 `hasOne` 和 `hasMany` 关联稍微复杂些。举个例子，一个用户可以拥有很多种角色，同时这些角色也被其他用户共享。例如，许多用户可能都有「管理员」这个角色。

9.2.3. 多态关系

多态关联允许目标模型借助单个关联从属于多个模型。

9.2.3.1. 一对一(多态)

表结构

一对一多态关联与简单的一对一关联类似；不过，目标模型能够在一个关联上从属于多个模型。例如，博客 `Post` 和 `User` 可能共享一个关联到 `Image` 模型的关系。使用一对一多态关联允许使用一个唯一图片列表同时用于博客文章和用户账户。

9.2.3.2. 一对多(多态)

一对多多态关联与简单的一对多关联类似；不过，目标模型可以在一个关联中从属于多个模型。假设应用中的用户可以同时「评论」文章和视频。使用多态关联，可以用单个 `comments` 表同时满足这些情况。

9.2.3.3. 多对多(多态)

多对多多态关联比 `morphOne` 和 `morphMany` 关联略微复杂一些。例如，博客 `Post` 和 `Video` 模型能够共享关联到 `Tag` 模型的多态关系。使用多对多多态关联允许使用一个唯一标签在博客文章和视频间共享。

9.2.4. 关联查询

9.2.4.1. 关联方法 Vs 动态属性

如果不需要向 Eloquent 关联查询添加额外的约束，可以像属性一样访问关联。例如，继续使用 `User` 和 `Post` 示例模型，可以这样访问用户的全部文章。

```
$user = App\User::find(1);
```



```
foreach ($user->posts as $post) {  
  
    //  
  
}
```

9.2.4.2. 查询已存在的关联

在访问模型记录时，可能希望基于关联的存在限制查询结果。比如想要获取至少存在一条评论的所有文章，可以通过给 `has` 和 `orHas` 方法传递关联名称来实现：

```
// 获取至少存在一条评论的所有文章...  
  
$posts = App\Post::has('comments')->get();
```

9.2.4.3. 查询不存在的关联

在访问模型记录时，可能希望基于关联不存在来限制查询结果。假设想要获取不存在任何评论的文章，可以通过向 `doesn'tHave` 和 `orDoesntHave` 方法传递关联名称来实现：

```
$posts = App\Post::doesn'tHave('comments')->get();
```

9.3. 集合

9.3.1. 简介

Eloquent 返回的所有结果集都是 `Illuminate\Database\Eloquent\Collection` 对象的实例，包括通过 `get` 方法检索或通过访问关联关系获取到的结果。Eloquent 的集合对象继承了 Laravel 的集合基类，因此它自然也继承了数十种能优雅地处理 Eloquent 模型底层数组的方法。

而且，所有的集合都可以作为迭代器，你可以像遍历简单的 PHP 数组一样来遍历它们：

```
$users = App\User::where('active', 1)->get();

foreach ($users as $user) {

    echo $user->name;

}
```

9.3.2. 可用方法

所有 Eloquent 的集合都继承了 Laravel Collection 对象；因此，他们也继承了所有集合基类提供的强大的方法。

另外，Illuminate\Database\Eloquent\Collection 类提供了一套上层的方法来帮你管理你的模型集合。大多数方法返回 Illuminate\Database\Eloquent\Collection 实例；然而，也会有一些方法返回基于 Illuminate\Support\Collection 类的实例。

9.3.3. 自定义集合

如果你需要使用自定义 Collection 对象与自己的扩展方法一起使用，可以在你的模型上重写 newCollection 方法：

```
<?php

namespace App;

use App\CustomCollection;

use Illuminate\Database\Eloquent\Model;

class User extends Model
```

```
{

    /**
     * 创建一个 Eloquent 集合实例.
     *
     * @param array $models
     * @return \Illuminate\Database\Eloquent\Collection
     */
    public function newCollection(array $models = [])
    {
        return new CustomCollection($models);
    }
}
```

9.4. 访问器和修改器

9.4.1. 简介

当你在 Eloquent 模型实例中获取或设置某些属性值的时候，访问器和修改器允许你对 Eloquent 属性值进行格式化。例如，你可能需要使用 Laravel 加密器来加密保存在数据库中的值，而在使用 Eloquent 模型访问该属性的时候自动进行解密其值。

除了自定义访问器和修改器外，Eloquent 也会自动将日期字段类型转换为 Carbon 实例，或将文本类型转换为 JSON。

9.4.2. 访问器&修改器

9.4.2.1. 定义一个访问器

若要定义一个访问器，则需在模型上创建一个 `getFooAttribute` 方法，要

访问的 `foo` 字段需使用「驼峰式」命名。在这个示例中，我们将为 `first_name` 属性定义一个访问器。当 Eloquent 尝试获取 `first_name` 属性时，将自动调用此访问器：

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * 获取用户的姓名.
     *
     * @param string $value
     * @return string
     */
    public function getFirstNameAttribute($value)
    {
        return ucfirst($value);
    }
}
```

9.4.2.2. 定义一个修改器

要定义一个修改器，则需在模型上面定义 `setFooAttribute` 方法。要访问的 `Foo` 字段使用「驼峰式」命名。让我们再来定义一个 `first_name` 属性的修改器。当我们尝试在模式上在设置 `first_name` 属性值时，该修改器将被自动调用：

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * 设置用户的姓名.
     *
     * @param string $value
     * @return void
     */
    public function setFirstNameAttribute($value)
    {
        $this->attributes['first_name'] = strtolower($value);
    }
}
```

9.4.3. 日期修改器

默认情况下，Eloquent 会将 `created_at` 和 `updated_at` 字段转换为 Carbon 实例，它继承了 PHP 原生的 `DateTime` 类并提供了各种有用的方法。你可以通过设置模型的 `$dates` 属性来添加其他日期属性：

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * 应该转换为日期格式的属性.
     *
     * @var array
     */
    protected $dates = [
        'seen_at',
    ];
}
```

9.4.4. 属性类型转换

模型中的 `$casts` 属性提供了一个便利的方法来将属性转换为常见的数据类

型。`$casts` 属性应是一个数组，且数组的键是那些需要被转换的属性名称，值则是你希望转换的数据类型。支持转换的数据类型有：`integer`, `real`, `float`, `double`, `decimal:<digits>`, `string`, `boolean`, `object`, `array`, `collection`, `date`, `datetime`, 和 `timestamp`。当需要转换为 `decimal` 类型时，你需要定义小数位的个数，如：`decimal:2`。

9.4.4.1. 自定义类型转换

Laravel 内置了多种常用的类型转换。但是，用户偶尔会需要将数据转换成自定义类型。现在，该需求可以通过定义一个实现 `CastsAttributes` 接口的类来完成。

实现了该接口的类必须事先定义一个 `get` 和 `set` 方法。

9.4.4.2. 数组&JSON 转换

当你在数据库存储序列化的 JSON 的数据时，`array` 类型的转换非常有用。比如：如果你的数据库具有被序列化为 JSON 的 JSON 或 TEXT 字段类型，并且在 Eloquent 模型中加入了 `array` 类型转换，那么当你访问的时候就会自动被转换为 PHP 数组。

9.4.4.3. Date 类型转换

当使用 `date` 或 `datetime` 属性时，可以指定日期的格式。

9.4.4.4. 查询时类型转换

有时候需要在查询执行过程中对特定属性进行类型转换，例如需要从数据库表中获取数据的时候。

9.5. API 资源类

9.5.1. 简介

当构建 API 时，你往往需要一个转换层来联结你的 Eloquent 模型和实际返回给用户的 JSON 响应。Laravel 的资源类能够让你以更直观简便的方式将模型和模型集合转化成 JSON。

9.5.2. 生成资源

你可以使用 `make:resource` Artisan 命令来生成一个资源类。默认情况下生成的资源都会被放置在应用程序的 `app/Http/Resources` 文件夹下。资源继承自 `Illuminate\Http\Resources\Json\JsonResource` 类：

```
php artisan make:resource User
```

9.5.3. 核心概念

在深入了解如何定制化编写你的资源之前，让我们先来看看在 Laravel 中如何使用资源。一个资源类表示一个单一模型需要被转换成 JSON 格式。

9.5.3.1. 资源集合

你可以在路由或者控制器中使用 `collection` 方法来创建资源实例，以返回多个资源的集合或分页响应：

```
use App\Http\Resources\User as UserResource;

use App\User;

Route::get('/user', function () {

    return UserResource::collection(User::all());

});
```

9.5.4. 编写资源

从本质上来说，资源的作用很简单。它们只需要将一个给定的模型转换成一个数组。所以每一个资源都包含一个 `toArray` 方法用来将你的模型属性转换成一个可以返回给用户的 API 友好数组。

9.5.4.1. 数据包裹

默认情况下，当资源响应被转换成 JSON 时，顶层资源将会被包裹在 `data`

键中。因此一个典型的资源集合响应如下所示：

```
{
  "data": [
    {
      "id": 1,
      "name": "Eladio Schroeder Sr.",
      "email": "therese28@example.com",
    },
    {
      "id": 2,
      "name": "Liliana Mayert",
      "email": "evandervort@example.com",
    }
  ]
}
```

9.5.4.2. 包裹嵌套资源

你可以完全自由地决定资源关联如何被包裹。如果你希望无论怎样嵌套，都将所有资源集合包裹在 `data` 键中，那么你需要为每个资源都定义一个资源集合类，并将返回的集合包裹在 `data` 键中。

当然，你可能会担心这样顶层资源将会被包裹在两个 `data` 键中。请放心，Laravel 将永远不会让你的资源被双层包裹，因此你不必担心被转换的资源集合会被多重嵌套：

```
<?php
```

```
namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\ResourceCollection;

class CommentsCollection extends ResourceCollection
{
    /**
     * 将资源集合转换成数组
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array
     */
    public function toArray($request)
    {
        return ['data' => $this->collection];
    }
}
```

9.5.4.3. 数据包裹和分页

当在资源响应中返回分页集合时，即使你调用了 `withoutWrapping` 方法，Laravel 也会将你的资源数据包裹在 `data` 键中。

9.5.4.4. 分页

你可以将分页实例传递给资源的 `collection` 方法或者自定义的资源集合：

```
use App\Http\Resources\UserCollection;

use App\User;

Route::get('/users', function () {

    return new UserCollection(User::paginate());

});
```

9.5.4.5. 条件属性

有些时候，你可能希望在给定条件满足时添加属性到资源响应里。例如，你可能希望如果当前用户是「管理员」时添加某个值到资源响应中。在这种情况下 Laravel 提供了一些辅助方法来帮助你解决问题。when 方法可以被用来有条件地向资源响应添加属性：

```
/**

 * 将资源转换成数组

 *

 * @param  \Illuminate\Http\Request  $request

 * @return array

 */

public function toArray($request)

{

    return [

        'id' => $this->id,

        'name' => $this->name,

        'email' => $this->email,
```

```
        'secret' => $this->when(Auth::user()->isAdmin(),
'secret-value'),

        'created_at' => $this->created_at,

        'updated_at' => $this->updated_at,

    ];
}
```

9.5.4.6. 条件关联

除了有条件地添加属性之外，你还可以根据模型关联是否已加载来有条件地在你的资源响应中包含关联。这允许你在控制器中决定加载哪些模型关联，这样你的资源可以在模型关联被加载后才添加它们。

9.5.4.7. 添加元数据

一些 JSON API 标准需要你在资源和资源集合响应中添加元数据。这通常包括资源或相关资源的 `links`，或一些关于资源本身的元数据。如果你需要返回有关资源的其他元数据，只需要将它们包含在 `toArray` 方法中即可。

9.5.5. 资源响应

就像你知道的那样，资源可以直接在路由和控制器中被返回：

```
use App\Http\Resources\User as UserResource;

use App\User;

Route::get('/user', function () {

    return new UserResource(User::find(1));

});
```

9.6. 序列化

9.6.1. 简介

当构建 JSON API 时，经常需要转化模型和关联关系为数组或 JSON。针对这些操作，Eloquent 提供了一些便捷方法，以及对序列化中的属性控制。

9.6.2. 序列化模型&集合

9.6.2.1. 序列化为数组

要转化模型及其加载的 关联 为数组，可以使用 `toArray` 方法。这是一个递归的方法，因此所有的属性和关联（包括关联的关联）都将转化成数组：

```
$user = App\User::with('roles')->first();  
  
return $user->toArray();
```

9.6.2.2. 序列化为 JSON

方法 `toJson` 可以把模型转化成 JSON。和方法 `toArray` 一样，`toJson` 方法也是递归的，因此所有属性和关联都会转化成 JSON，你还可以指定由 PHP 支持的 JSON 编码选项：

```
$user = App\User::find(1);  
  
return $user->toJson();  
  
return $user->toJson(JSON_PRETTY_PRINT);
```

9.6.3. 在 JSON 中隐藏属性

有时要将模型数组或 JSON 中的某些属性进行隐藏，比如密码，则可以在模型中添加 `$hidden` 属性：

```
<?php
```

```
namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * 数组中的属性会被隐藏
     *
     * @var array
     */
    protected $hidden = ['password'];
}
```

9.6.4. 追加值到 JSON

有时，需要在数组或 JSON 中添加一些数据库中不存在字段的对应属性。要实现这个功能，首先要定义一个 修改器。

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
```

```
{

    /**
     * 为用户获取管理员标识。
     *
     * @return bool
     */
    public function getIsAdminAttribute()
    {
        return $this->attributes['admin'] === 'yes';
    }
}
```

9.6.5. 日期序列化

自定义默认日期格式

你可以通过覆盖 `serializeDate` 方法自定义默认的序列化格式:

```
/**
 * 为数组/ JSON 序列化准备一个日期。
 *
 * @param \DateTimeInterface $date
 * @return string
 */
protected function serializeDate(DateTimeInterface $date)
{
    return $date->format('Y-m-d');
}
```

```
}
```

第十章 测试系列

10.1. 快速入门

10.1.1. 简介

内置的 PHPUnit 对测试提供支持是开箱即用的。Laravel 默认就支持用 PHPUnit 来做测试，并为你的应用程序配置好了 `phpunit.xml` 文件。框架还提供了一些便利的辅助函数，让你可以更直观的测试你的应用程序。

默认情况，你的应用 `tests` 目录中包含两个子目录：`Feature` 和 `Unit`。单元测试是针对你的代码中非常少，而且相对独立的一部分代码来进行的测试。实际上，大部分单元测试都是针对单个方法进行的。功能测试是针对大面积代码进行的测试，包括多个对象之间的交互，甚至是对 JSON 端点的完整 HTTP 请求。

`Feature` 和 `Unit` 目录中都提供一个 `ExampleTest.php` 测试示例文件。安装一个新的 Laravel 应用程序之后，在命令行下运行 `vendor/bin/phpunit` 命令，即可运行测试。

10.1.2. 环境

在使用 `phpunit` 进行测试时，Laravel 将根据 `phpunit.xml` 文件设定的环境变量自动将环境设置为 `testing`，并将 Session 及缓存以 `array` 的形式存储，也就是说在测试时不会持久化任何 Session 或缓存数据。

你可以随意创建其它必要的测试环境配置。`testing` 环境变量可以在 `phpunit.xml` 文件中修改，但是在运行测试之前，请确保使用 `config:clear` Artisan 命令来清除配置信息的缓存！

此外，你还可以在你的项目根目录下创建一个 `.env.testing` 文件，在运行单元测试或使用带有 `--env=testing` 选项的 Artisan 命令时，`.env` 文件中的变量会被这个文件覆盖。

10.1.3. 创建并运行测试

可以使用 `Artisan` 命令 `make:test` 创建一个新的测试用例：

```
// 在 Feature 目录下创建一个测试类...

php artisan make:test UserTest

// 在 Unit 目录下创建一个测试类...

php artisan make:test UserTest --unit
```

10.1.3.1. Artisan 测试运行器

除了 `phpunit` 命令之外，您还可以使用 `test Artisan` 命令来运行你的测试。`Artisan` 测试运行器提供了关于当前正在运行的测试的更多信息，并将在第一次测试失败时自动停止：

```
php artisan test
```

10.2. HTTP 测试

10.2.1. 简介

`Laravel` 提供了一个非常流畅的 `API`，用于向您的应用程序发出 `HTTP` 请求并检查输出。例如，看一下下面定义的功能测试：

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
```

```
use Tests\TestCase;

class ExampleTest extends TestCase
{
    /**
     * 一个基本的测试示例。
     *
     * @return void
     */
    public function testBasicTest()
    {
        $response = $this->get('/');

        $response->assertStatus(200);
    }
}
```

10.2.1.1. 自定义请求头

您可以使用此 `withHeaders` 方法自定义请求的标头，然后再将其发送到应用程序。这使您可以将任何想要的自定义标头添加到请求中：

```
<?php

class ExampleTest extends TestCase
{
```

```
/**
 * 一个基本的功能测试示例。
 *
 * @return void
 */
public function testBasicExample()
{
    $response = $this->withHeaders([
        'X-Header' => 'Value',
    ])->json('POST', '/user', ['name' => 'Sally']);

    $response
        ->assertStatus(201)
        ->assertJson([
            'created' => true,
        ]);
}
}
```

10.2.1.2.测试响应

测试请求通过应用程序后，`dump` 和 `dumpHeaders` 方法将会在控制台打印响应体和响应头，可以用来分析和调试响应：

```
<?php
```

```
namespace Tests\Feature;

use Tests\TestCase;

use Illuminate\Foundation\Testing\RefreshDatabase;

use Illuminate\Foundation\Testing\WithoutMiddleware;

class ExampleTest extends TestCase
{
    /**
     * A basic test example.
     *
     * @return void
     */
    public function testBasicTest()
    {
        $response = $this->get('/');

        $response->dumpHeaders();

        $response->dump();
    }
}
```

10.2.2. Session/认证

Laravel 提供了几个可在 HTTP 测试时使用 Session 的辅助函数。首先，你需要传递一个数组给 `withSession` 方法来设置 Session 数据。这让你在应用程序的测试请求发送之前，先给数据加载 Session 变得简单：

```
<?php

class ExampleTest extends TestCase
{
    public function testApplication()
    {
        $response = $this->withSession(['foo' => 'bar'])
            ->get('/');
    }
}
```

10.2.3. 测试 JSON APIs

Laravel 也提供了几个辅助函数来测试 JSON APIs 和他们的响应。例如，`json`，`get`，`post`，`put`，`patch`，和 `delete` 可以被用于发送各种 HTTP 动作。你也可以轻松地将数据和请求头传递到这些方法中。

```
<?php

class ExampleTest extends TestCase
{
    /**
```

```
* A basic functional test example.

*

* @return void

*/

public function testBasicExample()

{

    $response = $this->json('POST', '/user', ['name' => 'Sally']);

    $response

        ->assertStatus(201)

        ->assertJson([

            'created' => true,

        ]);

}

}
```

10.2.3.1. 验证 JSON 完全匹配

如果你想验证给定的数组 完全 匹配应用返回的 JSON 结果，你应该使用 `assertExactJson` 方法：

```
<?php

class ExampleTest extends TestCase

{

    /**
```

```
* A basic functional test example.

*

* @return void

*/

public function testBasicExample()

{

    $response = $this->json('POST', '/user', ['name' => 'Sally']);

    $response

        ->assertStatus(201)

        ->assertExactJson([

            'created' => true,

        ]);

}

}
```

10.2.3.2. 验证 JSON 路径

如果你想验证 JSON 响应是否包含指定路径上的某些给定数据，则应使用 `assertJsonPath` 方法：

```
<?php

class ExampleTest extends TestCase

{

    /**
```

```
* A basic functional test example.

*

* @return void

*/

public function testBasicExample()

{

    $response = $this->json('POST', '/user', ['name' => 'Sally']);

    $response

        ->assertStatus(201)

        ->assertJsonPath('team.owner.name', 'foo')

}

}
```

10.2.4. 测试文件上传

`Illuminate\Http\UploadedFile` 提供了一个 `fake` 方法用于生成虚拟的文件或者图像以供测试之用。它可以和 `Storage facade` 的 `fake` 方法相结合，大幅度简化了文件上传测试。举个例子，你可以结合这两者的功能非常方便地进行头像上传表单测试：

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
```



```
use Illuminate\Foundation\Testing\WithoutMiddleware;

use Illuminate\Http\UploadedFile;

use Illuminate\Support\Facades\Storage;

use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testAvatarUpload()
    {
        Storage::fake('avatars');

        $file = UploadedFile::fake()->image('avatar.jpg');

        $response = $this->json('POST', '/avatar', [
            'avatar' => $file,
        ]);

        // Assert the file was stored...
        Storage::disk('avatars')->assertExists($file->hashName());

        // Assert a file does not exist...
        Storage::disk('avatars')->assertMissing('missing.jpg');
    }
}
```

```
}
```

10.3. 控制台测试

10.3.1. 简介

除了简化 HTTP 测试之外，Laravel 为测试用户输入的控制台应用提供了简单的 API。

10.3.2. 期望输入/输出

Laravel 使用 `expectsQuestion` 方法为控制台命令「模拟」用户输入。此外，还可以使用 `assertExitCode` 和 `expectsOutput` 方法指定控制台命令退出码和期望输出文本。举例如下：

```
Artisan::command('question', function () {

    $name = $this->ask('What is your name?');

    $language = $this->choice('Which language do you program in?', [

        'PHP',

        'Ruby',

        'Python',

    ]);

    $this->line('Your name is ' . $name . ' and you program in ' . $language . '.');

});
```

10.4. 数据库测试

10.4.1. 简介

Laravel 提供了各种有用的工具，可以更轻松地测试数据库驱动的应用程序。首先，你可以使用 `assertDatabaseHas` 辅助函数，来断言数据库中是否存在与指定条件互相匹配的数据。例如，如果我们想要验证 `users` 表中是否存在 `email` 值为 `sally@example.com` 的数据，你可以执行以下操作：

```
public function testDatabase()
{
    // 调用应该程序...

    $this->assertDatabaseHas('users', [
        'email' => 'sally@example.com',
    ]);
}
```

10.4.2. 生成模型工厂

使用 `make:factory Artisan command` 命令可以创建一个模型工厂：

```
php artisan make:factory PostFactory
```

10.4.3. 每次测试后重置数据库

在每次测试后重置数据库是很有用的，这样前一次测试的数据不会干扰后续测试。`RefreshDatabase trait` 会采用最优化的方法来迁移测试数据库，这取决于你使用的是内存数据库还是传统数据库。

10.4.4. 编写模型工厂

进行测试时，运行测试之前常常需要插入一些数据到数据库中。当你创建测试数据时，除了手动设置每个字段的值，Laravel 还可以使用 Eloquent 模型的工厂来设置每个属性的默认值。在开始之前，你可以先看一下应用程序 `database/factories/UserFactory.php` 的文件。开箱即用，这个文件包含一个模型工厂定义：

```
use Faker\Generator as Faker;

use Illuminate\Support\Str;

$factory->define(App\User::class, function (Faker $faker) {
    return [
        'name' => $faker->name,
        'email' => $faker->unique()->safeEmail,
        'email_verified_at' => now(),
        'password' =>
'$2y$10$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', //
password
        'remember_token' => Str::random(10),
    ];
});
```

10.4.4.1. 工厂扩展

如果扩展了模型，则可能还希望扩展其工厂，以便在测试和数据填充期间利用子模型的工厂属性。为此，你可以调用工厂制造商的 `raw` 方法来从任何给定工厂获得原始属性数组。

```
$factory->define(App\Admin::class, function (Faker\Generator $faker) {  
  
    return factory(App\User::class)->raw([  
  
        // ...  
  
    ]);  
  
});
```

10.4.4.2.工厂状态

工厂状态可以让你任意组合你的模型工厂，仅需要做出适当差异化的修改，就可以达到让模型拥有多种不同的状态。例如，你的 `User` 模型中可以修改某个默认属性值来达到标识一种 `delinquent` 状态。你可以使用 `state` 方法来进行这种状态转换。对于简单的工厂状态，你可以传入要修改的属性数组：

```
$factory->state(App\User::class, 'delinquent', [  
  
    'account_status' => 'delinquent',  
  
]);
```

10.4.4.3.工厂回调

工厂回调是使用 `afterMaking` 和 `afterCreating` 方法注册的，并且允许你在创建模型之后执行其他任务。例如，可以使用回调将附加模型与创建的模型相关联：

```
$factory->afterMaking(App\User::class, function ($user, $faker) {  
  
    // ...  
  
});  
  
$factory->afterCreating(App\User::class, function ($user, $faker) {  
  
    $user->accounts()->save(factory(App\Account::class)->make());  
  
});
```

```
});
```

10.5. 模拟器

10.5.1. 简介

在 Laravel 应用程序测试中，你可能希望「模拟」应用程序的某些功能的行为，从而避免该部分在测试中真正执行。例如：在控制器执行过程中会触发事件，从而避免该事件在测试控制器时真正执行。这允许你在仅测试控制器 HTTP 响应的情况时，而不必担心触发事件。当然，你也可以在单独的测试中测试该事件逻辑。

Laravel 针对事件、任务和 Facades 的模拟，提供了开箱即用的辅助函数。这些函数基于 **Mocker** 封装而成，使用非常方便，无需手动调用复杂的 **Mockery** 函数。当然你也可以使用 **Mockery** 或者使用 **PHPUnit** 创建自己的模拟器。

10.5.2. 模拟对象

当模拟一个对象将通过 Laravel 的服务容器注入到应用中时，你将需要将模拟实例作为 **instance** 绑定到容器中。这将告诉容器使用对象的模拟实例，而不是构造对象的真身：

```
use App\Service;

use Mockery;

$this->instance(Service::class, Mockery::mock(Service::class, function
($mock) {

    $mock->shouldReceive('process')->once();

})));
```

10.5.3. 任务模拟

作为模拟的替代方式,你可以使用 `Bus Facade` 的 `fake` 方法来防止任务被真正分发执行。

10.5.4. 事件模拟

作为 `mock` 的替代方法,你可以使用 `Event Facade` 的 `fake` 方法来模拟事件监听,测试的时候并不会真正触发事件监听器。然后你就可以测试断言事件运行了,甚至可以检查他们接收的数据。

10.5.5. 邮件模拟

你可以是用 `Mail Facade` 的 `fake` 方法来模拟邮件发送,测试时不会真的发送邮件,然后你可以断言 `mailables` 发送给了用户,甚至可以检查他们收到的内容。使用 `fakes` 时,断言一般放在测试代码的后面。

10.5.6. 通知模拟

可以使用 `Notification Facade` 的 `fake` 方法来模拟通知的发送,测试时并不会真的发出通知。然后你可以断言 `notifications` 发送给了用户,甚至可以检查他们收到的内容。使用 `fakes` 时,断言一般放在测试代码后面。

第十一章 附录：官方扩展包

11.1. Horizon 队列管理

11.1.1. 简介

`Horizon` 为你的 `Laravel Redis` 队列提供了一个美观的控制面板和代码驱动的配置;它可以方便的监控队列系统的关键指标:任务吞吐量、运行时间、工作故障。

所有的 `worker` 配置存储在一个简单的配置文件中,你可以在整个团队都可以进行协作的地方进行源码控制。

11.1.2. 安装

可以使用 Composer 将 Horizon 安装到你的 Laravel 项目里：

```
composer require laravel/horizon
```

11.1.2.1. 配置

Horizon 资源发布之后，其主要配置文件会被分配到 `config/horizon.php` 文件。可以用这个配置文件配置工作选项，每个配置选项包含一个用途描述，请务必仔细研究这个文件。

11.1.2.2. 控制面板授权

Horizon 在 `/horizon` 上显示了一个控制面板。默认情况下，你只能在 `local` 环境中访问这个面板。在你的 `app/Providers/HorizonServiceProvider.php` 文件中，有一个 `gate` 方法。控制着在非本地环境中对 Horizon 的访问。

11.1.3. 升级 Horizon

当你升级到 Horizon 的一个新的主要版本时，你需要仔细阅读 升级指南。

此外，升级到新的 Horizon 版本时，你应该重新发布 Horizon 资源：

```
php artisan horizon:publish
```

11.1.4. 运行 Horizon

在 `config/horizon.php` 中配置了你的 `workers` 之后，你可以使用 `horizon Artisan` 命令启动 Horizon。只需这一个命令你就可以启动你的所有已配置的 `workers`：

```
php artisan horizon
```

11.1.4.1. 部署 Horizon

如果要将 **Horizon** 部署到一个正在运行的服务器上, 应该配置一个进程监视器来监视 `php artisan horizon` 命令, 并在它意外退出时重新启动它。在将新代码部署到服务器时, 你需要终止 **Horizon** 主进程, 以便进程监视器重新启动它并接收代码的更改。

11.1.5. 标记

Horizon 允许你将 `tags` 分配给任务, 包括邮件、事件广播、通知和排队的事件监听器。实际上, **Horizon** 会根据附加到作业上的有 **Eloquent** 模型, 智能地、自动地标记大多数任务。

11.1.6. 通知

如果你希望在一个队列有较长的等待时间时得到通知, 你可以使用。

`Horizon::routeMailNotificationsTo,`

`Horizon::routeSlackNotificationsTo,` 和 `Horizon::routeSmsNotificationsTo` 方法。你可以从你的应用程序的 `HorizonServiceProvider` 调用这些方法:

```
Horizon::routeMailNotificationsTo('example@example.com');  
Horizon::routeSlackNotificationsTo('slack-webhook-url', '#channel');  
Horizon::routeSmsNotificationsTo('15556667777');
```

11.1.7. 指标

Horizon 有一个指标控制面板, 它提供了任务和队列的等待时间和吞吐量等信息。要让这些信息显示在这个控制面板上, 你应该配置 **Horizon** 的 `snapshot` **Artisan** 命令, 通过你的应用程序的 调度器 每五分钟运行一次:

```
/**  
  
 * 定义应用程序的命令调度  
  
 *  
  
 * @param \Illuminate\Console\Scheduling\Schedule $schedule
```

```
* @return void

*/

protected function schedule(Schedule $schedule)
{
    $schedule->command('horizon:snapshot')->everyFiveMinutes();
}
```

11.2. Sanctum 认证

11.2.1. 简介

Laravel Sanctum 为 SPA (单页面应用程序)、移动应用程序和简单的、基于令牌的 API 提供轻量级身份验证系统。Sanctum 允许应用程序的每个用户为他们的帐户生成多个 API 令牌。这些令牌可能被授予指定允许令牌执行哪些操作的能力 / 范围。

11.2.2. 安装

你可以通过 Composer 安装 Laravel Sanctum

```
composer require laravel/sanctum
```

11.2.3. API 令牌认证

11.2.3.1. 发行 API 令牌

以使用 Sanctum 发行 API 令牌 / 个人访问令牌 对你的 API 请求进行认证。当使用 API 令牌进行请求的时，令牌可以以 Bearer 的形式包含在 Authorization header 头里。

给用户发行令牌的时候，User 模型里应该使用 HasApiTokens trait:

```
use Laravel\Sanctum\HasApiTokens;
```

```
class User extends Authenticatable
{
    use HasApiTokens, Notifiable;
}
```

11.2.3.2. 令牌能力

Sanctum 可以为令牌分配 “abilities”，类似于 OAuth 的 “scopes”。可以将字符串能力数组作为第二个参数传递给 `createToken` 方法：

```
return $user->createToken(' token-name',
['server:update'])->plainTextToken;
```

11.2.3.3. 保护路由

为了保护路由，所有进来的请求都必须进行认证，应该将 Sanctum 认证守卫附加到 `routes/api.php` 的 API 路由里。如果一个请求是来自第三方的请求，这个守卫会确保进来的请求既是一个你的 SPA 应用的有状态的已认证请求，也是一个包含了有效令牌头的已认证请求：

```
Route::middleware('auth:sanctum')->get('/user', function (Request
$request) {
    return $request->user();
});
```

11.2.3.4. 撤销令牌

我们可以使用 `HasApiTokens` trait 提供的 `tokens` 关联关系从数据库删除它们，以达到撤销令牌的目的：

```
// Revoke all tokens...

$user->tokens()->delete();

// Revoke a specific token...

$user->tokens()->where('id', $id)->delete();
```

第十二章 补充说明

以上从官方文档翻译而来。

